

v2

# Threat Hunting Professional

## Suspicious Traffic Hunting

Section 02 | Module 02

© Caendra Inc. 2020  
All Rights Reserved

# Table of Contents

## MODULE 02 | SUSPICIOUS TRAFFIC HUNTING

2.1 Introduction

2.2 ARP Traffic

2.3 ICMP Traffic

2.4 TCP Traffic

2.5 DHCP Traffic

2.6 DNS Traffic

2.7 HTTP & HTTPS Traffic

2.8 Unknown Traffic

2.9 More Hunting Tools



# Introduction



## 2.1 Introduction

In this module, we'll cover how to recognize normal network traffic when analyzing packets.

It is important to understand what is normal so you can identify what is not normal, such as a connection to port 443 where the traffic is in cleartext for example.



## 2.1 Introduction

It is not feasible to sit at a workstation and look at hundreds of thousands of packets a day. We should have various enterprise products, even open source tools, that will aid us in this task and hopefully catch a lot of malicious traffic should it traverse our hunting grounds.

The goal of this module is to train your eye should you ever have to inspect network traffic, whether it's live traffic or saved traffic (PCAP).

# ARP Traffic



## 2.2.1 ARP

# We'll begin with the ARP protocol.

Again, even though we should be familiar with these protocols, we'll just briefly explain each protocol before analyzing traffic.



## 2.2.1 ARP

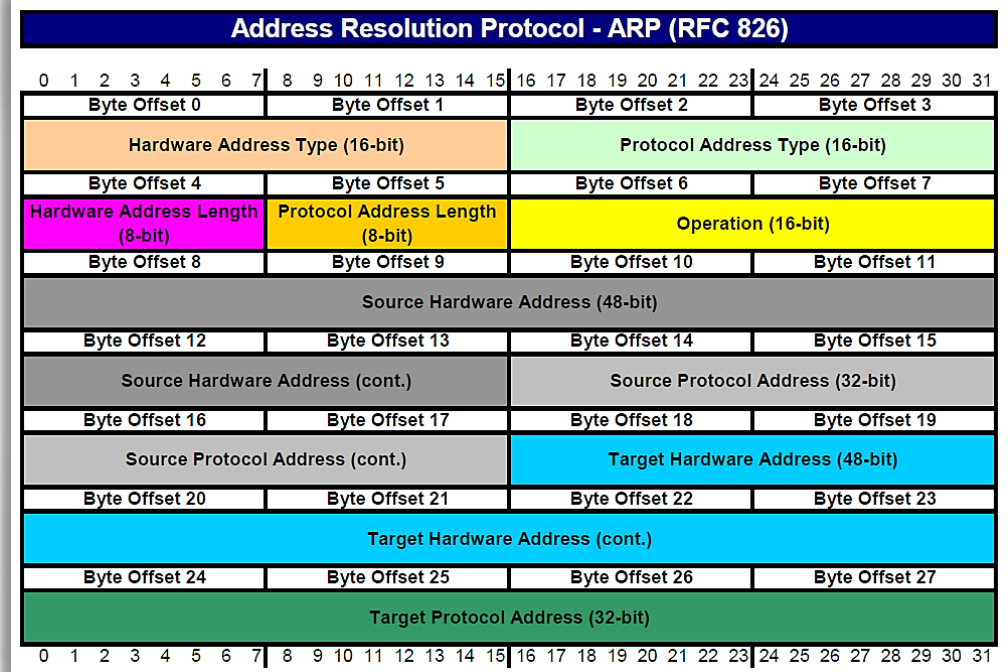
**ARP (Address Resolution Protocol)** is a Layer 2 protocol which is used by IP to map IP addresses to MAC addresses. ARP messages typically involve a **REQUEST message** and a **RESPONSE message**. The message header is completed with one of the following operation codes:

- Request (1)
- Reply (2)

You can read more about ARP in [RFC 826](https://www.ietf.org/rfc/rfc826.txt).

## 2.2.1 ARP

The ARP packet structure is displayed on the image to the right.



## 2.2.1 ARP

### Some facts to help distinguish normal and suspicious ARP traffic.

| Normal ARP Traffic                                                                                                                                                           | Suspicious ARP Traffic                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| ARP broadcasts are normal from both clients and servers, including network devices at a reasonable flow.                                                                     | Tens, hundreds, or even thousands of ARP broadcast messages in a small amount of time. |
| ARP Request typically follows a response, but it depends on who is making the request. A network device might send many ARP broadcasts into the network for various reasons. | Two identical MAC addresses in the network with different IP addresses.                |
| Legitimate gratuitous ARP packets.                                                                                                                                           | Gratuitous ARP packets sent by an attacker.                                            |



## 2.2.1 ARP

An attacker can manipulate other hosts' ARP cache tables by sending **gratuitous ARP replies**. Gratuitous ARP replies are unsolicited ARP Reply messages.

In other words, the attacker can send an ARP Reply without waiting for a host to perform an ARP Request. The ARP cache table can then contain fake information and the attacker can send gratuitous ARP replies every 30 seconds or so to prevent the poisoned entry from expiring.

## 2.2.1.1 Normal ARP

Below is a snapshot of 2 packets: 1 ARP Request & 1 ARP Reply.

| arp |          |                   |                   |          |        |                                        |
|-----|----------|-------------------|-------------------|----------|--------|----------------------------------------|
| No. | Time     | Source            | Destination       | Protocol | Length | Info                                   |
| 11  | 5.166850 | 26:11:59:88:53:02 | Vmware_a1:f4:d0   | ARP      | 42     | Who has 10.54.15.68? Tell 10.54.15.100 |
| 12  | 5.215241 | Vmware_a1:f4:d0   | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:f4:d0    |

To see the MAC Address for both the source and destination, we can make a quick change within Wireshark: **View > Name Resolution > Resolve Physical Addresses.**

| arp |          |                   |                   |          |        |                                        |
|-----|----------|-------------------|-------------------|----------|--------|----------------------------------------|
| No. | Time     | Source            | Destination       | Protocol | Length | Info                                   |
| 11  | 5.166850 | 26:11:59:88:53:02 | 00:50:56:a1:f4:d0 | ARP      | 42     | Who has 10.54.15.68? Tell 10.54.15.100 |
| 12  | 5.215241 | 00:50:56:a1:f4:d0 | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:f4:d0    |

## 2.2.1.1 Normal ARP

Here we see an **ARP Request** packet. We know it's an ARP Request by the Opcode, **request (1)**, in the highlighted line.

We can see that the device at 10.54.15.100 needs the MAC address for 10.54.15.68 to begin to establish communication with it.

If the device at 10.54.15.100 already knew that the MAC address for 10.54.15.68, then it would be contained within it's ARP table (**ARP** –a from the command line).

```
▶ Frame 11: 42 bytes on wire (336 bits), 42 bytes captured (336 bits)
▶ Ethernet II, Src: 26:11:59:88:53:02, Dst: 00:50:56:a1:f4:d0
◄ Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: 26:11:59:88:53:02
  Sender IP address: 10.54.15.100
  Target MAC address: 00:00:00:00:00:00
  Target IP address: 10.54.15.68
```

|      |                                                 |                  |
|------|-------------------------------------------------|------------------|
| 0000 | 00 50 56 a1 f4 d0 26 11 59 88 53 02 08 06 00 01 | .PV...&. Y.S.... |
| 0010 | 08 00 06 04 00 01 26 11 59 88 53 02 0a 36 0f 64 | ....&. Y.S..6.d  |
| 0020 | 00 00 00 00 00 00 0a 36 0f 44                   | .....6 .D        |

## 2.2.1.1 Normal ARP

In this packet, we have the reply to the previous packet.

This packet is the **ARP Reply** packet. We can quickly tell by the Opcode, **reply (2)**. Within this packet, we see that the Sender MAC address has been populated with the MAC address of the device at 10.54.15.68. This MAC address will be added to the ARP table of 10.54.15.100.

```
▶ Frame 12: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)
▶ Ethernet II, Src: 00:50:56:a1:f4:d0, Dst: 26:11:59:88:53:02
▶ Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: 00:50:56:a1:f4:d0
  Sender IP address: 10.54.15.68
  Target MAC address: 26:11:59:88:53:02
  Target IP address: 10.54.15.100
```

|      |                                                 |                 |
|------|-------------------------------------------------|-----------------|
| 0000 | 26 11 59 88 53 02 00 50 56 a1 f4 d0 08 06 00 01 | &.Y.S..P V..... |
| 0010 | 08 00 06 04 00 02 00 50 56 a1 f4 d0 0a 36 0f 44 | ....P V....6.D  |
| 0020 | 26 11 59 88 53 02 0a 36 0f 64 00 00 00 00 00 00 | &.Y.S.. .d..... |
| 0030 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....           |



## 2.2.1.1 Normal ARP

```
▷ Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 3
▷ Ethernet II, Src: 00:15:5d:0f:49:18, Dst: ff:ff:ff:ff:ff:ff
  Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: 00:15:5d:0f:49:18
    Sender IP address: 172.16.2.3
    Target MAC address: 00:00:00:00:00:00
    Target IP address: 172.16.2.27
```

```
0000  ff ff ff ff ff ff 00 15 5d 0f 49 18 08 06 00 01 ..... ]
0010  08 00 06 04 00 01 00 15 5d 0f 49 18 ac 10 02 03 ..... ]
0020  00 00 00 00 00 00 ac 10 02 1b ..... ]
```

```
▷ Frame 2: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 3
▷ Ethernet II, Src: d4:be:d9:af:3e:4d, Dst: 00:15:5d:0f:49:18
  Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: d4:be:d9:af:3e:4f
    Sender IP address: 172.16.2.27
    Target MAC address: 00:15:5d:0f:49:18
    Target IP address: 172.16.2.3
```

```
0000  00 15 5d 0f 49 18 d4 be d9 af 3e 4d 08 06 00 01 ...].I... ..>M...
0010  08 00 06 04 00 02 d4 be d9 af 3e 4f ac 10 02 1b ..... ..>O...
0020  00 15 5d 0f 49 18 ac 10 02 03 00 00 00 00 00 00 ...].I... .....
0030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..... ..
```

## 2.2.1.2 Suspicious ARP

Now we'll look at an example of suspicious ARP traffic using Wireshark. Remember the tips regarding normal and suspicious ARP traffic:

- **Normal:** ARP broadcasts are normal from both clients and servers, including network devices at a reasonable flow.
- **Suspicious:** Tens, hundreds, or even thousands of ARP broadcast messages in a small amount of time.

## 2.2.1.2 Suspicious ARP

Here we see a snapshot of a packet capture. It shows 7 ARP Request packets sent via broadcast. According to the physical address name resolution within Wireshark, the source device seems to be a Cisco device and it is checking on the status of various devices on the network.

| No. | Time     | Source            | Destination | Protocol | Length | Info                                      |
|-----|----------|-------------------|-------------|----------|--------|-------------------------------------------|
| 1   | 0.000000 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.173.159? Tell 24.166.172.1 |
| 2   | 0.098594 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.172.141? Tell 24.166.172.1 |
| 3   | 0.110617 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.173.161? Tell 24.166.172.1 |
| 4   | 0.211791 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 65.28.78.76? Tell 65.28.78.1      |
| 5   | 0.216744 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.173.163? Tell 24.166.172.1 |
| 6   | 0.307909 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.175.123? Tell 24.166.172.1 |
| 7   | 0.330433 | Cisco251_af:f4:54 | Broadcast   | ARP      | 60     | Who has 24.166.173.165? Tell 24.166.172.1 |

\*PCAP from Wireshark Sample Captures [page](https://wiki.wireshark.org/SampleCaptures).

## 2.2.1.2 Suspicious ARP

How would we know if this is suspicious traffic or not? How do we know it's not just a configuration issue within the Cisco device or if it's normal behavior? Do you even have Cisco equipment on the network?

Based on your response to these questions and others, you'll know whether this needs to be looked into further. In our case, we'll assume its normal.

## 2.2.1.2 Suspicious ARP

How about this traffic? Would it be considered suspicious or normal?

| No. | Time         | Source            | Destination | Protocol | Length | Info                                  |
|-----|--------------|-------------------|-------------|----------|--------|---------------------------------------|
| 15  | 61.162590056 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.1? Tell 172.16.5.67  |
| 16  | 61.164533730 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.2? Tell 172.16.5.67  |
| 17  | 61.166589500 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.3? Tell 172.16.5.67  |
| 18  | 61.171696684 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.4? Tell 172.16.5.67  |
| 19  | 61.173595193 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.5? Tell 172.16.5.67  |
| 20  | 61.175482595 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.6? Tell 172.16.5.67  |
| 21  | 61.177434405 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.7? Tell 172.16.5.67  |
| 22  | 61.179428423 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.8? Tell 172.16.5.67  |
| 23  | 61.181401311 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.9? Tell 172.16.5.67  |
| 24  | 61.183387692 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.10? Tell 172.16.5.67 |
| 25  | 61.185470650 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.11? Tell 172.16.5.67 |
| 26  | 61.187379238 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.12? Tell 172.16.5.67 |
| 27  | 61.189625522 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.13? Tell 172.16.5.67 |
| 28  | 61.191455492 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.14? Tell 172.16.5.67 |
| 29  | 61.193387656 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.15? Tell 172.16.5.67 |
| 30  | 61.195423342 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.16? Tell 172.16.5.67 |
| 31  | 61.197387752 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.17? Tell 172.16.5.67 |
| 32  | 61.199389322 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.18? Tell 172.16.5.67 |
| 33  | 61.201395568 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.19? Tell 172.16.5.67 |
| 34  | 61.203388474 | b2:fe:ed:db:02:32 | Broadcast   | ARP      | 42     | Who has 172.16.5.20? Tell 172.16.5.67 |

## 2.2.1.2 Suspicious ARP

That is definitely suspicious traffic. Even if you are not familiar with what legitimate device can potentially have that MAC address on your network, we can tell something is odd just based on the flow and speed of the ARP Requests.

Starting at packet 15, the IP addresses increment by 1 and time intervals between packets are relatively small which indicates a scan is being conducted.

```
18 def initialize(experiment, observations = {}, control = nil)
19   @experiment = experiment
20   @observations = observations
21   @control = control
22   @candidates = observations[:control]
23   evaluate_candidates
24
25   freeze
26   end
27
28 # MAIN: the experiment's context
29 def context
30   @experiment.context
31
32   @experiment_name
33   @experiment_name
34
35   @check
36
37   @experiment/result.sub 1.1
```

## 2.2.1.2 Suspicious ARP

Organizations might block/prevent pinging machines within the network, but the recon phase is not foiled by this.

An attacker can use other means, such as an ARP scan, to get the information they are looking for, which is mapping the internal network.



## 2.2.1.2 Suspicious ARP

If your organization doesn't implement some sort of USB lockdown procedures, then it's really easy to pop in a USB drive and launch a myriad of tools, including penetration testing operating systems.

Going back to the MAC address, remember it's quite easy to change the MAC address on a particular host to make it look like a legitimate device, so we can't really rely on that either.

```
24 # @param host - the experiment host name to use
25 # @param observations - an array of Observations, or Observations
26 # @param control - the control observation
27
28 def initialize(experiment, observations = [], control = nil)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   freeze
36
37   @context =
38     {
39       experiment_name:
40         experiment.name
41       hostnames:
42         hostnames
43     }
44   @context.freeze
45   @context
46 end
```

## 2.2 ARP Traffic

We need to remember the basic behavior of ARP from a client standpoint, and from a network device standpoint, in order to know when to investigate and when to ignore ARP traffic.

There are other techniques in which ARP can be used for nefarious purposes, such as **ARP Poisoning (Man-In-The-Middle Attacks)**.

## 2.2.2 Wireshark Tip

Refer to the Wireshark Display Filter Reference for ARP, [here](https://www.wireshark.org/docs/dfref/a/arp.html), on more techniques to filter ARP traffic.

# ICMP Traffic



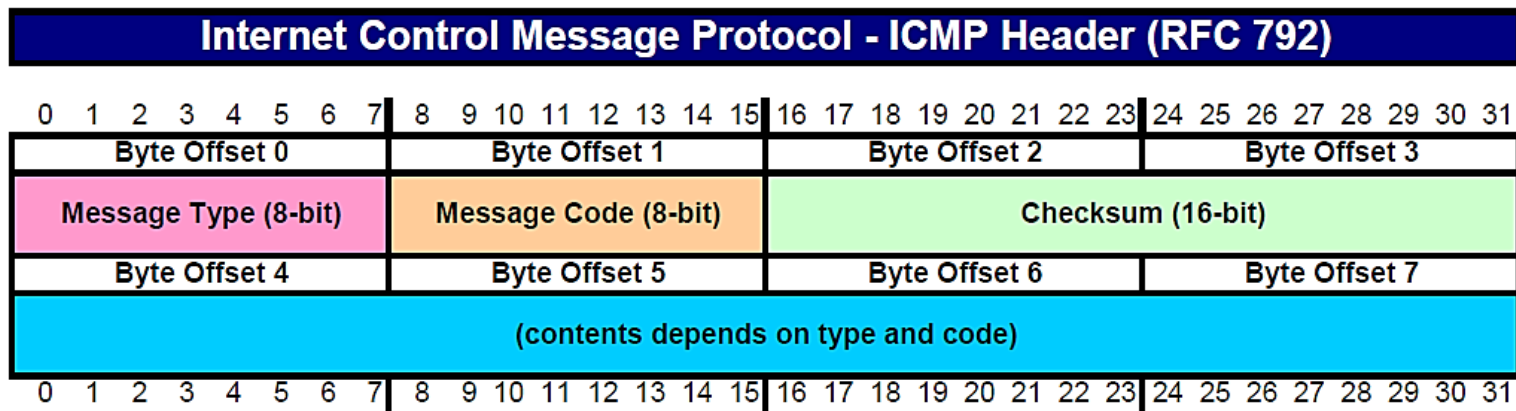
## 2.3.1 ICMP

**ICMP (Internet Control Message Protocol)** provides us information regarding different nodes on a network. It is typically used when troubleshooting. Utilities such as **ping** and **tracert** utilize ICMP.

You can read more about ICMP [here](#).

## 2.3.1 ICMP

Here is an example of an ICMP header.



## 2.3.1 ICMP

Below is a visualization depicting **ping**. We'll see what the packets look like in the upcoming slides.



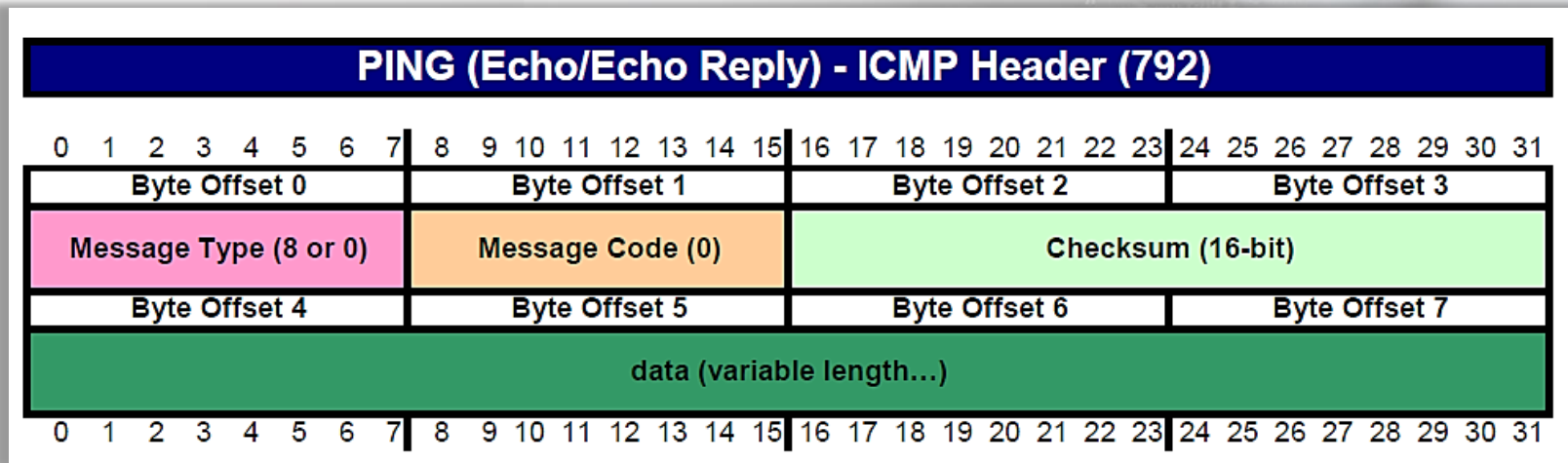
Echo (Ping) Request

Echo (Ping) Response



## 2.3.1 ICMP

Here we see a header for ping traffic.



## 2.3.1.1 Normal ICMP

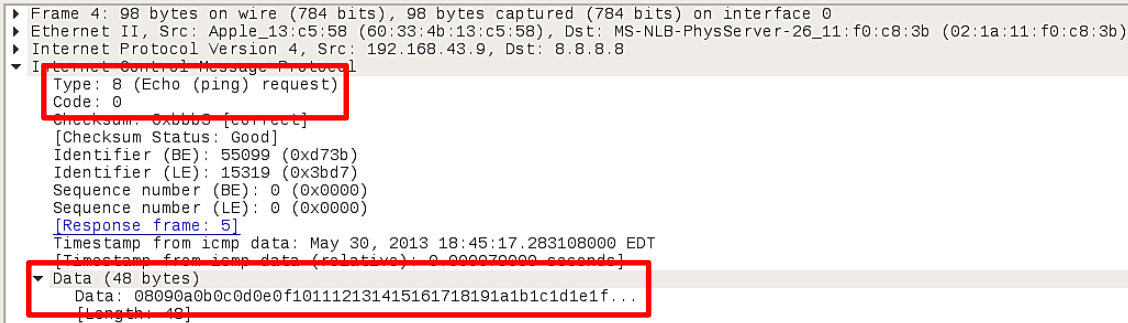
The packet capture below reflects **Echo (Ping) Requests** and **Echo (Ping) Replies**.

|   |          |              |              |      |                        |                                             |
|---|----------|--------------|--------------|------|------------------------|---------------------------------------------|
| 4 | 5.013334 | 192.168.43.9 | 8.8.8.8      | ICMP | 98 Echo (ping) request | id=0xd73b, seq=0/0, ttl=64 (reply in 5)     |
| 5 | 5.505538 | 8.8.8.8      | 192.168.43.9 | ICMP | 98 Echo (ping) reply   | id=0xd73b, seq=0/0, ttl=40 (request in 4)   |
| 6 | 6.019290 | 192.168.43.9 | 8.8.8.8      | ICMP | 98 Echo (ping) request | id=0xd73b, seq=1/256, ttl=64 (reply in 7)   |
| 7 | 6.153653 | 8.8.8.8      | 192.168.43.9 | ICMP | 98 Echo (ping) reply   | id=0xd73b, seq=1/256, ttl=40 (request in 6) |
| 8 | 7.015108 | 192.168.43.9 | 8.8.8.8      | ICMP | 98 Echo (ping) request | id=0xd73b, seq=2/512, ttl=64 (reply in 9)   |
| 9 | 7.781987 | 8.8.8.8      | 192.168.43.9 | ICMP | 98 Echo (ping) reply   | id=0xd73b, seq=2/512, ttl=40 (request in 8) |

\*PCAP from Wireshark Sample Captures [page](#).

## 2.3.1.1 Normal ICMP

Below we see packet details for the Echo (Ping) Request packet, packet 4.



The image shows a Wireshark packet capture window. The selected packet is 'Frame 4: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0'. The packet is an 'Internet Control Message Protocol' (ICMP) packet, specifically an 'Echo (ping) request' (Type: 8, Code: 0). The packet details are as follows:

- Frame 4: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0
- Ethernet II, Src: Apple\_13:c5:58 (60:33:4b:13:c5:58), Dst: MS-NLB-PhysServer-26\_11:f0:c8:3b (02:1a:11:f0:c8:3b)
- Internet Protocol Version 4, Src: 192.168.43.9, Dst: 8.8.8.8
- Internet Control Message Protocol
  - Type: 8 (Echo (ping) request)
  - Code: 0
  - Checksum: 0xbbb0 [correct]
  - [Checksum Status: Good]
  - Identifier (BE): 55099 (0xd73b)
  - Identifier (LE): 15319 (0x3bd7)
  - Sequence number (BE): 0 (0x0000)
  - Sequence number (LE): 0 (0x0000)
  - [Response frame: 5]
  - Timestamp from icmp data: May 30, 2013 18:45:17.283108000 EDT
  - [Timestamp from icmp data (relative): 0.000070000 seconds]
  - Data (48 bytes)
    - Data: 08090a0b0c0d0e0f0101112131415161718191a1b1c1d1e1f...
    - [Length: 48]

**Type 8 & Code 0** both indicate that this packet is an echo request. **Data** represents a random text string.

## 2.3.1.1 Normal ICMP

Below we see packet details for the Echo (Ping) Reply packet, packet 5.

```
▶ Frame 5: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0
▶ Ethernet II, Src: MS-NLB-PhysServer-26_11:f0:c8:3b (02:1a:11:f0:c8:3b), Dst: Apple_13:c5:58 (60:33:4b:13:c5:58)
▶ Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.43.9
▼ Internet Control Message Protocol
  Type: 0 (Echo (ping) reply)
  Code: 0
  Checksum: 0xc3b3 [correct]
  [Checksum Status: Good]
  Identifier (BE): 55099 (0xd73b)
  Identifier (LE): 15319 (0x3bd7)
  Sequence number (BE): 0 (0x0000)
  Sequence number (LE): 0 (0x0000)
  [Request frame: 4]
  [Response time: 492.204 ms]
  Timestamp from icmp data: May 30, 2013 18:45:17.283108000 EDT
  [Timestamp from icmp data (relative): 0.492283000 seconds]
▼ Data (48 bytes)
  Data: 08090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f...
  [Length: 48]
```

**Type 0 & Code 0** both indicate that this packet is an echo reply. **Data** should be the same random text string from echo request. If so, then ping utility will report success back to the command line.

## 2.3.1.2 Suspicious ICMP

As you can imagine the data field of an ICMP packet can be used as a covert channel or even an exfiltration channel.

**Large ICMP packets** should be a red flag.

Also be watchful for **unusual types/codes** within the ICMP packets followed by a request, such as a **Timestamp request**.

```
24 # the experiment data result is a dict
25 # observations is an array of Observations, an Observation
26 # control is the control Observation
27
28 def initialize(experiment, observations = [], control = null)
29
30   @experiment = experiment
31   @observations = observations
32   @control = control
33   @candidates = observations - [control]
34   evaluate_candidates
35
36   freeze
37
38   context
39   experiment.context
40
41   experiment_name
42   experiment.name
43   name
44
45   # @candidates will be the result a match between all
46   def matches?
47     @candidates
48
49   def matches?
50     @candidates
51
52   def matches?
53     @candidates
54
55   def matches?
56     @candidates
57
58   def matches?
59     @candidates
60
61   def matches?
62     @candidates
63
64   def matches?
65     @candidates
66
67   def matches?
68     @candidates
69
70   def matches?
71     @candidates
72
73   def matches?
74     @candidates
75
76   def matches?
77     @candidates
78
79   def matches?
80     @candidates
81
82   def matches?
83     @candidates
84
85   def matches?
86     @candidates
87
88   def matches?
89     @candidates
90
91   def matches?
92     @candidates
93
94   def matches?
95     @candidates
96
97   def matches?
98     @candidates
99
100  def matches?
101    @candidates
102
103  def matches?
104    @candidates
105
106  def matches?
107    @candidates
108
109  def matches?
110    @candidates
111
112  def matches?
113    @candidates
114
115  def matches?
116    @candidates
117
118  def matches?
119    @candidates
120
121  def matches?
122    @candidates
123
124  def matches?
125    @candidates
126
127  def matches?
128    @candidates
129
130  def matches?
131    @candidates
132
133  def matches?
134    @candidates
135
136  def matches?
137    @candidates
138
139  def matches?
140    @candidates
141
142  def matches?
143    @candidates
144
145  def matches?
146    @candidates
147
148  def matches?
149    @candidates
150
151  def matches?
152    @candidates
153
154  def matches?
155    @candidates
156
157  def matches?
158    @candidates
159
160  def matches?
161    @candidates
162
163  def matches?
164    @candidates
165
166  def matches?
167    @candidates
168
169  def matches?
170    @candidates
171
172  def matches?
173    @candidates
174
175  def matches?
176    @candidates
177
178  def matches?
179    @candidates
180
181  def matches?
182    @candidates
183
184  def matches?
185    @candidates
186
187  def matches?
188    @candidates
189
190  def matches?
191    @candidates
192
193  def matches?
194    @candidates
195
196  def matches?
197    @candidates
198
199  def matches?
200    @candidates
201
202  def matches?
203    @candidates
204
205  def matches?
206    @candidates
207
208  def matches?
209    @candidates
210
211  def matches?
212    @candidates
213
214  def matches?
215    @candidates
216
217  def matches?
218    @candidates
219
220  def matches?
221    @candidates
222
223  def matches?
224    @candidates
225
226  def matches?
227    @candidates
228
229  def matches?
230    @candidates
231
232  def matches?
233    @candidates
234
235  def matches?
236    @candidates
237
238  def matches?
239    @candidates
240
241  def matches?
242    @candidates
243
244  def matches?
245    @candidates
246
247  def matches?
248    @candidates
249
250  def matches?
251    @candidates
252
253  def matches?
254    @candidates
255
256  def matches?
257    @candidates
258
259  def matches?
260    @candidates
261
262  def matches?
263    @candidates
264
265  def matches?
266    @candidates
267
268  def matches?
269    @candidates
270
271  def matches?
272    @candidates
273
274  def matches?
275    @candidates
276
277  def matches?
278    @candidates
279
280  def matches?
281    @candidates
282
283  def matches?
284    @candidates
285
286  def matches?
287    @candidates
288
289  def matches?
290    @candidates
291
292  def matches?
293    @candidates
294
295  def matches?
296    @candidates
297
298  def matches?
299    @candidates
300
301  def matches?
302    @candidates
303
304  def matches?
305    @candidates
306
307  def matches?
308    @candidates
309
310  def matches?
311    @candidates
312
313  def matches?
314    @candidates
315
316  def matches?
317    @candidates
318
319  def matches?
320    @candidates
321
322  def matches?
323    @candidates
324
325  def matches?
326    @candidates
327
328  def matches?
329    @candidates
330
331  def matches?
332    @candidates
333
334  def matches?
335    @candidates
336
337  def matches?
338    @candidates
339
340  def matches?
341    @candidates
342
343  def matches?
344    @candidates
345
346  def matches?
347    @candidates
348
349  def matches?
350    @candidates
351
352  def matches?
353    @candidates
354
355  def matches?
356    @candidates
357
358  def matches?
359    @candidates
360
361  def matches?
362    @candidates
363
364  def matches?
365    @candidates
366
367  def matches?
368    @candidates
369
370  def matches?
371    @candidates
372
373  def matches?
374    @candidates
375
376  def matches?
377    @candidates
378
379  def matches?
380    @candidates
381
382  def matches?
383    @candidates
384
385  def matches?
386    @candidates
387
388  def matches?
389    @candidates
390
391  def matches?
392    @candidates
393
394  def matches?
395    @candidates
396
397  def matches?
398    @candidates
399
400  def matches?
401    @candidates
402
403  def matches?
404    @candidates
405
406  def matches?
407    @candidates
408
409  def matches?
410    @candidates
411
412  def matches?
413    @candidates
414
415  def matches?
416    @candidates
417
418  def matches?
419    @candidates
420
421  def matches?
422    @candidates
423
424  def matches?
425    @candidates
426
427  def matches?
428    @candidates
429
430  def matches?
431    @candidates
432
433  def matches?
434    @candidates
435
436  def matches?
437    @candidates
438
439  def matches?
440    @candidates
441
442  def matches?
443    @candidates
444
445  def matches?
446    @candidates
447
448  def matches?
449    @candidates
450
451  def matches?
452    @candidates
453
454  def matches?
455    @candidates
456
457  def matches?
458    @candidates
459
460  def matches?
461    @candidates
462
463  def matches?
464    @candidates
465
466  def matches?
467    @candidates
468
469  def matches?
470    @candidates
471
472  def matches?
473    @candidates
474
475  def matches?
476    @candidates
477
478  def matches?
479    @candidates
480
481  def matches?
482    @candidates
483
484  def matches?
485    @candidates
486
487  def matches?
488    @candidates
489
490  def matches?
491    @candidates
492
493  def matches?
494    @candidates
495
496  def matches?
497    @candidates
498
499  def matches?
500    @candidates
501
502  def matches?
503    @candidates
504
505  def matches?
506    @candidates
507
508  def matches?
509    @candidates
510
511  def matches?
512    @candidates
513
514  def matches?
515    @candidates
516
517  def matches?
518    @candidates
519
520  def matches?
521    @candidates
522
523  def matches?
524    @candidates
525
526  def matches?
527    @candidates
528
529  def matches?
530    @candidates
531
532  def matches?
533    @candidates
534
535  def matches?
536    @candidates
537
538  def matches?
539    @candidates
540
541  def matches?
542    @candidates
543
544  def matches?
545    @candidates
546
547  def matches?
548    @candidates
549
550  def matches?
551    @candidates
552
553  def matches?
554    @candidates
555
556  def matches?
557    @candidates
558
559  def matches?
560    @candidates
561
562  def matches?
563    @candidates
564
565  def matches?
566    @candidates
567
568  def matches?
569    @candidates
570
571  def matches?
572    @candidates
573
574  def matches?
575    @candidates
576
577  def matches?
578    @candidates
579
580  def matches?
581    @candidates
582
583  def matches?
584    @candidates
585
586  def matches?
587    @candidates
588
589  def matches?
590    @candidates
591
592  def matches?
593    @candidates
594
595  def matches?
596    @candidates
597
598  def matches?
599    @candidates
600
601  def matches?
602    @candidates
603
604  def matches?
605    @candidates
606
607  def matches?
608    @candidates
609
610  def matches?
611    @candidates
612
613  def matches?
614    @candidates
615
616  def matches?
617    @candidates
618
619  def matches?
620    @candidates
621
622  def matches?
623    @candidates
624
625  def matches?
626    @candidates
627
628  def matches?
629    @candidates
630
631  def matches?
632    @candidates
633
634  def matches?
635    @candidates
636
637  def matches?
638    @candidates
639
640  def matches?
641    @candidates
642
643  def matches?
644    @candidates
645
646  def matches?
647    @candidates
648
649  def matches?
650    @candidates
651
652  def matches?
653    @candidates
654
655  def matches?
656    @candidates
657
658  def matches?
659    @candidates
660
661  def matches?
662    @candidates
663
664  def matches?
665    @candidates
666
667  def matches?
668    @candidates
669
670  def matches?
671    @candidates
672
673  def matches?
674    @candidates
675
676  def matches?
677    @candidates
678
679  def matches?
680    @candidates
681
682  def matches?
683    @candidates
684
685  def matches?
686    @candidates
687
688  def matches?
689    @candidates
690
691  def matches?
692    @candidates
693
694  def matches?
695    @candidates
696
697  def matches?
698    @candidates
699
700  def matches?
701    @candidates
702
703  def matches?
704    @candidates
705
706  def matches?
707    @candidates
708
709  def matches?
710    @candidates
711
712  def matches?
713    @candidates
714
715  def matches?
716    @candidates
717
718  def matches?
719    @candidates
720
721  def matches?
722    @candidates
723
724  def matches?
725    @candidates
726
727  def matches?
728    @candidates
729
730  def matches?
731    @candidates
732
733  def matches?
734    @candidates
735
736  def matches?
737    @candidates
738
739  def matches?
740    @candidates
741
742  def matches?
743    @candidates
744
745  def matches?
746    @candidates
747
748  def matches?
749    @candidates
750
751  def matches?
752    @candidates
753
754  def matches?
755    @candidates
756
757  def matches?
758    @candidates
759
760  def matches?
761    @candidates
762
763  def matches?
764    @candidates
765
766  def matches?
767    @candidates
768
769  def matches?
770    @candidates
771
772  def matches?
773    @candidates
774
775  def matches?
776    @candidates
777
778  def matches?
779    @candidates
780
781  def matches?
782    @candidates
783
784  def matches?
785    @candidates
786
787  def matches?
788    @candidates
789
790  def matches?
791    @candidates
792
793  def matches?
794    @candidates
795
796  def matches?
797    @candidates
798
799  def matches?
800    @candidates
801
802  def matches?
803    @candidates
804
805  def matches?
806    @candidates
807
808  def matches?
809    @candidates
810
811  def matches?
812    @candidates
813
814  def matches?
815    @candidates
816
817  def matches?
818    @candidates
819
820  def matches?
821    @candidates
822
823  def matches?
824    @candidates
825
826  def matches?
827    @candidates
828
829  def matches?
830    @candidates
831
832  def matches?
833    @candidates
834
835  def matches?
836    @candidates
837
838  def matches?
839    @candidates
840
841  def matches?
842    @candidates
843
844  def matches?
845    @candidates
846
847  def matches?
848    @candidates
849
850  def matches?
851    @candidates
852
853  def matches?
854    @candidates
855
856  def matches?
857    @candidates
858
859  def matches?
860    @candidates
861
862  def matches?
863    @candidates
864
865  def matches?
866    @candidates
867
868  def matches?
869    @candidates
870
871  def matches?
872    @candidates
873
874  def matches?
875    @candidates
876
877  def matches?
878    @candidates
879
880  def matches?
881    @candidates
882
883  def matches?
884    @candidates
885
886  def matches?
887    @candidates
888
889  def matches?
890    @candidates
891
892  def matches?
893    @candidates
894
895  def matches?
896    @candidates
897
898  def matches?
899    @candidates
900
901  def matches?
902    @candidates
903
904  def matches?
905    @candidates
906
907  def matches?
908    @candidates
909
910  def matches?
911    @candidates
912
913  def matches?
914    @candidates
915
916  def matches?
917    @candidates
918
919  def matches?
920    @candidates
921
922  def matches?
923    @candidates
924
925  def matches?
926    @candidates
927
928  def matches?
929    @candidates
929
```

## 2.3.2 Wireshark Tip

For more information on techniques to filter ICMP traffic, you can refer to the Wireshark Display Filter Reference for ICMP, [here](https://www.wireshark.org/docs/dfref/i/icmp.html).

# TCP Traffic



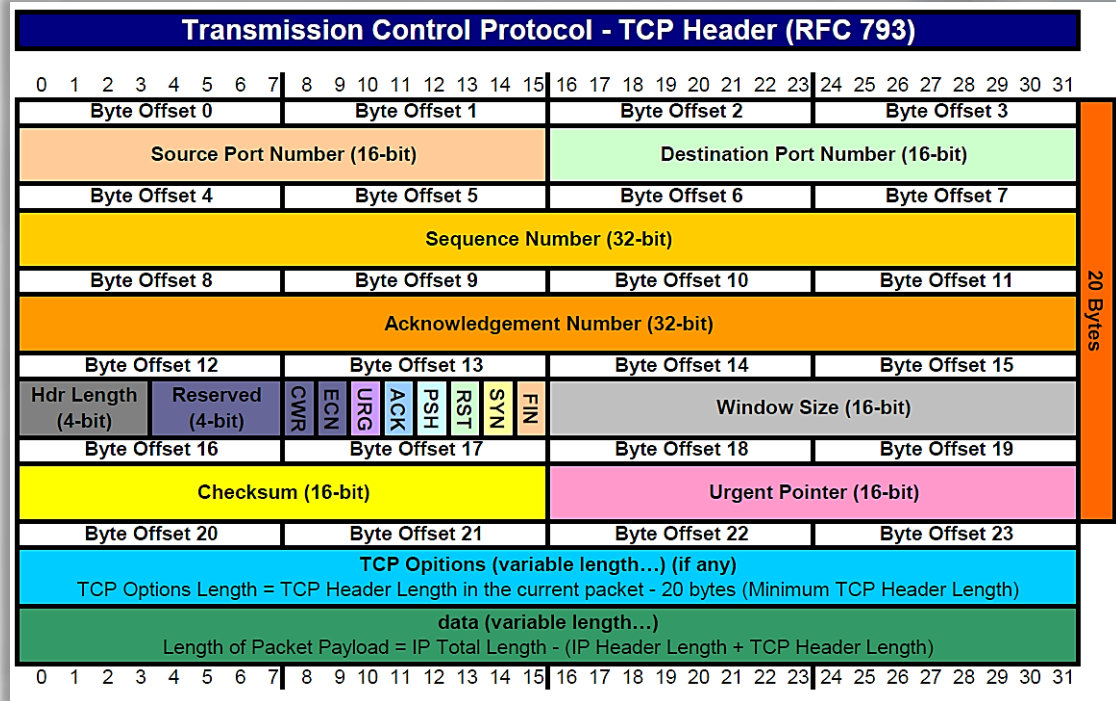
## 2.4.1 TCP

**TCP (Transmission Control Protocol)** ensures delivery of data from the source node to the destination node. TCP handles sequencing of packets and error recovery.

Prior to communication, TCP conducts a 3-way handshake between both nodes. You can read more about TCP [here](#).

## 2.4.1 TCP

Here is a TCP Header.



## 2.4.1 TCP



### 3-Way Handshake



Time

SYN

SYN + ACK

ACK



Time

## 2.4.1 TCP

### Some facts to help distinguish normal and suspicious TCP traffic.

| Normal TCP Traffic                  | Suspicious TCP Traffic                                                    |
|-------------------------------------|---------------------------------------------------------------------------|
| 3-way handshake (SYN, SYN/ACK, ACK) | Excessive SYN packets (scanning)                                          |
|                                     | Smart TCP attacks (usage of different flags)                              |
|                                     | Single host to multiple ports or single host to multiple nodes (scanning) |

## 2.4.1.1 Normal TCP

An example of a normal TCP 3-way handshake is shown below:

| Time       | Source       | Destination  | Protocol | Length | Info                                                                                                  |
|------------|--------------|--------------|----------|--------|-------------------------------------------------------------------------------------------------------|
| 1 0.000000 | 10.54.15.100 | 10.54.15.68  | TCP      | 74     | 51040 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2677172 TSecr=0 WS=128              |
| 2 0.054908 | 10.54.15.68  | 10.54.15.100 | TCP      | 74     | 80 → 51040 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=68008 TSecr=2677172 WS=4 |
| 3 0.054929 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51040 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2677186 TSecr=68008                                |

## 2.4.1.1 Normal TCP

The images below display each of the packets in greater details from the handshake displayed in the previous slide:

```

> Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
> Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0)
> Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68
> Transmission Control Protocol, Src Port: 51040, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 51040
  Destination Port: 80
  [Stream index: 0]
  [TCP Segment Len: 0]
  Sequence number: 0 (relative sequence number)
  Acknowledgment number: 0
  Header Length: 40 bytes
  Flags: 0x002 (SYN)
  Window size value: 29200
  [Calculated window size: 29200]
  Checksum: 0x1ef3 [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Time
```

```

> Frame 2: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
> Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
> Internet Protocol Version 4, Src: 10.54.15.68, Dst: 10.54.15.100
> Transmission Control Protocol, Src Port: 80, Dst Port: 51040, Seq: 0, Ack: 1, Len: 0
  Source Port: 80
  Destination Port: 51040
  [Stream index: 0]
  [TCP Segment Len: 0]
  Sequence number: 0 (relative sequence number)
  Acknowledgment number: 1 (relative ack number)
  Header Length: 40 bytes
  Flags: 0x012 (SYN, ACK)
  Window size value: 14480
  [Calculated window size: 14480]
  Checksum: 0xf9fa [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  [SEQ/ACK analysis]
```

Continued on next slide

# 2.4.1.1 Normal TCP

```
▶ Frame 2: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
▶ Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
▶ Internet Protocol Version 4, Src: 10.54.15.68, Dst: 10.54.15.100
▶ Transmission Control Protocol, Src Port: 80, Dst Port: 51040, Seq: 0, Ack: 1, Len: 0
```

```
Source Port: 80
Destination Port: 51040
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 0 (relative sequence number)
Acknowledgment number: 1 (relative ack number)
Header Length: 40 bytes
▶ Flags: 0x012 (SYN, ACK)
Window size value: 14480
[Calculated window size: 14480]
Checksum: 0xf9fa [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
▶ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation
  ▶ Maximum segment size: 1337 bytes
  ▶ TCP SACK Permitted Option: True
  ▶ Timestamps: TSval 68008, TSecr 2677172
  ▶ No-Operation (NOP)
  ▶ Window scale: 2 (multiply by 4)
▶ [SEQ/ACK analysis]
  [This is an ACK to the segment in frame: 1]
  [The RTT to ACK the segment was: 0.054908000 seconds]
  [iRTT: 0.054929000 seconds]
```

```
▶ Frame 3: 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0)
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68
```

```
▶ Transmission Control Protocol, Src Port: 51040, Dst Port: 80, Seq: 1, Ack: 1, Len: 0
```

```
Source Port: 51040
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 1 (relative sequence number)
Acknowledgment number: 1 (relative ack number)
Header Length: 32 bytes
▶ Flags: 0x010 (ACK)
Window size value: 229
[Calculated window size: 29312]
[Window size scaling factor: 128]
Checksum: 0x5fe4 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
▶ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▶ No-Operation (NOP)
  ▶ No-Operation (NOP)
  ▶ Timestamps: TSval 2677186, TSecr 68008
▶ [SEQ/ACK analysis]
  [This is an ACK to the segment in frame: 2]
  [The RTT to ACK the segment was: 0.000021000 seconds]
  [iRTT: 0.054929000 seconds]
```

## 2.4.1.1 Normal TCP

We will not be digging into every packet detail for TCP, but simply be able to recognize the important parts of TCP behavior and overall packet structure. One thing worth mentioning at this point is the **relative sequence number** and **relative acknowledgement (ack) number**.

The following two slides will display the official definition from the online Wireshark Wiki, [here](https://wiki.wireshark.org/TCP_Relative_Sequence_Numbers).

## 2.4.1.1 Normal TCP

“By default Wireshark and TShark will keep track of all TCP sessions and convert all Sequence Numbers (SEQ numbers) and Acknowledge Numbers (ACK Numbers) into relative numbers.

This means that instead of displaying the real/absolute SEQ and ACK numbers in the display, Wireshark will display a SEQ and ACK number relative to the first seen segment for that conversation.”

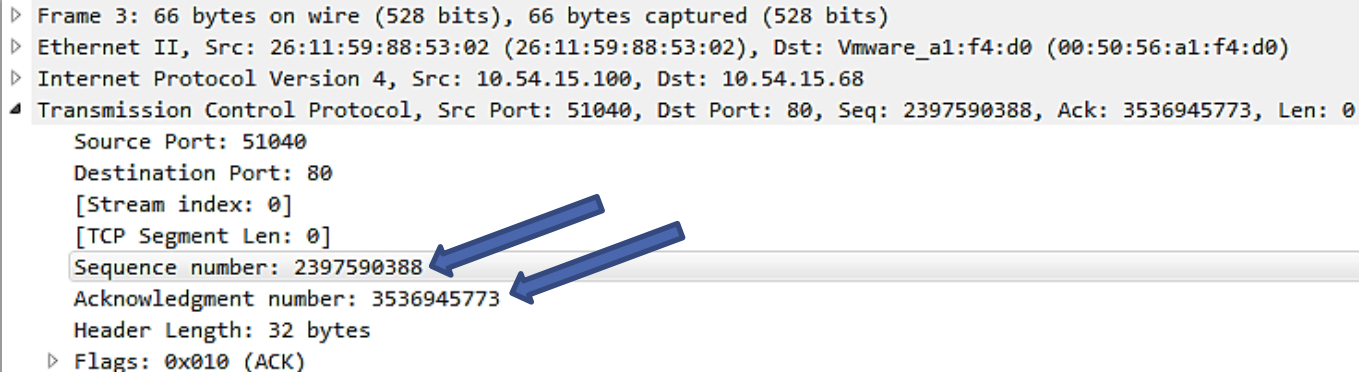
## 2.4.1.1 Normal TCP

“This means that all SEQ and ACK numbers always start at 0 for the first packet seen in each conversation.

This makes the numbers much smaller and easier to read and compare than the real numbers which normally are initialized to randomly selected numbers in the range 0 -  $(2^{32})-1$  during the SYN phase.”

## 2.4.1.1 Normal TCP

We can go into Wireshark and make a quick change, by going to **Edit > Preferences > Protocols > TCP > Relative sequence numbers (uncheck box)**, to see the original OR actual sequence and acknowledgment numbers.



```
▶ Frame 3: 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0)
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68
▲ Transmission Control Protocol, Src Port: 51040, Dst Port: 80, Seq: 2397590388, Ack: 3536945773, Len: 0
    Source Port: 51040
    Destination Port: 80
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 2397590388
    Acknowledgment number: 3536945773
    Header Length: 32 bytes
▶ Flags: 0x010 (ACK)
```

## 2.4.1.2 Suspicious TCP

Below is a snapshot of a ping sweep performed with Nmap. This particular ping sweep used **Echo (Ping) Requests** and **TCP SYN** connections to ports 80 & 443.

| No.  | Time         | Source      | Destination  | Protocol | Length | Info                                                                |
|------|--------------|-------------|--------------|----------|--------|---------------------------------------------------------------------|
| 252  | 1.884272105  | 172.16.5.50 | 10.50.96.115 | ICMP     | 42     | Echo (ping) request id=0x26f4, seq=0/0, ttl=56 (no response found!) |
| 528  | 3.239891218  | 172.16.5.50 | 10.50.96.115 | ICMP     | 42     | Echo (ping) request id=0xc7a1, seq=0/0, ttl=51 (no response found!) |
| 1018 | 5.882700328  | 172.16.5.50 | 10.50.96.115 | TCP      | 58     | 51286 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460                     |
| 1054 | 6.163446779  | 172.16.5.50 | 10.50.96.115 | TCP      | 58     | 51287 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460                     |
| 1674 | 10.779530015 | 172.16.5.50 | 10.50.96.115 | TCP      | 54     | 51286 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0                         |
| 1703 | 10.977220446 | 172.16.5.50 | 10.50.96.115 | TCP      | 54     | 51287 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0                         |
| 2620 | 15.474233425 | 172.16.5.50 | 10.50.96.115 | ICMP     | 54     | Timestamp request id=0xa837, seq=0/0, ttl=41                        |
| 3006 | 16.791005321 | 172.16.5.50 | 10.50.96.115 | ICMP     | 54     | Timestamp request id=0xe11a, seq=0/0, ttl=55                        |
| 3406 | 18.151715145 | 172.16.5.50 | 10.50.96.115 | ICMP     | 54     | Timestamp request id=0x554a, seq=0/0, ttl=47                        |
| 3843 | 19.454155429 | 172.16.5.50 | 10.50.96.115 | ICMP     | 54     | Timestamp request id=0xfb82, seq=0/0, ttl=46                        |

Remember from the ICMP section, that it was mentioned to look out for **Timestamp Requests** following an Echo (Ping) Request. We are seeing that combination in the packets above.

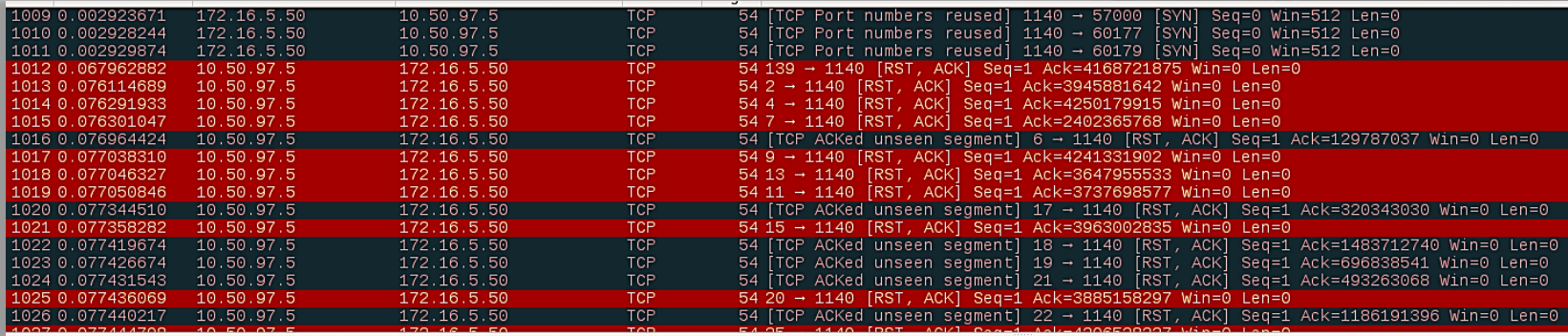
## 2.4.1.2 Suspicious TCP

As mentioned in the suspicious TCP traffic checklist, **many TCP SYN** packets **without** the corresponding **SYN/ACK** packets should raise a flag. Here we see a **port scan**.

|    |             |             |            |     |              |       |                     |
|----|-------------|-------------|------------|-----|--------------|-------|---------------------|
| 1  | 0.000000000 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 1  | [SYN] | Seq=0 Win=512 Len=0 |
| 2  | 0.000068731 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 2  | [SYN] | Seq=0 Win=512 Len=0 |
| 3  | 0.000072857 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 4  | [SYN] | Seq=0 Win=512 Len=0 |
| 4  | 0.000074818 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 6  | [SYN] | Seq=0 Win=512 Len=0 |
| 5  | 0.000076675 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 7  | [SYN] | Seq=0 Win=512 Len=0 |
| 6  | 0.000078473 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 9  | [SYN] | Seq=0 Win=512 Len=0 |
| 7  | 0.000080677 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 11 | [SYN] | Seq=0 Win=512 Len=0 |
| 8  | 0.000082544 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 13 | [SYN] | Seq=0 Win=512 Len=0 |
| 9  | 0.000084584 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 15 | [SYN] | Seq=0 Win=512 Len=0 |
| 10 | 0.000086544 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 17 | [SYN] | Seq=0 Win=512 Len=0 |
| 11 | 0.000088415 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 18 | [SYN] | Seq=0 Win=512 Len=0 |
| 12 | 0.000090121 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 19 | [SYN] | Seq=0 Win=512 Len=0 |
| 13 | 0.000091943 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 20 | [SYN] | Seq=0 Win=512 Len=0 |
| 14 | 0.000093777 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 21 | [SYN] | Seq=0 Win=512 Len=0 |
| 15 | 0.000095482 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 22 | [SYN] | Seq=0 Win=512 Len=0 |
| 16 | 0.000097189 | 172.16.5.50 | 10.50.97.5 | TCP | 54 1140 → 23 | [SYN] | Seq=0 Win=512 Len=0 |

## 2.4.1.2 Suspicious TCP

Wireshark gives us a helping hand to let us know something is awry with this traffic by highlighting it in different color. You may read more about the meaning colors represent [here](https://www.wireshark.org/docs/wsug_html_chunked/ChCustColorizationSection.html).



The image shows a Wireshark packet capture of a TCP connection. The packets are listed in a table with columns for Time, Source, Destination, Protocol, Length, and Details. The packets are numbered 1009 to 1026. The details column shows the TCP segment structure, including sequence numbers, acknowledgment numbers, window size, and flags. The packets are highlighted in red, indicating they are suspicious. The details for each packet are as follows:

| Time | Source      | Destination | Protocol | Length | Details                                                                          |
|------|-------------|-------------|----------|--------|----------------------------------------------------------------------------------|
| 1009 | 0.002923671 | 172.16.5.50 | TCP      | 54     | [TCP Port numbers reused] 1140 → 57000 [SYN] Seq=0 Win=512 Len=0                 |
| 1010 | 0.002928244 | 172.16.5.50 | TCP      | 54     | [TCP Port numbers reused] 1140 → 60177 [SYN] Seq=0 Win=512 Len=0                 |
| 1011 | 0.002929874 | 172.16.5.50 | TCP      | 54     | [TCP Port numbers reused] 1140 → 60179 [SYN] Seq=0 Win=512 Len=0                 |
| 1012 | 0.067962882 | 10.50.97.5  | TCP      | 54     | 139 → 1140 [RST, ACK] Seq=1 Ack=4168721875 Win=0 Len=0                           |
| 1013 | 0.076114689 | 10.50.97.5  | TCP      | 54     | 2 → 1140 [RST, ACK] Seq=1 Ack=3945881642 Win=0 Len=0                             |
| 1014 | 0.076291933 | 10.50.97.5  | TCP      | 54     | 4 → 1140 [RST, ACK] Seq=1 Ack=4250179915 Win=0 Len=0                             |
| 1015 | 0.076301047 | 10.50.97.5  | TCP      | 54     | 7 → 1140 [RST, ACK] Seq=1 Ack=2402365768 Win=0 Len=0                             |
| 1016 | 0.076964424 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 6 → 1140 [RST, ACK] Seq=1 Ack=129787037 Win=0 Len=0   |
| 1017 | 0.077038310 | 10.50.97.5  | TCP      | 54     | 9 → 1140 [RST, ACK] Seq=1 Ack=4241331902 Win=0 Len=0                             |
| 1018 | 0.077046327 | 10.50.97.5  | TCP      | 54     | 13 → 1140 [RST, ACK] Seq=1 Ack=3647955533 Win=0 Len=0                            |
| 1019 | 0.077050846 | 10.50.97.5  | TCP      | 54     | 11 → 1140 [RST, ACK] Seq=1 Ack=3737698577 Win=0 Len=0                            |
| 1020 | 0.077344510 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 17 → 1140 [RST, ACK] Seq=1 Ack=320343030 Win=0 Len=0  |
| 1021 | 0.077358282 | 10.50.97.5  | TCP      | 54     | 15 → 1140 [RST, ACK] Seq=1 Ack=3963002835 Win=0 Len=0                            |
| 1022 | 0.077419674 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 18 → 1140 [RST, ACK] Seq=1 Ack=1483712740 Win=0 Len=0 |
| 1023 | 0.077426674 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 19 → 1140 [RST, ACK] Seq=1 Ack=696838541 Win=0 Len=0  |
| 1024 | 0.077431543 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 21 → 1140 [RST, ACK] Seq=1 Ack=493263068 Win=0 Len=0  |
| 1025 | 0.077436069 | 10.50.97.5  | TCP      | 54     | 20 → 1140 [RST, ACK] Seq=1 Ack=3885158297 Win=0 Len=0                            |
| 1026 | 0.077440217 | 10.50.97.5  | TCP      | 54     | [TCP ACKed unseen segment] 22 → 1140 [RST, ACK] Seq=1 Ack=1186191396 Win=0 Len=0 |

## 2.4.1.2 Suspicious TCP

Below we are looking at another packet capture of a SYN scan. The traffic was filtered to focus on a specific node within the noise, 10.50.97.5.

|      |             |             |             |      |                           |                                              |
|------|-------------|-------------|-------------|------|---------------------------|----------------------------------------------|
| 430  | 1.160997136 | 172.16.5.50 | 10.50.97.5  | ICMP | 42 Echo (ping) request    | id=0x4601, seq=0/0, ttl=50 (reply in 469)    |
| 469  | 1.203123102 | 10.50.97.5  | 172.16.5.50 | ICMP | 42 Echo (ping) reply      | id=0x4601, seq=0/0, ttl=127 (request in 430) |
| 4097 | 9.663041665 | 172.16.5.50 | 10.50.97.5  | TCP  | 58 53894 → 135 [SYN]      | Seq=0 Win=1024 Len=0 MSS=1460                |
| 4107 | 9.735178880 | 10.50.97.5  | 172.16.5.50 | TCP  | 58 135 → 53894 [SYN, ACK] | Seq=0 Ack=1 Win=64240 Len=0 MSS=1337         |
| 4108 | 9.735182807 | 172.16.5.50 | 10.50.97.5  | TCP  | 54 53894 → 135 [RST]      | Seq=1 Win=0 Len=0                            |

We see the following with the last 3 packets:

- SYN to start communication with 10.50.97.5
- SYN,ACK response which tells us the port is open
- RST to end communication.

## 2.4.1.2 Suspicious TCP

We will not cover every possible scenario. Some more examples of suspicious TCP traffic will be seen within the videos/labs in this module.

Please refer to the NMAP page on Port Scanning Techniques [here](https://nmap.org/book/man-port-scanning-techniques.html) to get familiar with them.

## 2.4.2 UDP

It is worth mentioning that the NMAP page on Port Scanning Techniques will also cover **UDP (User Datagram Protocol)** scans so we will not dedicate time explaining that, please refer to the [page](https://nmap.org/book/man-port-scanning-techniques.html).

We have seen or will see UDP usage with other protocols, such as DNS and DHCP.

## 2.4.3 Wireshark Tip

Refer to the Wireshark Display Filter Reference for TCP, [here](https://www.wireshark.org/docs/dfref/t/tcp.html), on more techniques to filter TCP traffic.

# DHCP Traffic



## 2.5.1 DHCP

### **DHCP (Dynamic Host Configuration Protocol)**

automatically assigns an IP address to every node on a network and allows the devices to communicate with other devices within and outside the internal network using IP-based routing.

You can learn more, or freshen up, on DHCP [here](https://www.ietf.org/rfc/rfc2131.txt).

## 2.5.1 DHCP

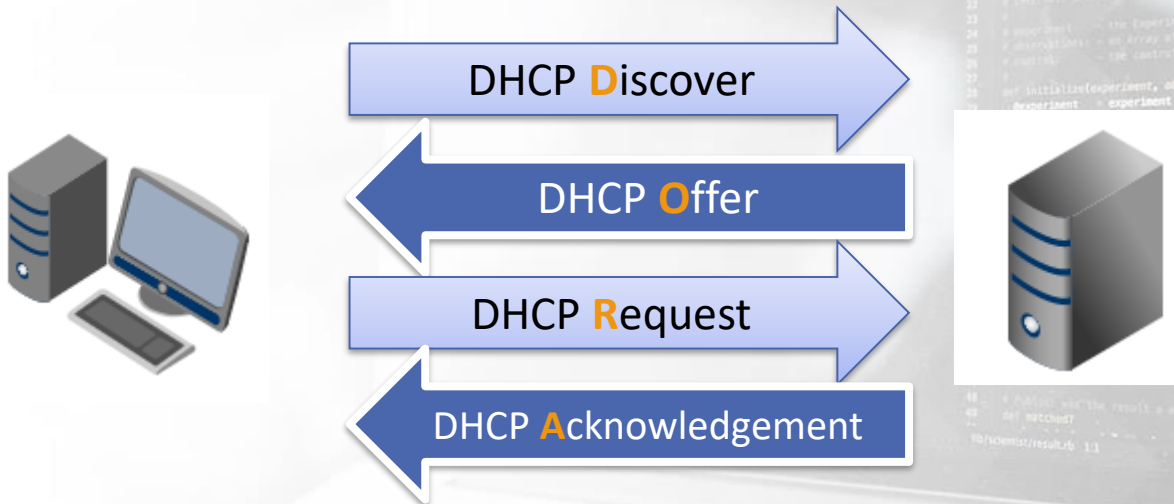
A few things to point out regarding **DHCP (Dynamic Host Configuration Protocol)**:

- It automatically assigns IP addresses
- It also provides other information, such as DNS server(s), Gateway, etc.
- The DORA process is used (DHCP Discover, DHCP Offer, DHCP Request, DHCP Acknowledgement)
- It uses UDP, Ports 67 & 68



## 2.5.1 DHCP

Below is a visualization depicting **DORA**. We'll see this as packets in the upcoming slides.



## 2.5.1 DHCP

On the image to the right, we can see the DHCP packet header.

| Dynamic Host Configuration Protocol (DHCP) |       |                                    |               |                 |       |
|--------------------------------------------|-------|------------------------------------|---------------|-----------------|-------|
| Offsets                                    | Octet | 0                                  | 1             | 2               | 3     |
| Octet                                      | Bit   | 0–7                                | 8–15          | 16–23           | 24–31 |
| 0                                          | 0     | OpCode                             | Hardware Type | Hardware Length | Hops  |
| 4                                          | 32    | Transaction ID                     |               |                 |       |
| 8                                          | 64    | Seconds Elapsed                    |               | Flags           |       |
| 12                                         | 96    | Client IP Address                  |               |                 |       |
| 16                                         | 128   | Your IP Address                    |               |                 |       |
| 20                                         | 160   | Server IP Address                  |               |                 |       |
| 24                                         | 192   | Gateway IP Address                 |               |                 |       |
| 28                                         | 224   | Client IP Address                  |               |                 |       |
| 32                                         | 256   | Client Hardware Address (16 bytes) |               |                 |       |
| 36                                         | 288   |                                    |               |                 |       |
| 40                                         | 320   |                                    |               |                 |       |
| 44                                         | 352   |                                    |               |                 |       |
| 48+                                        | 384+  | Server Host Name (64 bytes)        |               |                 |       |
|                                            |       | Boot File (128 bytes)              |               |                 |       |
|                                            |       | Options                            |               |                 |       |

## 2.5.1.1 Normal DHCP

Below we see DORA packets (DHCP Discover, DHCP Offer, DHCP Request, & DHCP ACK).

| No. | Time     | Source      | Destination     | Protocol | Length | Info                                  |
|-----|----------|-------------|-----------------|----------|--------|---------------------------------------|
| 1   | 0.000000 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Discover - Transaction ID 0x3d1d |
| 2   | 0.000295 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP Offer - Transaction ID 0x3d1d    |
| 3   | 0.070031 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Request - Transaction ID 0x3d1e  |
| 4   | 0.070345 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP ACK - Transaction ID 0x3d1e      |

\*PCAP from Wireshark Sample Captures [page](#).

## 2.5.1.1 Normal DHCP

We see this is a DHCP Discover packet. The protocol and ports being used look correct.

```

> Frame 1: 314 bytes on wire (2512 bits), 314 bytes captured (2512 bits)
> Ethernet II, Src: Grandstr_01:fc:42 (00:0b:82:01:fc:42), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 68, Dst Port: 67
  Bootstrap Protocol (Discover)
    Message type: Boot Request (1)
    Hardware type: Ethernet (0x01)
    Hardware address length: 6
    Hops: 0
    Transaction ID: 0x00003d1d
    Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
    Client IP address: 0.0.0.0
    Your (client) IP address: 0.0.0.0
    Next server IP address: 0.0.0.0
    Relay agent IP address: 0.0.0.0
    Client MAC address: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
    Client hardware address padding: 00000000000000000000
    Server host name not given
    Boot file name not given
    Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Discover)
    Length: 1
    DHCP: Discover (1)
  > Option: (61) Client Identifier
  > Option: (50) Requested IP Address
  > Option: (55) Parameter Request List
```

## 2.5.1.1 Normal DHCP

```
▶ Frame 1: 314 bytes on wire (2512 bits), 314 bytes captured (2512 bits)
▶ Ethernet II, Src: Grandstr_01:fc:42 (00:0b:82:01:fc:42), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▶ Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
▶ User Datagram Protocol, Src Port: 68, Dst Port: 67
▶ Bootstrap Protocol (Discover)
```

These are additional packet details from the same packet that was shown on the previous slide. We see that the node is requesting an IP address, along with additional info from the DHCP server; this is sent via broadcast because the node doesn't know which node is the DHCP server.

```
▶ Option: (53) DHCP Message Type (Discover)
  Length: 1
  DHCP: Discover (1)
▶ Option: (61) Client identifier
  Length: 7
  Hardware type: Ethernet (0x01)
  Client MAC address: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
▶ Option: (50) Requested IP Address
  Length: 4
  Requested IP Address: 0.0.0.0
▶ Option: (55) Parameter Request List
  Length: 4
  Parameter Request List Item: (1) Subnet Mask
  Parameter Request List Item: (3) Router
  Parameter Request List Item: (6) Domain Name Server
  Parameter Request List Item: (42) Network Time Protocol Servers
▶ Option: (255) End
  Option End: 255
  Padding: 0000000000000000
```

## 2.5.1.1 Normal DHCP

This is the DHCP Offer packet. The DHCP server is offering an IP address to the node which requested it.

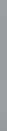
Other details are also provided within this packet, as we'll see in the next slide, such as netmask, lease time, DHCP server IP address, etc.

```

> Frame 2: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits)
> Ethernet II, Src: Dell_ad:f1:9b (00:08:74:ad:f1:9b), Dst: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
> Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.10
> User Datagram Protocol, Src Port: 67, Dst Port: 68
  < Bootstrap Protocol (Offer)
    Message type: Boot Reply (2)
    Hardware type: Ethernet (0x01)
    Hardware address length: 6
    Hops: 0
    Transaction ID: 0x00003d1d
    Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
    Client IP address: 0.0.0.0
    Your (client) IP address: 192.168.0.10
    Next server IP address: 192.168.0.1
    Relay agent IP address: 0.0.0.0
    Client MAC address: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
    Client hardware address padding: 00000000000000000000
    Server host name not given
    Boot file name not given
    Magic cookie: DHCP
  < Option: (53) DHCP Message Type (Offer)
    Length: 1
    DHCP: Offer (2)

```

Here is an example of normal DHCP traffic.

- 

## 2.5.1.1 Normal DHCP

Here is the 3<sup>rd</sup> step of DORA.

Again, we see the IP address that is being offered and the IP address of the DHCP server making the offer.

This transaction, 0x3d13, will end with the DHCP ACK packet.

| No. | Time     | Source      | Destination     | Protocol | Length | Info                                  |
|-----|----------|-------------|-----------------|----------|--------|---------------------------------------|
| 1   | 0.000000 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Discover - Transaction ID 0x3d1d |
| 2   | 0.000295 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP Offer - Transaction ID 0x3d1d    |
| 3   | 0.070031 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Request - Transaction ID 0x3d1e  |
| 4   | 0.070345 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP ACK - Transaction ID 0x3d1e      |

```
0... .. = Broadcast flag: Unicast
.000 0000 0000 0000 = Reserved flags: 0x0000
Client IP address: 0.0.0.0
Your (client) IP address: 0.0.0.0
Next server IP address: 0.0.0.0
Relay agent IP address: 0.0.0.0
Client MAC address: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
Client hardware address padding: 00000000000000000000
Server host name not given
Boot file name not given
Magic cookie: DHCP
Option: (53) DHCP Message Type (Request)
  Length: 1
  DHCP: Request (3)
Option: (61) Client Identifier
  Length: 7
  Hardware type: Ethernet (0x01)
  Client MAC address: Grandstr_01:fc:42 (00:0b:82:01:fc:42)
Option: (50) Requested IP Address
  Length: 4
  Requested IP Address: 192.168.0.10
Option: (54) DHCP Server Identifier
  Length: 4
  DHCP Server Identifier: 192.168.0.1
Option: (55) Parameter Request List
Option: (255) End
Padding: 00
```

## 2.5.1.1 Normal DHCP

This is the last packet within the transaction, DHCP ACK. This is when the node which made the request accepts the offer from the DHCP server.

So, as you might have guessed or known, one of the indicators to detect something is awry is by checking the DHCP server offering and fulfilling DHCP requests.

| No. | Time       | Source      | Destination     | Protocol | Length | Info                                  |
|-----|------------|-------------|-----------------|----------|--------|---------------------------------------|
| [   | 1 0.000000 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Discover - Transaction ID 0x3d1d |
|     | 2 0.000295 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP Offer - Transaction ID 0x3d1d    |
|     | 3 0.070031 | 0.0.0.0     | 255.255.255.255 | DHCP     | 314    | DHCP Request - Transaction ID 0x3d1e  |
|     | 4 0.070345 | 192.168.0.1 | 192.168.0.10    | DHCP     | 342    | DHCP ACK - Transaction ID 0x3d1e      |

[illegible]

## 2.5.1.2 Suspicious DHCP

In regard to suspicious DHCP traffic, we must be on the look out for machines acting as rogue DHCP servers.

An adversary who successfully issues IP address to victims can launch man-in-the-middle attacks against the victims.



## 2.5.2 Wireshark Tip

For more information on techniques to filter DHCP traffic, refer to the Wireshark Display Filter Reference for DHCP, [here](#).

Remember that within Wireshark, DHCP is still referenced as **BOOTP**.

# DNS Traffic



## 2.6.1 DNS

**DNS (Domain Name System)** is a protocol that resolves names to IP addresses. We will not dive into how DNS works exactly but we'll simply focus on how to recognize normal DNS traffic and suspicious DNS traffic.

You can learn more, or freshen up, on DNS [here](https://www.ietf.org/rfc/rfc1034.txt) and [here](https://www.ietf.org/rfc/rfc1035.txt).

## 2.6.1 DNS

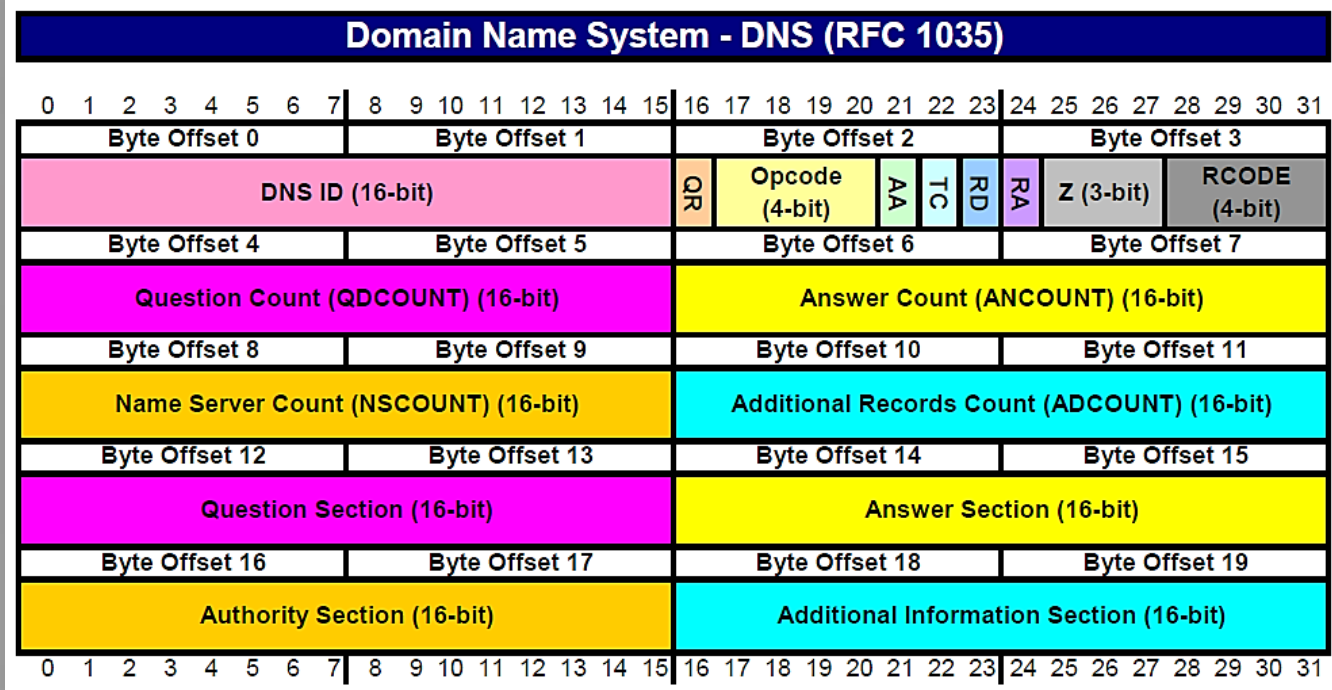
A few things to point out regarding **DNS (Domain Name System)**:

- DNS is a query-response protocol.
- DNS traffic normally uses UDP on port 53.
- DNS traffic should go to DNS servers only.



## 2.6.1 DNS

This is the format of a DNS packet.



## 2.6.1 DNS

### Some facts to help distinguish normal and suspicious DNS traffic.

| Normal DNS Traffic                      | Suspicious DNS Traffic                                   |
|-----------------------------------------|----------------------------------------------------------|
| Port 53, UDP                            | Traffic on port 53 but using TCP instead of UDP          |
| Should only go to DNS Servers           | DNS traffic not going to DNS Servers                     |
| Should see DNS Responses to DNS Queries | A lot of DNS Queries with no DNS responses or vice versa |

## 2.6.1.1 Normal DNS

Here we see 4 packets. 2 packets for **DNS Queries** and 2 for **DNS Responses**.

| dns |              |              |              |          |        |                                                                                                   |
|-----|--------------|--------------|--------------|----------|--------|---------------------------------------------------------------------------------------------------|
| No. | Time         | Source       | Destination  | Protocol | Length | Info                                                                                              |
| 16  | 26.200151138 | 172.16.5.100 | 172.16.5.10  | DNS      | 83     | Standard query 0xde40 PTR 5.5.16.172.in-addr.arpa                                                 |
| 19  | 26.272980431 | 172.16.5.10  | 172.16.5.100 | DNS      | 127    | Standard query response 0xde40 PTR 5.5.16.172.in-addr.arpa PTR wkst-techsupport.sportsfoo.com     |
| 41  | 56.605405613 | 172.16.5.100 | 172.16.5.10  | DNS      | 94     | Standard query 0xa620 PTR 5.5.16.172.in-addr.arpa OPT                                             |
| 42  | 56.639661726 | 172.16.5.10  | 172.16.5.100 | DNS      | 138    | Standard query response 0xa620 PTR 5.5.16.172.in-addr.arpa PTR wkst-techsupport.sportsfoo.com OPT |

Also notice that there are 2 different transaction IDs for the packets: 0xde40 & 0xa620.

## 2.6.1.1 Normal DNS

### DNS Transaction ID:

“A 16-bit field identifying a specific DNS transaction. The transaction ID is created by the message originator and is copied by the responder into its response message. Using the transaction ID, the DNS client can match responses to its requests.” – [Microsoft Technet](https://technet.microsoft.com/en-us/library/dd197470(v=ws.10).aspx)

In the next slide, we'll look at the packets with transaction ID 0xde40.

## 2.6.1.1 Normal DNS

Below we can verify that this is a UDP packet and it's using an expected port, 53.

```
▶ Frame 16: 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface 0
▶ Ethernet II, Src: b2:fe:ed:db:02:32 (b2:fe:ed:db:02:32), Dst: Vmware_a1:a4:5f (00:50:56:a1:a4:5f)
▶ Internet Protocol Version 4, Src: 172.16.5.100, Dst: 172.16.5.10
▶ User Datagram Protocol, Src Port: 42653, Dst Port: 53
▼ Domain Name System (query)
    [Response In: 19]
    Transaction ID: 0xde40
    ▶ Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
    ▼ Queries
        ▼ 5.5.16.172.in-addr.arpa: type PTR, class IN
            Name: 5.5.16.172.in-addr.arpa
            [Name Length: 23]
0000  00 50 56 a1 a4 5f b2 fe  ed db 02 32 08 00 45 00  .PV... ..2..E.
0010  00 45 ae 11 00 00 40 11  6a 08 ac 10 05 64 ac 10  .E...@. j....d..
0020  05 0a a6 9d 00 35 00 31  2d 00 de 40 01 00 00 01  ....5.1 -..@....
0030  00 00 00 00 00 00 01 35  01 35 02 31 36 03 31 37  .....5 .5.16.17
0040  32 07 69 6e 2d 61 64 64  72 04 61 72 70 61 00 00  2.in-add r.arpa..
0050  0c 00 01                                     ...
```

## 2.6.1.1 Normal DNS

This is the **DNS Response** to the **DNS Query** in packet 16. All looks normal as far as the DNS protocol is concerned.

A question to ask yourself is - would this be a recon scan or normal behavior within your network?

```
▶ Frame 19: 127 bytes on wire (1016 bits), 127 bytes captured (1016 bits) on interface 0
▶ Ethernet II, Src: Vmware_a1:a4:5f (00:50:56:a1:a4:5f), Dst: b2:fe:ed:db:02:32 (b2:fe:ed:db:02:32)
▶ Internet Protocol Version 4, Src: 172.16.5.10, Dst: 172.16.5.100
▶ User Datagram Protocol, Src Port: 53, Dst Port: 42653
▼ Domain Name System (response)
  [Request In: 16]
  [Time: 0.072829293 seconds]
  Transaction ID: 0xde40
  ▶ Flags: 0x8580 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ 5.5.16.172.in-addr.arpa: type PTR, class IN
      Name: 5.5.16.172.in-addr.arpa
      [Name Length: 23]
      [Label Count: 6]
      Type: PTR (domain name PoinTeR) (12)
      Class: IN (0x0001)
  ▼ Answers
    ▼ 5.5.16.172.in-addr.arpa: type PTR, class IN, wkst-techsupport.sportsfoo.com
      Name: 5.5.16.172.in-addr.arpa
      Type: PTR (domain name PoinTeR) (12)
      Class: IN (0x0001)
      Time to live: 3600
      Data length: 32
      Domain Name: wkst-techsupport.sportsfoo.com
```

## 2.6.1.1 Normal DNS

Here is another packet capture displaying various DNS queries, but you can see the pattern of the DNS Query followed by the DNS Response.

| No. | Time       | Source         | Destination    | Protocol | Length | Info                                                                                         |
|-----|------------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------|
| 1   | 0.000000   | 192.168.170.8  | 192.168.170.20 | DNS      | 70     | Standard query 0x1032 TXT google.com                                                         |
| 2   | 0.000530   | 192.168.170.20 | 192.168.170.8  | DNS      | 98     | Standard query response 0x1032 TXT google.com TXT                                            |
| 3   | 4.005222   | 192.168.170.8  | 192.168.170.20 | DNS      | 70     | Standard query 0xf76f MX google.com                                                          |
| 4   | 4.837355   | 192.168.170.20 | 192.168.170.8  | DNS      | 298    | Standard query response 0xf76f MX google.com MX 40 smtp4.google.com MX 10 smtp5.google.com.  |
| 5   | 12.817185  | 192.168.170.8  | 192.168.170.20 | DNS      | 70     | Standard query 0x49a1 LOC google.com                                                         |
| 6   | 12.956209  | 192.168.170.20 | 192.168.170.8  | DNS      | 70     | Standard query response 0x49a1 LOC google.com                                                |
| 7   | 20.824827  | 192.168.170.8  | 192.168.170.20 | DNS      | 85     | Standard query 0x9bbb PTR 104.9.192.66.in-addr.arpa                                          |
| 8   | 20.825333  | 192.168.170.20 | 192.168.170.8  | DNS      | 129    | Standard query response 0x9bbb PTR 104.9.192.66.in-addr.arpa PTR 66-192-9-104.gen.twteleco.. |
| 9   | 92.189905  | 192.168.170.8  | 192.168.170.20 | DNS      | 74     | Standard query 0x75c0 A www.netbsd.org                                                       |
| 10  | 92.238816  | 192.168.170.20 | 192.168.170.8  | DNS      | 90     | Standard query response 0x75c0 A www.netbsd.org A 204.152.190.12                             |
| 11  | 108.965135 | 192.168.170.8  | 192.168.170.20 | DNS      | 74     | Standard query 0xf0d4 AAAA www.netbsd.org                                                    |
| 12  | 109.202803 | 192.168.170.20 | 192.168.170.8  | DNS      | 102    | Standard query response 0xf0d4 AAAA www.netbsd.org AAAA 2001:4f8:4:7:2e0:81ff:fe52:9a6b      |
| 13  | 169.027394 | 192.168.170.8  | 192.168.170.20 | DNS      | 74     | Standard query 0x7f39 AAAA www.netbsd.org                                                    |
| 14  | 169.027781 | 192.168.170.20 | 192.168.170.8  | DNS      | 102    | Standard query response 0x7f39 AAAA www.netbsd.org AAAA 2001:4f8:4:7:2e0:81ff:fe52:9a6b      |

Frame 1: 70 bytes on wire (560 bits), 70 bytes captured (560 bits)  
Ethernet II, Src: AsustekI\_b1:0c:ad (00:e0:18:b1:0c:ad), Dst: QuantaCo\_32:41:8c (00:c0:9f:32:41:8c)  
Internet Protocol Version 4, Src: 192.168.170.8, Dst: 192.168.170.20  
User Datagram Protocol, Src Port: 32795, Dst Port: 53  
Domain Name System (query)

\*PCAP from Wireshark Sample Captures [page](#).

## 2.6.1.2 Suspicious DNS

Now we'll look at an example of suspicious DNS traffic using Wireshark.

Remember the tips regarding normal and suspicious DNS traffic:

- **Normal:** Port 53, UDP.
- **Suspicious:** Traffic on port 53 but using TCP instead of UDP.

## 2.6.1.2 Suspicious DNS

Here we see 3 packets and all packets share 1 transaction ID. We also see some **AXFR**, which let's us know that this DNS Query is pertaining to a DNS Zone Transfer. If you are not familiar with DNS Zone Transfers, read [here](#).

|    |              |              |              |     |                                                                                          |
|----|--------------|--------------|--------------|-----|------------------------------------------------------------------------------------------|
| 60 | 82.399428543 | 172.16.5.100 | 172.16.5.10  | DNS | 110 Standard query 0xfc66 AXFR sportsfoo.com OPT                                         |
| 61 | 82.434731625 | 172.16.5.10  | 172.16.5.100 | DNS | 172 Standard query response 0xfc66 AXFR sportsfoo.com SOA els-wins2003.sportsfoo.com OPT |
| 63 | 82.469319332 | 172.16.5.10  | 172.16.5.100 | DNS | 598 Standard query response 0xfc66 AXFR sportsfoo.com SOA els-wins2003.sportsfoo.com     |

Let's take a closer look at the packet details for this zone transfer.

## 2.6.1.2 Suspicious DNS

Here we see details of packet 60, which is a zone transfer. We also see that instead of using **UDP/53** this DNS Query used **TCP/53**. Also, in case you didn't know that AXFR referred to a zone transfer, the clue is within the packet details.

So why was TCP used instead of UDP? This occurs when the response to a DNS query is too large to fit within a single UDP packet. So, the query is resubmitted via TCP to retrieve the entire contents of the response. DNS Responses for zone transfer occur over TCP/53.

```
▶ Frame 60: 110 bytes on wire (880 bits), 110 bytes captured (880 bits) on interface 0
▶ Ethernet II, Src: b2:fe:ed:db:02:32 (b2:fe:ed:db:02:32), Dst: Vmware_a1:a4:5f (00:50:56:a1:a4:5f)
▶ Internet Protocol Version 4, Src: 172.16.5.100, Dst: 172.16.5.10
▶ Transmission Control Protocol, Src Port: 5859, Dst Port: 53, Seq: 1, Ack: 1, Len: 44
▼ Domain Name System (query)
  [Response In: 63]
  Length: 42
  Transaction ID: 0xfc66
  ▶ Flags: 0x0020 Standard query
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 1
  ▼ Queries
    ▼ sportsfoo.com: type AXFR, class IN
      Name: sportsfoo.com
      [Name Length: 13]
      [Label Count: 2]
      Type: AXFR (transfer of an entire zone) (252)
      Class: IN (0x0001)
  ▶ Additional records
```

## 2.6.1.2 Suspicious DNS

The image on the right displays the response to the DNS query on the previous slide.

```
▶ Frame 61: 172 bytes on wire (1376 bits), 172 bytes captured (1376 bits) on interface 0
▶ Ethernet II, Src: Vmware_a1:a4:5f (00:50:56:a1:a4:5f), Dst: b2:fe:ed:db:02:32 (b2:fe:ed:db:02:32)
▶ Internet Protocol Version 4, Src: 172.16.5.10, Dst: 172.16.5.100
▶ Transmission Control Protocol, Src Port: 53, Dst Port: 58595, Seq: 1, Ack: 45, Len: 106
▼ Domain Name System (response)
  [Request In: 60]
  [Time: 0.035303082 seconds]
  Length: 104
  Transaction ID: 0xfc66
  ▶ Flags: 0x8080 Standard query response, No error
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 1
  ▼ Queries
    ▼ sportsfoo.com: type AXFR, class IN
      Name: sportsfoo.com
      [Name Length: 13]
      [Label Count: 2]
      Type: AXFR (transfer of an entire zone) (252)
      Class: IN (0x0001)
  ▼ Answers
    ▼ sportsfoo.com: type SOA, class IN, mname els-winsner2003.sportsfoo.com
      Name: sportsfoo.com
      Type: SOA (Start Of a zone of Authority) (6)
      Class: IN (0x0001)
      Time to live: 3600
      Data length: 50
      Primary name server: els-winsner2003.sportsfoo.com
      Responsible authority's mailbox: hostmaster.sportsfoo.com
      Serial Number: 19
      Refresh Interval: 900 (15 minutes)
      Retry Interval: 600 (10 minutes)
      Expire limit: 86400 (1 day)
      Minimum TTL: 3600 (1 hour)
  ▶ Additional records
```

## 2.6.1.2 Suspicious DNS

DNS over TCP should be considered suspicious. If your organization is not blocking TCP/53, then it should at least be monitored; this can potentially hide malicious traffic in plain sight.

There are other techniques in which DNS can be used for nefarious purposes, such as **DNS tunnels**. We need to remember the basic behavior of DNS and know when to investigate and/or when to ignore DNS traffic.

```
23  * @param {Object} experiment - the experiment data object to use
24  * @param {Array} observations - an array of observations, in the format
25  * @param {Object} control - the control observation
26  *
27  * @return {Object} the result of the experiment
28  *
29  * @example
30  * let initialize(experiment, observations = [], control = null)
31  *
32  * @example
33  * let experiment = {
34  *   @experiment : experiment
35  *   @observations : observations
36  *   @control : control
37  *   @candidates : observations.concat(control)
38  *   evaluate_candidates
39  * }
40  *
41  * freeze
42  *
43  * @example
44  * let experiment = {
45  *   @experiment : experiment
46  *   @observations : observations
47  *   @control : control
48  *   @candidates : observations.concat(control)
49  *   evaluate_candidates
50  * }
```



## 2.6.2 Wireshark Tip

For more information on techniques to filter DNS traffic, refer to the Wireshark Display Filter Reference for DNS, [here](https://www.wireshark.org/docs/dfref/d/dns.html).

# HTTP & HTTPS Traffic



## 2.7 HTTP & HTTPS Traffic

Let's begin by looking at HTTP and HTTPS traffic.

Even though we should be familiar with what is normal HTTP and HTTPS traffic, as we probably use the Internet daily and browse numerous websites, it should still be covered.

```
20 with open('data/experiment1.json', 'r') as f:
21     data = json.load(f)
22
23     # Create the experiment object
24     experiment = Experiment(
25         name=data['name'],
26         observations=data['observations'],
27         candidates=data['candidates'],
28         control=data['control']
29     )
30
31     # Initialize the experiment
32     experiment.initialize()
33
34     # Evaluate the experiment
35     experiment.evaluate()
36
37     # Print the results
38     print('Experiment results:')
39     print('Name: %s' % experiment.name)
40     print('Observations: %s' % experiment.observations)
41     print('Candidates: %s' % experiment.candidates)
42     print('Control: %s' % experiment.control)
43
44     # Print the results of the experiment
45     print('Experiment results:')
46     print('Name: %s' % experiment.name)
47     print('Observations: %s' % experiment.observations)
48     print('Candidates: %s' % experiment.candidates)
49     print('Control: %s' % experiment.control)
50     print('Results: %s' % experiment.results)
```



## 2.7.1 HTTP

A few things to remember about HTTP:

- HTTP traffic consists of a series of requests and responses known as **messages**.
  - The client will make a request and the server will respond.
  - HTTP responses include a **3-digit status code**.
- HTTP messages include a **message header** and **body**.
- HTTP uses **methods** to perform various operations.
  - 8 HTTP methods are defined in [RFC 2616](https://www.rfc-editor.org/rfc/rfc2616).
  - Not all methods will be permitted by the web server.

## 2.7.1 HTTP

### Some facts to help distinguish normal and suspicious HTTP traffic.

| Normal HTTP Traffic                                                                      | Suspicious HTTP Traffic                                                                                                                          |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Port 80, TCP<br>Port 8080, TCP (used as alternate)<br>Port 8088, TCP (used as alternate) | Malicious binaries (backdoors), scripts, web shells, etc. will use this port because typically, in all corporate environments, the port is open. |
| Plaintext traffic                                                                        | If the traffic is encrypted, then most likely it's being used for malicious traffic. Malicious traffic can be in plaintext as well.              |
| Web server typically in FQDN format.                                                     | The server will point to an IP address instead of FQDN format.                                                                                   |

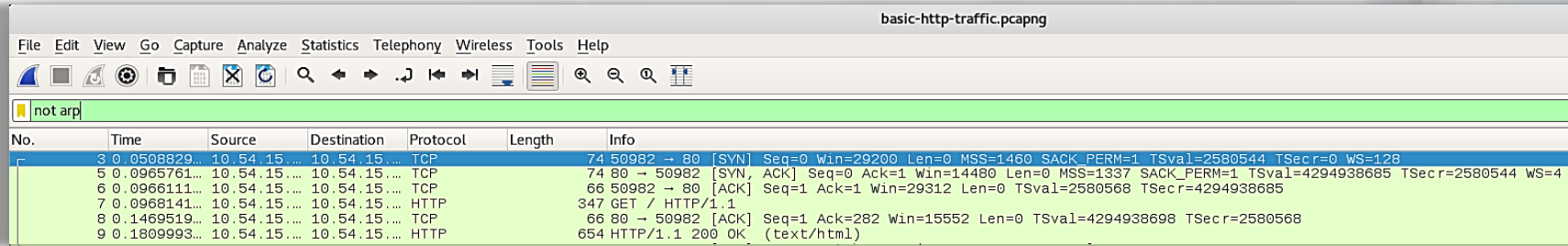
## 2.7.1 HTTP

Now, let's look at 2 basic PCAPs with some HTTP traffic.

We will use some display filters and introduce some other techniques within Wireshark that you might not be familiar with to aid us in the hunt.



## 2.7.1.1 Normal HTTP



The image shows a Wireshark packet capture window titled "basic-http-traffic.pcapng". The filter bar shows "not arp". The packet list shows 9 packets. Packets 3-6 are TCP packets forming a handshake. Packets 7 and 9 are HTTP packets (GET and 200 OK).

| No. | Time         | Source      | Destination | Protocol | Length | Info                                                                                                       |
|-----|--------------|-------------|-------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 3   | 0.0508829... | 10.54.15... | 10.54.15... | TCP      | 74     | 50982 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2580544 TSecr=0 WS=128                   |
| 5   | 0.0965761... | 10.54.15... | 10.54.15... | TCP      | 74     | 80 → 50982 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294938685 TSecr=2580544 WS=4 |
| 6   | 0.0966111... | 10.54.15... | 10.54.15... | TCP      | 66     | 50982 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2580568 TSecr=4294938685                                |
| 7   | 0.0968141... | 10.54.15... | 10.54.15... | HTTP     | 347    | GET / HTTP/1.1                                                                                             |
| 8   | 0.1469519... | 10.54.15... | 10.54.15... | TCP      | 66     | 80 → 50982 [ACK] Seq=1 Ack=282 Win=15552 Len=0 TSval=4294938698 TSecr=2580568                              |
| 9   | 0.1809993... | 10.54.15... | 10.54.15... | HTTP     | 654    | HTTP/1.1 200 OK (text/html)                                                                                |

Here is a brief summary of the above image:

- We are seeing 6 packets (4 relating to TCP and 2 on HTTP).
- Packets 3-6 is the **TCP Handshake**. HTTP relies on TCP for reliability.
- In packet 7, we notice an **HTTP method** (GET).
- In packet 9, we see an **HTTP response code** (200 OK).

## 2.7.1.1 Normal HTTP

| No.                                                                                                 | Time         | Source      | Destination | Protocol | Length | Info                                                                                                       |
|-----------------------------------------------------------------------------------------------------|--------------|-------------|-------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 3                                                                                                   | 0.0508829... | 10.54.15... | 10.54.15... | TCP      | 74     | 50982 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2580544 TSecr=0 WS=128                   |
| 5                                                                                                   | 0.0965761... | 10.54.15... | 10.54.15... | TCP      | 74     | 80 → 50982 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294938685 TSecr=2580544 WS=4 |
| 6                                                                                                   | 0.0966111... | 10.54.15... | 10.54.15... | TCP      | 66     | 50982 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2580568 TSecr=4294938685                                |
| 7                                                                                                   | 0.0968141... | 10.54.15... | 10.54.15... | HTTP     | 347    | GET / HTTP/1.1                                                                                             |
| 8                                                                                                   | 0.1469519... | 10.54.15... | 10.54.15... | TCP      | 66     | 80 → 50982 [ACK] Seq=1 Ack=282 Win=15552 Len=0 TSval=4294938698 TSecr=2580568                              |
| 9                                                                                                   | 0.1809993... | 10.54.15... | 10.54.15... | HTTP     | 654    | HTTP/1.1 200 OK (text/html)                                                                                |
| ▶ Frame 7: 347 bytes on wire (2776 bits), 347 bytes captured (2776 bits) on interface 0             |              |             |             |          |        |                                                                                                            |
| ▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:61:66 (00:50:56:a1:61:66) |              |             |             |          |        |                                                                                                            |
| ▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68                                  |              |             |             |          |        |                                                                                                            |
| ▶ Transmission Control Protocol, Src Port: 50982, Dst Port: 80, Seq: 1, Ack: 1, Len: 281            |              |             |             |          |        |                                                                                                            |
| ▶ Hypertext Transfer Protocol                                                                       |              |             |             |          |        |                                                                                                            |

In the image above, we see the packet information as it pertains to the different layers of the OSI model.

- Ethernet II = MAC Addresses (SRC, DST)
- Internet Protocol Version 4 = IP Address (SRC, DST)
- Transmission Control Protocol = TCP (SRC PORT, DST PORT)
- Hypertext Transfer Protocol = HTTP

## 2.7.1.1 Normal HTTP

| No. | Time         | Source      | Destination | Protocol | Length | Info                                                                                                       |
|-----|--------------|-------------|-------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 3   | 0.0508829... | 10.54.15... | 10.54.15... | TCP      | 74     | 50982 → 80 [S/N] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2580544 TSecr=0 WS=128                   |
| 5   | 0.0965761... | 10.54.15... | 10.54.15... | TCP      | 74     | 80 → 50982 [S/N, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294938685 TSecr=2580544 WS=4 |
| 6   | 0.0966111... | 10.54.15... | 10.54.15... | TCP      | 66     | 50982 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2580568 TSecr=4294938685                                |
| 7   | 0.0968141... | 10.54.15... | 10.54.15... | HTTP     | 347    | GET / HTTP/1.1                                                                                             |
| 8   | 0.1469519... | 10.54.15... | 10.54.15... | TCP      | 66     | 80 → 50982 [ACK] Seq=1 Ack=282 Win=15552 Len=0 TSval=4294938698 TSecr=2580568                              |
| 9   | 0.1809993... | 10.54.15... | 10.54.15... | HTTP     | 654    | HTTP/1.1 200 OK (text/html)                                                                                |

▶ Frame 7: 347 bytes on wire (2776 bits), 347 bytes captured (2776 bits) on interface 0  
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware\_al:61:66 (00:50:56:a1:61:66)  
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68  
▶ Transmission Control Protocol, Src Port: 50982, Dst Port: 80, Seq: 1, Ack: 1, Len: 281  
▶ Hypertext Transfer Protocol

So far, we can tell that the traffic is using an expected port, port 80. Remember that this port doesn't necessarily have to be port 80. An organization can decide to use a different port altogether, but we know the standard port is 80, and if it's port 80 then we're expecting web traffic.

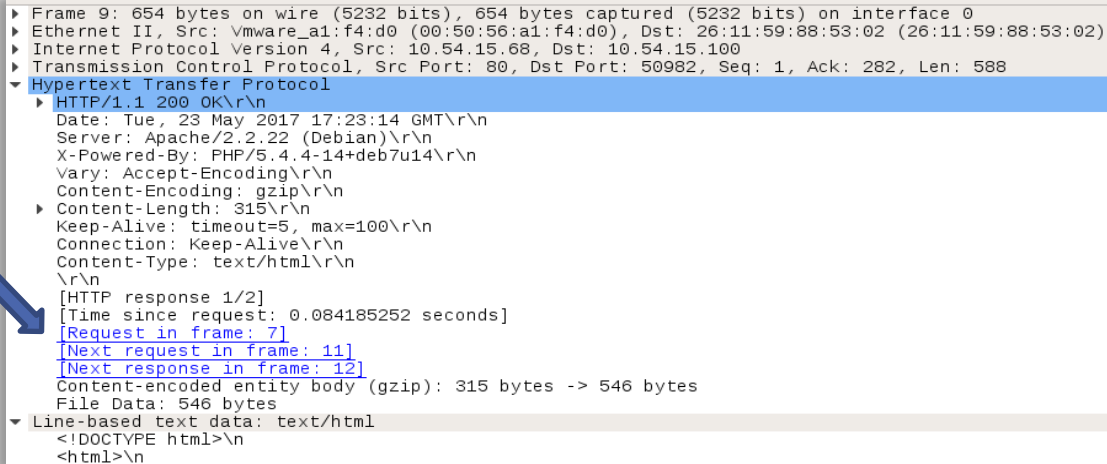
## 2.7.1.1 Normal HTTP

Here we are looking at the HTTP section of the packet details. The HTTP method is **GET** and the host is *not* in FQDN format but an IP address. Typically, direct connection to IP addresses would be suspicious but, in this case, it's pointing to an internal server and that is normal behavior in corporate environments. The source host has an IP of 10.54.15.100.

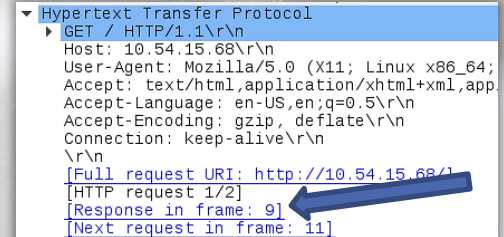
```
▶ Frame 7: 347 bytes on wire (2776 bits), 347 bytes captured (2776 bits) on interface 0
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:61:66 (00:50:56:a1:61:66)
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68
▶ Transmission Control Protocol, Src Port: 50982, Dst Port: 80, Seq: 1, Ack: 1, Len: 281
▶ Hypertext Transfer Protocol
  ▶ GET / HTTP/1.1\r\n
    Host: 10.54.15.68\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://10.54.15.68/]
    [HTTP request 1/2]
    [Response in frame: 9]
    [Next request in frame: 11]
```

## 2.7.1.1 Normal HTTP

If we double click the link **[Response in frame: 9]** in the HTTP section of the packet details of packet 7, we'll jump to packet 9, which contains the packet details of the **HTTP response** to the **HTTP request**. Within this packet, we can do the same jump back to packet 7 using the link **[Request in frame: 7]**.



▶ Frame 9: 654 bytes on wire (5232 bits), 654 bytes captured (5232 bits) on interface 0  
▶ Ethernet II, Src: Vmware\_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)  
▶ Internet Protocol Version 4, Src: 10.54.15.68, Dst: 10.54.15.100  
▶ Transmission Control Protocol, Src Port: 80, Dst Port: 50982, Seq: 1, Ack: 282, Len: 588  
▼ Hypertext Transfer Protocol  
    ▶ HTTP/1.1 200 OK\r\n  
        Date: Tue, 23 May 2017 17:23:14 GMT\r\n  
        Server: Apache/2.2.22 (Debian)\r\n  
        X-Powered-By: PHP/5.4.4-14+deb7u14\r\n  
        Vary: Accept-Encoding\r\n  
        Content-Encoding: gzip\r\n  
        Content-Length: 315\r\n  
        Keep-Alive: timeout=5, max=100\r\n  
        Connection: Keep-Alive\r\n  
        Content-Type: text/html\r\n  
        \r\n  
        [HTTP response 1/2]  
        [Time since request: 0.084185252 seconds]  
        [\[Request in frame: 7\]](#)  
        [\[Next request in frame: 11\]](#)  
        [\[Next response in frame: 12\]](#)  
        Content-encoded entity body (gzip): 315 bytes -> 546 bytes  
        File Data: 546 bytes  
▼ Line-based text data: text/html  
    <!DOCTYPE html>\r\n  
    <html>\r\n



▼ Hypertext Transfer Protocol  
    ▶ GET / HTTP/1.1\r\n  
        Host: 10.54.15.68\r\n  
        User-Agent: Mozilla/5.0 (X11; Linux x86\_64;  
        Accept: text/html,application/xhtml+xml,application/javascript;q=0.9,\*/\*;q=0.8\r\n  
        Accept-Language: en-US,en;q=0.5\r\n  
        Accept-Encoding: gzip, deflate\r\n  
        Connection: keep-alive\r\n  
        \r\n  
        [Full request URI: http://10.54.15.68/]  
        [HTTP request 1/2]  
        [\[Response in frame: 9\]](#)  
        [\[Next request in frame: 11\]](#)

## 2.7.1.1 Normal HTTP

Here we can see a section called **Line-based text data: text/html** within the packet details.

We can clearly see that the packet is returning a login page for the server.

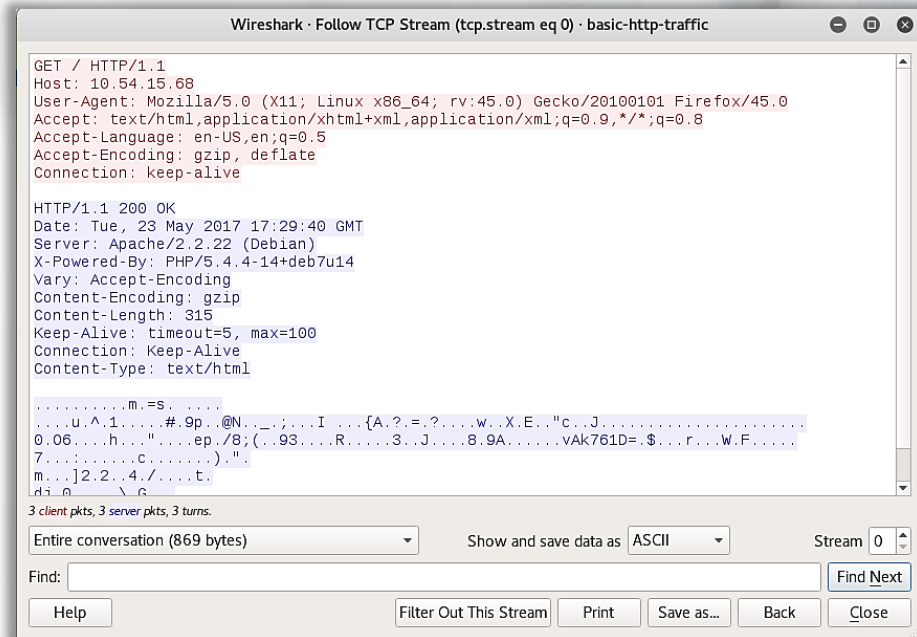
```
▼ Line-based text data: text/html
<!DOCTYPE html>\n
<html>\n
\n
<head>\n
\n
  <meta charset="UTF-8">\n
\n
  <title>Log-in - login.labs</title>\n
\n
    <link rel="stylesheet" href="css/style.css" media="screen" type="text/css" />\n
\n
</head>\n
\n
<body>\n
\n
  <div class="login-card">\n
    <h1>Log-in</h1><br>\n
    <form method="post" action="login.php">\n
      <input type="text" name="user" autocomplete="off" placeholder="Username">\n
      <input type="password" name="pass" placeholder="Password">\n
      <input type="submit" name="login" class="login login-submit" value="login">\n
    </form>\n
  </div>\n
\n
</body>\n
\n
</html>\n
```

## 2.7.1.1 Normal HTTP

The same information can be obtained by right-clicking packet 7 or packet 9 and selecting **Follow > Follow TCP Stream**.

Here we see the same details, but only information from the packet that pertains to HTTP.

Remember that HTTP traffic is supposed to be in cleartext, which we have seen so far. Now let's look at the packet stream after logging into the internal server to see credentials in cleartext.



## 2.7.1.1 Normal HTTP

Before looking at cleartext credentials, let's test our packet analysis skills. How many TCP streams are there in the packet capture below?

| No. | Time         | Source                         | Destination       | Protocol | Length | Info                                                                                                       |
|-----|--------------|--------------------------------|-------------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000000  | 26:11:59:88:53:02              | Broadcast         | ARP      | 42     | Who has 10.54.15.68? Tell 10.54.15.100                                                                     |
| 2   | 0.050864169  | Vmware_a1:61:66                | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:61:66                                                                        |
| 3   | 0.050882951  | 10.54.15.100                   | 10.54.15.68       | TCP      | 74     | 50982 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2580544 TSecr=0 WS=128                   |
| 4   | 0.050906984  | Vmware_a1:f4:d0                | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:f4:d0                                                                        |
| 5   | 0.096576117  | 10.54.15.68                    | 10.54.15.100      | TCP      | 74     | 80 → 50982 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294938685 TSecr=2580544 WS=4 |
| 6   | 0.096611102  | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2580568 TSecr=4294938685                                |
| 7   | 0.096814120  | 10.54.15.100                   | 10.54.15.68       | HTTP     | 347    | GET / HTTP/1.1                                                                                             |
| 8   | 0.146951934  | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 50982 [ACK] Seq=1 Ack=282 Win=15552 Len=0 TSval=4294938698 TSecr=2580568                              |
| 9   | 0.180999372  | 10.54.15.68                    | 10.54.15.100      | HTTP     | 654    | HTTP/1.1 200 OK (text/html)                                                                                |
| 10  | 0.181016327  | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=282 Ack=589 Win=30464 Len=0 TSval=2580589 TSecr=4294938706                            |
| 11  | 0.212109429  | 10.54.15.100                   | 10.54.15.68       | HTTP     | 345    | GET /css/style.css HTTP/1.1                                                                                |
| 12  | 0.286521761  | 10.54.15.100                   | 10.54.15.68       | HTTP     | 1242   | HTTP/1.1 200 OK (text/css)                                                                                 |
| 13  | 0.327365188  | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=561 Ack=1765 Win=33280 Len=0 TSval=2580626 TSecr=4294938732                           |
| 14  | 0.290026896  | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 50982 [FIN, ACK] Seq=1765 Ack=561 Win=16624 Len=0 TSval=4294939983 TSecr=2580626                      |
| 15  | 0.290130606  | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 50982 → 80 [FIN, ACK] Seq=561 Ack=1766 Win=33280 Len=0 TSval=2581866 TSecr=4294939983                      |
| 16  | 0.335860407  | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 50982 [ACK] Seq=1766 Ack=562 Win=16624 Len=0 TSval=4294939995 TSecr=2581866                           |
| 17  | 18.175909185 | fe80::2411:59ff:fe8... ff02::2 |                   | ICMPv6   | 70     | Router Solicitation from 26:11:59:88:53:02                                                                 |
| 18  | 20.862049221 | 10.54.15.100                   | 10.54.15.68       | TCP      | 74     | 51008 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2585759 TSecr=0 WS=128                   |
| 19  | 20.907960258 | 10.54.15.68                    | 10.54.15.100      | TCP      | 74     | 80 → 51008 [ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294943888 TSecr=2585759 WS=4      |
| 20  | 20.907988940 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51008 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2585771 TSecr=4294943888                                |
| 21  | 20.908172432 | 10.54.15.100                   | 10.54.15.68       | HTTP     | 358    | GET /favicon.ico HTTP/1.1                                                                                  |
| 22  | 20.955920739 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51008 [ACK] Seq=1 Ack=293 Win=15552 Len=0 TSval=4294943900 TSecr=2585771                              |
| 23  | 20.957862825 | 10.54.15.68                    | 10.54.15.100      | HTTP     | 568    | HTTP/1.1 404 Not Found (text/html)                                                                         |
| 24  | 20.957877026 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51008 → 80 [ACK] Seq=293 Ack=503 Win=30336 Len=0 TSval=2585783 TSecr=4294943900                            |
| 25  | 25.062862870 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51008 [FIN, ACK] Seq=503 Ack=293 Win=15552 Len=0 TSval=4294945152 TSecr=2585783                       |
| 26  | 25.062789254 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51008 → 80 [FIN, ACK] Seq=293 Ack=504 Win=30336 Len=0 TSval=2587034 TSecr=4294945152                       |
| 27  | 26.008812475 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51008 [ACK] Seq=504 Ack=294 Win=15552 Len=0 TSval=4294945163 TSecr=2587034                            |
| 28  | 42.917398520 | 10.54.15.100                   | 10.54.15.68       | TCP      | 74     | 51012 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2591273 TSecr=0 WS=128                   |
| 29  | 42.969116869 | 10.54.15.68                    | 10.54.15.100      | TCP      | 74     | 80 → 51012 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294949402 TSecr=2591273 WS=4 |
| 30  | 42.969140002 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51012 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2591286 TSecr=4294949402                                |
| 31  | 42.969353931 | 10.54.15.100                   | 10.54.15.68       | HTTP     | 501    | POST /login.php HTTP/1.1 (application/x-www-form-urlencoded)                                               |
| 32  | 43.016754695 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51012 [ACK] Seq=1 Ack=436 Win=15552 Len=0 TSval=4294949415 TSecr=2591286                              |
| 33  | 43.033810917 | 10.54.15.68                    | 10.54.15.100      | HTTP     | 604    | HTTP/1.1 200 OK (text/html)                                                                                |
| 34  | 43.033923724 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51012 → 80 [ACK] Seq=436 Ack=539 Win=30336 Len=0 TSval=2591302 TSecr=4294949420                            |
| 35  | 48.040277295 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51012 [FIN, ACK] Seq=539 Ack=436 Win=15552 Len=0 TSval=4294950671 TSecr=2591302                       |
| 36  | 48.040506072 | 10.54.15.100                   | 10.54.15.68       | TCP      | 66     | 51012 → 80 [FIN, ACK] Seq=436 Ack=540 Win=30336 Len=0 TSval=2592554 TSecr=4294950671                       |
| 37  | 48.088082920 | 10.54.15.68                    | 10.54.15.100      | TCP      | 66     | 80 → 51012 [ACK] Seq=540 Ack=437 Win=15552 Len=0 TSval=4294950683 TSecr=2592554                            |
| 38  | 91.903620490 | fe80::2411:59ff:fe8... ff02::2 |                   | ICMPv6   | 70     | Router Solicitation from 26:11:59:88:53:02                                                                 |

## 2.7.1.1 Normal HTTP

There are a couple ways to determine this within the Wireshark GUI:

1. Look at the TCP packets and then look at source ports and destination ports.
2. Use display filters and increment the numbers to show a particular TCP stream until nothing is displayed.
3. Select **Statistics > Conversations** and then click the **TCP tab**.
4. In the Follow TCP Stream window, at the bottom right, you can navigate through the different streams that Wireshark detected.

## 2.7.1.1 Normal HTTP

1. From the packet capture, we can see different source ports. So, each different source port will have its own TCP Handshake followed by its corresponding traffic.

| No. | Time         | Source                   | Destination       | Protocol | Length | Info                                                                                                       |
|-----|--------------|--------------------------|-------------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000000  | 26:11:59:88:53:02        | Broadcast         | ARP      | 42     | Who has 10.54.15.68? Tell 10.54.15.100                                                                     |
| 2   | 0.0508864169 | Vmware_a1:61:66          | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:61:66                                                                        |
| 3   | 0.050882951  | 10.54.15.100             | 10.54.15.68       | TCP      | 74     | 50982 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2580544 TSecr=0 WS=128                   |
| 4   | 0.050906994  | Vmware_a1:f4:d0          | 26:11:59:88:53:02 | ARP      | 60     | 10.54.15.68 is at 00:50:56:a1:f4:d0                                                                        |
| 5   | 0.0965761117 | 10.54.15.68              | 10.54.15.100      | TCP      | 74     | 80 → 50982 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294938685 TSecr=2580544 WS=4 |
| 6   | 0.096511102  | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2580568 TSecr=4294938685                                |
| 7   | 0.096814120  | 10.54.15.68              | 10.54.15.68       | HTTP     | 347    | GET / HTTP/1.1                                                                                             |
| 8   | 0.146951934  | 10.54.15.100             | 10.54.15.100      | TCP      | 66     | 80 → 50982 [ACK] Seq=1 Ack=282 Win=15552 Len=0 TSval=4294938698 TSecr=2580568                              |
| 9   | 0.180999372  | 10.54.15.68              | 10.54.15.100      | HTTP     | 654    | HTTP/1.1 200 OK (text/html)                                                                                |
| 10  | 0.181016327  | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=282 Ack=589 Win=30464 Len=0 TSval=2580589 TSecr=4294938706                            |
| 11  | 0.212109429  | 10.54.15.100             | 10.54.15.68       | HTTP     | 345    | GET /css/style.css HTTP/1.1                                                                                |
| 12  | 0.286521761  | 10.54.15.68              | 10.54.15.100      | HTTP     | 1242   | HTTP/1.1 200 OK (text/css)                                                                                 |
| 13  | 0.327365188  | 10.54.15.68              | 10.54.15.68       | TCP      | 66     | 50982 → 80 [ACK] Seq=561 Ack=1765 Win=33280 Len=0 TSval=2580626 TSecr=4294938732                           |
| 14  | 5.290026896  | 10.54.15.100             | 10.54.15.100      | TCP      | 66     | 80 → 50982 [FIN, ACK] Seq=1765 Ack=561 Win=16624 Len=0 TSval=4294939983 TSecr=2580626                      |
| 15  | 5.290130606  | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 50982 → 80 [FIN, ACK] Seq=561 Ack=1766 Win=33280 Len=0 TSval=2581866 TSecr=4294939983                      |
| 16  | 5.335860407  | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 50982 [ACK] Seq=1766 Ack=562 Win=16624 Len=0 TSval=4294939995 TSecr=2581866                           |
| 17  | 18.175909185 | fe80::2411:59ff:fe80::12 | ff02::2           | ICMPv6   | 70     | Router Solicitation from 26:11:59:88:53:02                                                                 |
| 18  | 20.862049221 | 10.54.15.100             | 10.54.15.68       | TCP      | 74     | 51008 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2585759 TSecr=0 WS=128                   |
| 19  | 20.907960258 | 10.54.15.68              | 10.54.15.100      | TCP      | 74     | 80 → 51008 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294943888 TSecr=2585759 WS=4 |
| 20  | 20.907988940 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51008 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2585771 TSecr=4294943888                                |
| 21  | 20.908172432 | 10.54.15.100             | 10.54.15.68       | HTTP     | 358    | GET /favicon.ico HTTP/1.1                                                                                  |
| 22  | 20.955920739 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51008 [ACK] Seq=1 Ack=293 Win=15552 Len=0 TSval=4294943900 TSecr=2585771                              |
| 23  | 20.957862825 | 10.54.15.68              | 10.54.15.100      | HTTP     | 568    | HTTP/1.1 404 Not Found (text/html)                                                                         |
| 24  | 20.957877026 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51008 → 80 [ACK] Seq=293 Ack=503 Win=30336 Len=0 TSval=2585783 TSecr=4294943900                            |
| 25  | 25.962682870 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51008 [FIN, ACK] Seq=503 Ack=293 Win=15552 Len=0 TSval=4294945152 TSecr=2585783                       |
| 26  | 25.962789254 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51008 → 80 [FIN, ACK] Seq=293 Ack=504 Win=30336 Len=0 TSval=2587034 TSecr=4294945152                       |
| 27  | 26.008812475 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51008 [ACK] Seq=504 Ack=294 Win=15552 Len=0 TSval=4294945163 TSecr=2587034                            |
| 28  | 42.016983520 | 10.54.15.100             | 10.54.15.68       | TCP      | 74     | 51012 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2591273 TSecr=0 WS=128                   |
| 29  | 42.060116869 | 10.54.15.68              | 10.54.15.100      | TCP      | 74     | 80 → 51012 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=4294949402 TSecr=2591273 WS=4 |
| 30  | 42.069140002 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51012 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2591286 TSecr=4294949402                                |
| 31  | 42.069353931 | 10.54.15.68              | 10.54.15.68       | HTTP     | 501    | POST /login.php HTTP/1.1 (application/x-www-form-urlencoded)                                               |
| 32  | 43.016754695 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51012 [ACK] Seq=1 Ack=436 Win=15552 Len=0 TSval=4294949415 TSecr=2591286                              |
| 33  | 43.033810817 | 10.54.15.68              | 10.54.15.100      | HTTP     | 604    | HTTP/1.1 200 OK (text/html)                                                                                |
| 34  | 43.033823724 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51012 → 80 [ACK] Seq=436 Ack=539 Win=30336 Len=0 TSval=2591302 TSecr=4294949420                            |
| 35  | 48.040277295 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51012 [FIN, ACK] Seq=539 Ack=436 Win=15552 Len=0 TSval=4294950671 TSecr=2591302                       |
| 36  | 48.040506072 | 10.54.15.100             | 10.54.15.68       | TCP      | 66     | 51012 → 80 [FIN, ACK] Seq=436 Ack=540 Win=30336 Len=0 TSval=2592554 TSecr=4294950671                       |
| 37  | 48.088082920 | 10.54.15.68              | 10.54.15.100      | TCP      | 66     | 80 → 51012 [ACK] Seq=540 Ack=437 Win=15552 Len=0 TSval=4294950683 TSecr=2592554                            |
| 38  | 91.903620490 | fe80::2411:59ff:fe80::12 | ff02::2           | ICMPv6   | 70     | Router Solicitation from 26:11:59:88:53:02                                                                 |

## 2.7.1.1 Normal HTTP

2. Another way this can be done is by using a display filter and increment the number of the TCP Stream. You will know it's not a valid TCP Stream because there will be no visible packets.

| tcp.stream eq 0 |             |              |              |          |        |                             |
|-----------------|-------------|--------------|--------------|----------|--------|-----------------------------|
| No.             | Time        | Source       | Destination  | Protocol | Length | Info                        |
| 3               | 0.050882951 | 10.54.15.100 | 10.54.15.68  | TCP      | 74     | 50982 → 80 [SYN] Seq=0      |
| 5               | 0.096576117 | 10.54.15.68  | 10.54.15.100 | TCP      | 74     | 80 → 50982 [SYN, ACK] Seq=1 |
| 6               | 0.096611102 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 50982 → 80 [ACK] Seq=1      |
| 7               | 0.096614122 | 10.54.15.100 | 10.54.15.68  | HTTP     | 347    | GET / HTTP/1.1              |
| 8               | 0.146951934 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 50982 [ACK] Seq=1      |
| 9               | 0.180999372 | 10.54.15.68  | 10.54.15.100 | HTTP     | 654    | HTTP/1.1 200 OK (text/      |
| 10              | 0.181016327 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 50982 → 80 [ACK] Seq=28     |
| 11              | 0.212109429 | 10.54.15.100 | 10.54.15.68  | HTTP     | 345    | GET /css/style.css HTTP     |
| 12              | 0.286521761 | 10.54.15.68  | 10.54.15.100 | HTTP     | 1242   | HTTP/1.1 200 OK (text/      |
| 13              | 0.327365188 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 50982 → 80 [ACK] Seq=56     |
| 14              | 5.290026896 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 50982 [FIN, ACK] S     |
| 15              | 5.290130606 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 50982 → 80 [FIN, ACK] S     |
| 16              | 5.335860407 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 50982 [ACK] Seq=17     |

| tcp.stream eq 2 |              |              |              |          |        |                         |
|-----------------|--------------|--------------|--------------|----------|--------|-------------------------|
| No.             | Time         | Source       | Destination  | Protocol | Length | Info                    |
| 28              | 42.917398520 | 10.54.15.100 | 10.54.15.68  | TCP      | 74     | 51012 → 80 [SYN] Seq=0  |
| 29              | 42.969116869 | 10.54.15.68  | 10.54.15.100 | TCP      | 74     | 80 → 51012 [SYN, ACK] S |
| 30              | 42.969140002 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51012 → 80 [ACK] Seq=1  |
| 31              | 42.969353931 | 10.54.15.100 | 10.54.15.68  | HTTP     | 501    | POST /login.php HTTP/1  |
| 32              | 43.016754695 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51012 [ACK] Seq=1  |
| 33              | 43.033810817 | 10.54.15.68  | 10.54.15.100 | HTTP     | 604    | HTTP/1.1 200 OK (text/  |
| 34              | 43.033823724 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51012 → 80 [ACK] Seq=43 |
| 35              | 48.040277295 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51012 [FIN, ACK] S |
| 36              | 48.040506072 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51012 → 80 [FIN, ACK] S |
| 37              | 48.088082920 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51012 [ACK] Seq=54 |

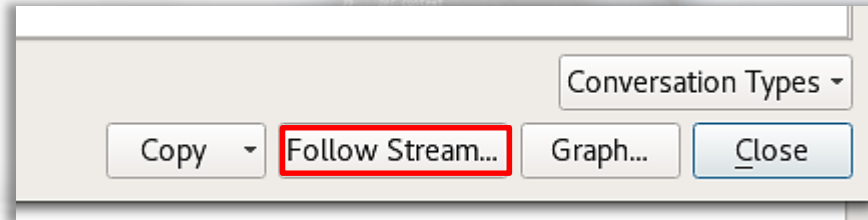
| tcp.stream eq 1 |              |              |              |          |        |                         |
|-----------------|--------------|--------------|--------------|----------|--------|-------------------------|
| No.             | Time         | Source       | Destination  | Protocol | Length | Info                    |
| 18              | 20.862049221 | 10.54.15.100 | 10.54.15.68  | TCP      | 74     | 51008 → 80 [SYN] Seq=0  |
| 19              | 20.907960258 | 10.54.15.68  | 10.54.15.100 | TCP      | 74     | 80 → 51008 [SYN, ACK] S |
| 20              | 20.907988940 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51008 → 80 [ACK] Seq=1  |
| 21              | 20.908172432 | 10.54.15.100 | 10.54.15.68  | HTTP     | 358    | GET /favicon.ico HTTP/2 |
| 22              | 20.955920739 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51008 [ACK] Seq=1  |
| 23              | 20.957862825 | 10.54.15.68  | 10.54.15.100 | HTTP     | 568    | HTTP/1.1 404 Not Found  |
| 24              | 20.957877026 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51008 → 80 [ACK] Seq=25 |
| 25              | 25.962682870 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51008 [FIN, ACK] S |
| 26              | 25.962789254 | 10.54.15.100 | 10.54.15.68  | TCP      | 66     | 51008 → 80 [FIN, ACK] S |
| 27              | 26.008812475 | 10.54.15.68  | 10.54.15.100 | TCP      | 66     | 80 → 51008 [ACK] Seq=50 |

| tcp.stream eq 3 |      |        |             |          |        |      |
|-----------------|------|--------|-------------|----------|--------|------|
| No.             | Time | Source | Destination | Protocol | Length | Info |

## 2.7.1.1 Normal HTTP

3. Under the TCP tab in Conversations, we can see there are three TCP Streams. From here, we can select a stream and choose Follow Stream from the bottom right corner.

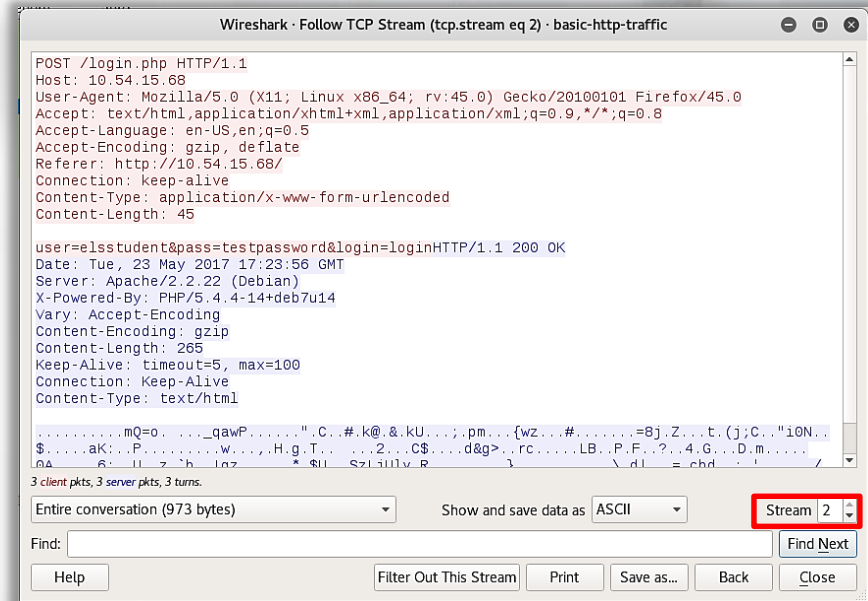
| Ethernet · 4 |        | IPv4 · 1    |        | IPv6 · 1 |       | TCP · 3       |             | UDP           |             |           |          |              |              |
|--------------|--------|-------------|--------|----------|-------|---------------|-------------|---------------|-------------|-----------|----------|--------------|--------------|
| Address A    | Port A | Address B   | Port B | Packets  | Bytes | Packets A → B | Bytes A → B | Packets B → A | Bytes B → A | Rel Start | Duration | Bits/s A → B | Bits/s B → A |
| 10.54.15.100 | 50982  | 10.54.15.68 | 80     | 13       | 3198  | 7             | 1030        | 6             | 2168        | 0.050883  | 5.2850   | 1559         |              |
| 10.54.15.100 | 51008  | 10.54.15.68 | 80     | 10       | 1470  | 5             | 630         | 5             | 840         | 20.862049 | 5.1468   | 979          |              |
| 10.54.15.100 | 51012  | 10.54.15.68 | 80     | 10       | 1649  | 5             | 773         | 5             | 876         | 42.917399 | 5.1707   | 1195         |              |



## 2.7.1.1 Normal HTTP

4. At the bottom right, we can see a dropdown box and we can select the TCP stream we would like to view.

By inspecting each of the TCP Streams, we can see from the image that the 3rd TCP Stream (zero indexed) will contain the cleartext credentials. This is indeed valid HTTP traffic and typical HTTP behavior.



## 2.7.1.1 Normal HTTP

Two other useful areas within Wireshark where we can look to analyze the details of the packet are:

### 1. Protocol Hierarchy Statistics

- Select **Statistics > Protocol Hierarchy**

### 2. Export HTML Objects

- Select **File Export Objects > HTML**

## 2.7.1.1.1 Protocol Hierarchy Statistics

1. Here we can see the protocols within the packet and their rate of occurrence. Within this image, there is one packet we have selected which we will use as a display filter and open the packet details in a new window to inspect further.

Wireshark · Protocol Hierarchy Statistics · basic-http-traffic

| Protocol                             | Percent Packets | Packets | Percent Bytes | Bytes | Bits/s | End Packets | End Bytes | End Bits/s |
|--------------------------------------|-----------------|---------|---------------|-------|--------|-------------|-----------|------------|
| ▼ Frame                              | 100.0           | 38      | 100.0         | 6619  | 576    | 0           | 0         | 0          |
| ▼ Ethernet                           | 100.0           | 38      | 8.0           | 532   | 46     | 0           | 0         | 0          |
| Text item                            | 2.6             | 1       | 0.0           | 0     | 0      | 0           | 0         | 0          |
| ▼ Internet Protocol Version 6        | 5.3             | 2       | 1.7           | 112   | 9      | 0           | 0         | 0          |
| Internet Control Message Protocol v6 | 5.3             | 2       | 0.5           | 32    | 2      | 2           | 32        | 2          |
| ▼ Internet Protocol Version 4        | 86.8            | 33      | 10.0          | 660   | 57     | 0           | 0         | 0          |
| ▼ Transmission Control Protocol      | 86.8            | 33      | 78.5          | 5195  | 452    | 25          | 848       | 73         |
| ▼ Hypertext Transfer Protocol        | 21.1            | 8       | 61.8          | 4091  | 356    | 3           | 852       | 74         |
| Line-based text data                 | 10.5            | 4       | 56.0          | 3709  | 322    | 4           | 3709      | 322        |
| HTML Form URL Encoded                | 2.6             | 1       | 0.7           | 45    | 3      | 1           | 45        | 3          |
| Address Resolution Protocol          | 7.9             | 3       | 1.3           | 84    | 7      | 3           | 84        | 7          |

## 2.7.1.1.1 Protocol Hierarchy Statistics

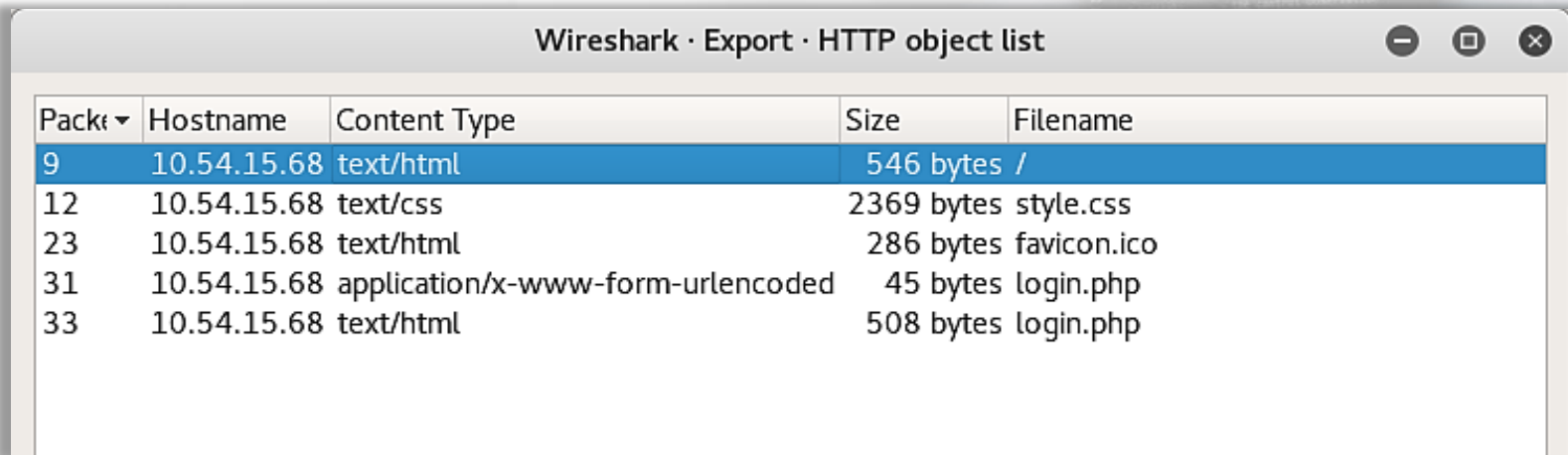
Using this method, we can see interesting packet details and again confirm it is legitimate HTTP traffic. Of course, in an enterprise environment this traffic should be encrypted, which we'll look at shortly.

```
Wireshark - Packet 31 - basic-http-traffic
> Frame 31: 501 bytes on wire (4008 bits), 501 bytes captured (4008 bits) on interface 0
> Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:61:66 (00:50:56:a1:61:66)
> Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.68
> Transmission Control Protocol, Src Port: 51012, Dst Port: 80, Seq: 1, Ack: 1, Len: 435
> Hypertext Transfer Protocol
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "user" = "elsstudent"
      Key: user
      Value: elsstudent
    > Form item: "pass" = "testpassword"
      Key: pass
      Value: testpassword
    > Form item: "login" = "login"
      Key: login
      Value: login
```

| urlencoded-form |              |              |             |          |        |                                                              |
|-----------------|--------------|--------------|-------------|----------|--------|--------------------------------------------------------------|
| No.             | Time         | Source       | Destination | Protocol | Length | Info                                                         |
| 31              | 42.969353... | 10.54.15.100 | 10.54.15.68 | HTTP     | 501    | POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |

## 2.7.1.1.2 Export HTTP Object

2. Using the Export HTTP Object feature, we can quickly see pertinent information regarding the packet. By saving the objects, they can be opened later for further inspection, especially if any binaries or scripts are exported.

A screenshot of the 'Wireshark · Export · HTTP object list' dialog box. The window has a title bar with standard macOS window controls (red, yellow, green buttons). Below the title bar is a table with five columns: 'Packet #', 'Hostname', 'Content Type', 'Size', and 'Filename'. The table contains five rows of data. The first row is highlighted in blue. To the right of the table, there is a vertical sidebar with four icons: a globe, a folder, a play button, and a flask.

| Packet # | Hostname    | Content Type                      | Size       | Filename    |
|----------|-------------|-----------------------------------|------------|-------------|
| 9        | 10.54.15.68 | text/html                         | 546 bytes  | /           |
| 12       | 10.54.15.68 | text/css                          | 2369 bytes | style.css   |
| 23       | 10.54.15.68 | text/html                         | 286 bytes  | favicon.ico |
| 31       | 10.54.15.68 | application/x-www-form-urlencoded | 45 bytes   | login.php   |
| 33       | 10.54.15.68 | text/html                         | 508 bytes  | login.php   |

## 2.7.1.1.2 Export HTTP Object

With the previous PCAP example, we took some time to look at different techniques to use with Wireshark to further analyze packet details.

With the upcoming examples, we will simply use the aforementioned techniques without going into details on how we reached that specific location within Wireshark.



## 2.7.1.2 Suspicious HTTP

Now we'll look at an example of suspicious traffic via port 80 using Wireshark and the techniques already demonstrated.

Remember the tips regarding normal HTTP traffic:

- Typically port 80
- Cleartext web-based traffic
- Hosts are accessed using FQDNs instead of IP addresses

## 2.7.1.2 Suspicious HTTP

Before we dive deeper into this PCAP, let's take a moment to glance at it to see if we can spot anything that might look odd.

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                                           |
|-----|-----------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000  | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33020 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=125613 TSecr=0 WS=128                        |
| 2   | 0.056720  | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33020 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=222206 TSecr=125613 WS=4          |
| 3   | 0.056747  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=125627 TSecr=222206                                         |
| 4   | 0.056824  | 10.124.211.200 | 10.124.211.96  | HTTP     | 349    | GET / HTTP/1.1                                                                                                 |
| 5   | 0.133531  | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [ACK] Seq=1 Ack=284 Win=15552 Len=0 TSval=222223 TSecr=125627                                       |
| 6   | 0.133549  | 10.124.211.96  | 10.124.211.200 | HTTP     | 101    | [TCP Previous segment not captured] Continuation                                                               |
| 7   | 0.133556  | 10.124.211.200 | 10.124.211.96  | TCP      | 78     | [TCP Window Update] 33020 → 80 [ACK] Seq=284 Ack=1 Win=30336 Len=0 TSval=125647 TSecr=222223 SLE=1326 SRE=1361 |
| 8   | 0.156230  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP out-of-order] 80 → 33020 [ACK] Seq=1 Ack=284 Win=15552 Len=1325 TSval=222224 TSecr=125627                 |
| 9   | 0.156259  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=284 Ack=1361 Win=33280 Len=0 TSval=125652 TSecr=222224                                    |
| 10  | 3.481485  | 10.124.211.200 | 10.124.211.96  | HTTP     | 389    | GET /news.php HTTP/1.1                                                                                         |
| 11  | 3.552272  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 12  | 3.552305  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=607 Ack=2686 Win=36096 Len=0 TSval=126501 TSecr=223083                                    |
| 13  | 3.552578  | 10.124.211.96  | 10.124.211.200 | HTTP     | 1353   | HTTP/1.1 200 OK (text/html)                                                                                    |
| 14  | 3.552589  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=607 Ack=3973 Win=39040 Len=0 TSval=126501 TSecr=223083                                    |
| 15  | 5.968993  | 10.124.211.200 | 10.124.211.96  | HTTP     | 410    | GET /newsdetails.php?id=26 HTTP/1.1                                                                            |
| 16  | 6.020439  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 17  | 6.020461  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=951 Ack=5298 Win=41856 Len=0 TSval=127118 TSecr=223701                                    |
| 18  | 6.020470  | 10.124.211.96  | 10.124.211.200 | HTTP     | 83     | HTTP/1.1 200 OK (text/html)                                                                                    |
| 19  | 6.020471  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=951 Ack=5315 Win=41856 Len=0 TSval=127118 TSecr=223701                                    |
| 20  | 9.453656  | 10.124.211.200 | 10.124.211.96  | HTTP     | 373    | GET /newsdetails.php?id=26%27 HTTP/1.1                                                                         |
| 21  | 9.517192  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 22  | 9.517210  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6640 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 23  | 9.517216  | 10.124.211.96  | 10.124.211.200 | HTTP     | 68     | HTTP/1.1 200 OK (text/html)                                                                                    |
| 24  | 9.517218  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6642 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 25  | 14.520484 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [FIN, ACK] Seq=6642 Ack=1258 Win=18768 Len=0 TSval=225826 TSecr=127992                              |
| 26  | 14.520662 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [FIN, ACK] Seq=1258 Ack=6643 Win=44800 Len=0 TSval=129243 TSecr=225826                              |
| 27  | 14.582082 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [ACK] Seq=6643 Ack=1259 Win=18768 Len=0 TSval=225842 TSecr=129243                                   |
| 28  | 17.437742 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33022 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=129973 TSecr=0 WS=128                        |
| 29  | 17.503661 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33022 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=226572 TSecr=129973 WS=4          |

## 2.7.1.2 Suspicious HTTP

This PCAP is much larger than the previous example and we cannot see all the juicy details in one screenshot, but there is one thing that stands out right away which is underlined below.

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                                           |
|-----|-----------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000  | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33020 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=125613 TSecr=0 WS=128                        |
| 2   | 0.056720  | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33020 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=222206 TSecr=125613 WS=4          |
| 3   | 0.056747  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=125627 TSecr=222206                                         |
| 4   | 0.056824  | 10.124.211.200 | 10.124.211.96  | HTTP     | 349    | GET / HTTP/1.1                                                                                                 |
| 5   | 0.133531  | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [ACK] Seq=1 Ack=284 Win=15552 Len=0 TSval=222223 TSecr=125627                                       |
| 6   | 0.133549  | 10.124.211.96  | 10.124.211.200 | HTTP     | 101    | [TCP Previous segment not captured] Continuation                                                               |
| 7   | 0.133556  | 10.124.211.200 | 10.124.211.96  | TCP      | 78     | [TCP Window Update] 33020 → 80 [ACK] Seq=284 Ack=1 Win=30336 Len=0 TSval=125647 TSecr=222223 SLE=1326 SRE=1361 |
| 8   | 0.156230  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP Out-of-Order] 80 → 33020 [ACK] Seq=1 Ack=284 Win=15552 Len=1325 TSval=222224 TSecr=125627                 |
| 9   | 0.156259  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=284 Ack=1361 Win=33280 Len=0 TSval=125652 TSecr=222224                                    |
| 10  | 3.481485  | 10.124.211.200 | 10.124.211.96  | HTTP     | 389    | GET /news.php HTTP/1.1                                                                                         |
| 11  | 3.552272  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 12  | 3.552305  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=607 Ack=2686 Win=36096 Len=0 TSval=126501 TSecr=223083                                    |
| 13  | 3.552578  | 10.124.211.96  | 10.124.211.200 | HTTP     | 1353   | HTTP/1.1 200 OK (text/html)                                                                                    |
| 14  | 3.552589  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=607 Ack=3973 Win=39040 Len=0 TSval=126501 TSecr=223083                                    |
| 15  | 5.968993  | 10.124.211.200 | 10.124.211.96  | HTTP     | 410    | GET /newsdetails.php?id=26 HTTP/1.1                                                                            |
| 16  | 6.020439  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 17  | 6.020461  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=951 Ack=5298 Win=41856 Len=0 TSval=127118 TSecr=223701                                    |
| 18  | 6.020470  | 10.124.211.96  | 10.124.211.200 | HTTP     | 83     | HTTP/1.1 200 OK (text/html)                                                                                    |
| 19  | 6.020471  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=951 Ack=5315 Win=41856 Len=0 TSval=127118 TSecr=223701                                    |
| 20  | 9.453656  | 10.124.211.200 | 10.124.211.96  | HTTP     | 373    | GET /newsdetails.php?id=26%27 HTTP/1.1                                                                         |
| 21  | 9.517192  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 22  | 9.517210  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6640 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 23  | 9.517216  | 10.124.211.96  | 10.124.211.200 | HTTP     | 68     | HTTP/1.1 200 OK (text/html)                                                                                    |
| 24  | 9.517218  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6642 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 25  | 14.520484 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [FIN, ACK] Seq=6642 Ack=1258 Win=18768 Len=0 TSval=225826 TSecr=127992                              |
| 26  | 14.520662 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [FIN, ACK] Seq=1258 Ack=6643 Win=44800 Len=0 TSval=129243 TSecr=225826                              |
| 27  | 14.582082 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [ACK] Seq=6643 Ack=1259 Win=18768 Len=0 TSval=225842 TSecr=129243                                   |
| 28  | 17.437742 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33022 → 80 [SYN] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=129973 TSecr=0 WS=128                  |
| 29  | 17.503661 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33022 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=226572 TSecr=129973 WS=4          |

## 2.7.1.2 Suspicious HTTP

In the screenshot, we can quickly see normal requests to web pages, such as **news.php** and **newsdetails.php?id=26**, then we see another request to **newsdetails.php?id=26%27**.

If we URL decode **%27**, it corresponds to a **single quote** (**'**). This behavior is typical when looking for SQL injection (SQLi). Now, was someone looking for a SQLi, or did someone make a typo in their web page request? Let's look at the PCAP further.

## 2.7.1.2 Suspicious HTTP

Glancing further at the PCAP, a few packets down we can see more indications of SQLi attempts. Looking at packets 31 and 44, we can see that the person didn't make a typo request at packet 20. This is indeed an attempt to find SQLi on the server.

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                                           |
|-----|-----------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------------------------|
| 20  | 9.453656  | 10.124.211.200 | 10.124.211.96  | HTTP     | 373    | GET /newsdetails.php?id=26%27 HTTP/1.1                                                                         |
| 21  | 9.517192  | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                             |
| 22  | 9.517210  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6640 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 23  | 9.517216  | 10.124.211.96  | 10.124.211.200 | HTTP     | 68     | HTTP/1.1 200 OK (text/html)                                                                                    |
| 24  | 9.517218  | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [ACK] Seq=1258 Ack=6642 Win=44800 Len=0 TSval=127992 TSecr=224575                                   |
| 25  | 14.520484 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [FIN, ACK] Seq=6642 Ack=1258 Win=18768 Len=0 TSval=225826 TSecr=127992                              |
| 26  | 14.520662 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33020 → 80 [FIN, ACK] Seq=1258 Ack=6643 Win=44800 Len=0 TSval=129243 TSecr=225826                              |
| 27  | 14.582082 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33020 [ACK] Seq=6643 Ack=1259 Win=18768 Len=0 TSval=225842 TSecr=129243                                   |
| 28  | 17.437742 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33022 → 80 [SYN, Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=129973 TSecr=0 WS=128                        |
| 29  | 17.503661 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33022 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=226572 TSecr=129973 WS=4          |
| 30  | 17.503697 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33022 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=129989 TSecr=226572                                         |
| 31  | 17.504112 | 10.124.211.200 | 10.124.211.96  | HTTP     | 389    | GET /newsdetails.php?id=26%20and%201=1;--%20- HTTP/1.1                                                         |
| 32  | 17.575934 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33022 [ACK] Seq=1 Ack=324 Win=15552 Len=0 TSval=226590 TSecr=129989                                       |
| 33  | 17.578773 | 10.124.211.96  | 10.124.211.200 | TCP      | 84     | [TCP Previous segment not captured] [TCP segment of a reassembled PDU]                                         |
| 34  | 17.578787 | 10.124.211.200 | 10.124.211.96  | TCP      | 78     | [TCP Window Update] 33022 → 80 [ACK] Seq=324 Ack=1 Win=30336 Len=0 TSval=130008 TSecr=226590 SLE=1326 SRE=1344 |
| 35  | 17.579068 | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP Out-Of-Order] 80 → 33022 [ACK] Seq=1 Ack=324 Win=15552 Len=1325 TSval=226591 TSecr=129989                 |
| 36  | 17.579085 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33022 → 80 [ACK] Seq=324 Ack=1344 Win=33280 Len=0 TSval=130008 TSecr=226591                                    |
| 37  | 22.579448 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33022 → 80 [FIN, ACK] Seq=324 Ack=1344 Win=33280 Len=0 TSval=131258 TSecr=226591                               |
| 38  | 22.584131 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33022 [FIN, ACK] Seq=1344 Ack=324 Win=15552 Len=0 TSval=227842 TSecr=130008                               |
| 39  | 22.584144 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 33022 → 80 [ACK] Seq=325 Ack=1345 Win=33280 Len=0 TSval=131259 TSecr=227842                                    |
| 40  | 22.625046 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33022 [ACK] Seq=1345 Ack=325 Win=15552 Len=0 TSval=227852 TSecr=131258                                    |
| 41  | 25.101234 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 33024 → 80 [SYN, Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=131888 TSecr=0 WS=128                        |
| 42  | 25.141132 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33024 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=228481 TSecr=131888 WS=4          |
| 43  | 25.141157 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33024 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=131898 TSecr=228481                                         |
| 44  | 25.141361 | 10.124.211.200 | 10.124.211.96  | HTTP     | 389    | GET /newsdetails.php?id=26%20and%201=2;--%20- HTTP/1.1                                                         |
| 45  | 25.186826 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33024 [ACK] Seq=1 Ack=324 Win=15552 Len=0 TSval=228492 TSecr=131899                                       |
| 46  | 25.187157 | 10.124.211.96  | 10.124.211.200 | HTTP     | 1285   | HTTP/1.1 200 OK (text/html)                                                                                    |
| 47  | 25.187168 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33024 → 80 [ACK] Seq=324 Ack=1220 Win=32128 Len=0 TSval=131910 TSecr=228493                                    |
| 48  | 30.188296 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33024 → 80 [FIN, ACK] Seq=324 Ack=1220 Win=32128 Len=0 TSval=133160 TSecr=228493                               |

## 2.7.1.2 Suspicious HTTP

Remember, to be an effective Threat Hunter, you must know and understand Red Team tactics and recognize malicious actions. Something this obvious can't get past a hunter. He/she must think like an adversary.

Now knowing that the individual is conducting SQLi queries against the server, is it manual or is the individual using a tool?



## 2.7.1.2 Suspicious HTTP

We begin at the first packet in question, packet 20. By further inspection, we see that the User-Agent is Firefox and the OS is Linux. Of course we should already know that this can be spoofed but not always, as we will see shortly.

```
▶ Frame 20: 373 bytes on wire (2984 bits), 373 bytes captured (2984 bits)
▶ Ethernet II, Src: 1a:3a:46:bf:43:91 (1a:3a:46:bf:43:91), Dst: Vmware_a1:4e:f0 (00:50:56:a1:4e:f0)
▶ Internet Protocol Version 4, Src: 10.124.211.200, Dst: 10.124.211.96
▶ Transmission Control Protocol, Src Port: 33020, Dst Port: 80, Seq: 951, Ack: 5315, Len: 307
▶ Hypertext Transfer Protocol
  ▶ GET /newsdetails.php?id=26%27 HTTP/1.1\r\n
    Host: 10.124.211.96\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://10.124.211.96/newsdetails.php?id=26%27]
    [HTTP request 4/4]
    [Prev request in frame: 15]
    [Response in frame: 23]
```

## 2.7.1.2 Suspicious HTTP

We see that the User-Agent is the same for packets 31 and 44.

Let's keep looking to see if we can find anything else of interest.

```
▶ Frame 44: 389 bytes on wire (3112 bits), 389 bytes captured (3112 bits)
▶ Ethernet II, Src: 1a:3a:46:bf:43:91 (1a:3a:46:bf:43:91), Dst: Vmware_a1:4e:f0 (00:50:56:a1:4e:f0)
▶ Internet Protocol Version 4, Src: 10.124.211.200, Dst: 10.124.211.96
▶ Transmission Control Protocol, Src Port: 33024, Dst Port: 80, Seq: 1, Ack: 1, Len: 323
▶ Hypertext Transfer Protocol
  ▶ GET /newsdetails.php?id=26%20and%201=2;--%20- HTTP/1.1\r\n
    Host: 10.124.211.96\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://10.124.211.96/newsdetails.php?id=26%20and%201=2;--%20-]
    [HTTP request 1/1]
    [Response in frame: 46]
```

```
▶ Frame 31: 389 bytes on wire (3112 bits), 389 bytes captured (3112 bits)
▶ Ethernet II, Src: 1a:3a:46:bf:43:91 (1a:3a:46:bf:43:91), Dst: Vmware_a1:4e:f0 (00:50:56:a1:4e:f0)
▶ Internet Protocol Version 4, Src: 10.124.211.200, Dst: 10.124.211.96
▶ Transmission Control Protocol, Src Port: 33022, Dst Port: 80, Seq: 1, Ack: 1, Len: 323
▶ Hypertext Transfer Protocol
  ▶ GET /newsdetails.php?id=26%20and%201=1;--%20- HTTP/1.1\r\n
    Host: 10.124.211.96\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://10.124.211.96/newsdetails.php?id=26%20and%201=1;--%20-]
    [HTTP request 1/1]
```

## 2.7.1.2 Suspicious HTTP

By looking back to the initial packet of interest, packet 20, we can look at the **Line-based text data: text/html** portion of the packet details and notice that error messages are being displayed to the individual conducting the SQLi queries.

```
\t<!-- end sidebar -->\n\t<div id="content">\n\t\t<div></div>\n\t\t<div class="boxed">\n\t\t\t<h1 class="title2">\n\t\t\t\n\t\t\t<br />\n\t\t\t<b>Warning</b>: mysqli_fetch_array() expects parameter 1 to be mysqli_result, boolean given in <b>/var/www/newsdetails.php</b> on line <b>11</b><br />\n\t\t\t\n\t\t\t</h1>\n\t\t\t<p>\n\t\t\t<br />\n\t\t\t<b>Warning</b>: mysqli_fetch_array() expects parameter 1 to be mysqli_result, boolean given in <b>/var/www/newsdetails.php</b> on line <b>29</b><br />\n\t\t\t</p>\n\t\t\t\n\t\t\t</div>\n\t\t\t
```

## 2.7.1.2 Suspicious HTTP

```
33 17.578773 10.124.211.96 10.124.211.200 TCP 84 [TCP Previous segment not captured] [TCP segment of a reassembled PDU]
34 17.578787 10.124.211.200 10.124.211.96 TCP 78 [TCP Window Update] 33022 → 80 [ACK] Seq=324 Ack=1 Win=30336 Len=0 TSval=130008 TSecr=226590 SLE=1326 SRE=1344
35 17.579068 10.124.211.96 10.124.211.200 TCP 1391 [TCP Out-Of-Order] 80 → 33022 [ACK] Seq=1 Ack=324 Len=1325 TSval=226591 TSecr=129989
36 17.579095 10.124.211.200 10.124.211.96 TCP 66 33022 → 80 [ACK] Seq=324 Ack=1344 Win=33280 Len=0 TSval=130008 TSecr=226591
```

Packet 33 in the screenshot indicates that something went wrong and the response to the request made in packet 31 was not captured. So, we'll look at the packet containing the response for the next packet of interest, which will be packet 46.

## 2.7.1.2 Suspicious HTTP

Interestingly, this packet shows that it doesn't contain any error-based MySQL messages, even though clearly a SQLi query was executed.

This is common behavior where an attacker will manually inspect the target before running automated tools against it.

Once enough information is gathered, the attacker often would switch to an automated tool against the target server.

```
\t\t\t\t\t<form id="form1" method="post" action="login.php">\n\t\t\t\t\t<fieldset>\n\t\t\t\t\t<label for="inputtext1">Email:</label>\n\t\t\t\t\t<input id="inputtext1" type="text" name="username" value="" />\n\t\t\t\t\t<label for="inputtext2">Password:</label>\n\t\t\t\t\t<input id="inputtext2" type="password" name="password" value="" />\n\t\t\t\t\t<input id="inputsubmit1" type="submit" name="submit" value="Login" />\n\t\t\t\t\t<p><a href="#">Forgo your password?</a><br />\n\t\t\t\t\t</fieldset>\n\t\t\t\t\t</form>\n\t\t\t\t</div>\n\t\t</div>\n\t\t<!-- end sidebar -->\n\t\t<div id="content">\n\t\t\t<div></div>\n\t\t\t<div class="boxed">\n\t\t\t\t<h1 class="title2">\n\t\t\t\t\n\t\t\t\t\n\t\t\t\t\n\t\t\t\t</h1>\n\t\t\t\t<p>\n\t\t\t\t</p>\n\t\t\t\t\n\t\t\t\t</div>\n\t\t\t</div>\n\t\t<!-- end content -->\n\t\t<div style="clear: both;">&nbsp;</div>\n\t\t<!-- end page -->\n\t\t<div id="footer">\n\t\t\t<p id="legal">Copyright &copy; 2017 AwdMgmt.</p>\n\t\t\t<p id="links"><a href="#">Privacy terms</a> | <a href="#">Licensing</a></p>\n\t\t\t</div>
```

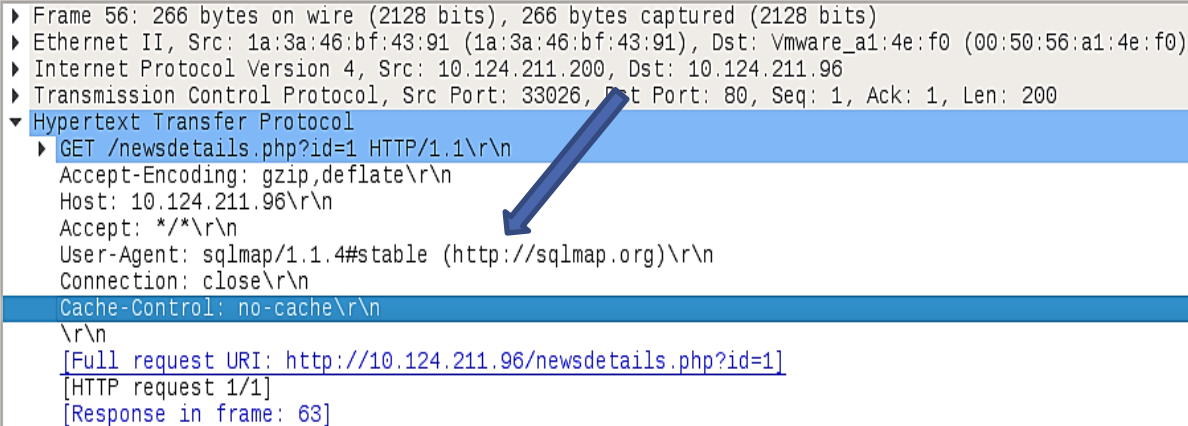
## 2.7.1.2 Suspicious HTTP

Strangely the next packet, packet 56, and the packet after that, packet 73, do not seem to contain any SQLi queries. Maybe the individual decided not to continue the attack?

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                                  |
|-----|-----------|----------------|----------------|----------|--------|-------------------------------------------------------------------------------------------------------|
| 56  | 34.163959 | 10.124.211.200 | 10.124.211.96  | HTTP     | 266    | GET /newsdetails.php?id=1 HTTP/1.1                                                                    |
| 57  | 34.169322 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33028 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=230739 TSecr=134145 WS=4 |
| 58  | 34.169364 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33028 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134156 TSecr=230739                                |
| 59  | 34.180457 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33030 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134158 TSecr=0 WS=128               |
| 60  | 34.202983 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33026 [ACK] Seq=1 Ack=201 Win=15552 Len=0 TSval=230747 TSecr=134154                              |
| 61  | 34.206414 | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                    |
| 62  | 34.206432 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33026 → 80 [ACK] Seq=201 Ack=1326 Win=32128 Len=0 TSval=134165 TSecr=230748                           |
| 63  | 34.206495 | 10.124.211.96  | 10.124.211.200 | HTTP     | 82     | HTTP/1.1 200 OK (text/html)                                                                           |
| 64  | 34.206501 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33026 → 80 [ACK] Seq=201 Ack=1342 Win=32128 Len=0 TSval=134165 TSecr=230748                           |
| 65  | 34.206508 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33026 [FIN, ACK] Seq=1342 Ack=201 Win=15552 Len=0 TSval=230748 TSecr=134154                      |
| 66  | 34.207131 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33026 → 80 [FIN, ACK] Seq=201 Ack=1343 Win=32128 Len=0 TSval=134165 TSecr=230748                      |
| 67  | 34.218945 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33030 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=230751 TSecr=134158 WS=4 |
| 68  | 34.218972 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33030 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134168 TSecr=230751                                |
| 69  | 34.230022 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33032 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134171 TSecr=0 WS=128               |
| 70  | 34.248217 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33026 [ACK] Seq=1343 Ack=202 Win=15552 Len=0 TSval=230758 TSecr=134165                           |
| 71  | 34.269905 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33032 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=230764 TSecr=134171 WS=4 |
| 72  | 34.269934 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33032 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134181 TSecr=230764                                |
| 73  | 35.126264 | 10.124.211.200 | 10.124.211.96  | HTTP     | 266    | GET /newsdetails.php?id=1 HTTP/1.1                                                                    |
| 74  | 35.126569 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33034 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134395 TSecr=0 WS=128               |
| 75  | 35.166226 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33028 [ACK] Seq=1 Ack=201 Win=15552 Len=0 TSval=230988 TSecr=134395                              |
| 76  | 35.166267 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33034 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=230988 TSecr=134395 WS=4 |
| 77  | 35.166341 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33034 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134405 TSecr=230988                                |
| 78  | 35.168806 | 10.124.211.96  | 10.124.211.200 | TCP      | 1391   | [TCP segment of a reassembled PDU]                                                                    |
| 79  | 35.168835 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33028 → 80 [ACK] Seq=201 Ack=1326 Win=32128 Len=0 TSval=134405 TSecr=230988                           |
| 80  | 35.168926 | 10.124.211.96  | 10.124.211.200 | HTTP     | 82     | HTTP/1.1 200 OK (text/html)                                                                           |
| 81  | 35.168943 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33028 → 80 [ACK] Seq=201 Ack=1342 Win=32128 Len=0 TSval=134405 TSecr=230988                           |
| 82  | 35.168962 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33028 [FIN, ACK] Seq=1342 Ack=201 Win=15552 Len=0 TSval=230988 TSecr=134395                      |
| 83  | 35.170154 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33028 → 80 [FIN, ACK] Seq=201 Ack=1343 Win=32128 Len=0 TSval=134406 TSecr=230988                      |

## 2.7.1.2 Suspicious HTTP

Let's take a closer look at packet 56. Indeed no SQLi is visible within this packet but something else should have already jumped out at us, the User-Agent. The User-Agent for this HTTP Get Request is **Sqlmap**. So the attacker didn't quit, he/she escalated.



```
▶ Frame 56: 266 bytes on wire (2128 bits), 266 bytes captured (2128 bits)
▶ Ethernet II, Src: 1a:3a:46:bf:43:91 (1a:3a:46:bf:43:91), Dst: Vmware_a1:4e:f0 (00:50:56:a1:4e:f0)
▶ Internet Protocol Version 4, Src: 10.124.211.200, Dst: 10.124.211.96
▶ Transmission Control Protocol, Src Port: 33026, Dst Port: 80, Seq: 1, Ack: 1, Len: 200
▶ Hypertext Transfer Protocol
  ▶ GET /newsdetails.php?id=1 HTTP/1.1\r\n
    Accept-Encoding: gzip,deflate\r\n
    Host: 10.124.211.96\r\n
    Accept: */*\r\n
    User-Agent: sqlmap/1.1.4#stable (http://sqlmap.org)\r\n
    Connection: close\r\n
    Cache-Control: no-cache\r\n
    \r\n
    [Full request URI: http://10.124.211.96/newsdetails.php?id=1]
    [HTTP request 1/1]
    [Response in frame: 63]
```

## 2.7.1.2 Suspicious HTTP

A few of the subsequent HTTP Get Requests that follow will also look like legitimate HTTP traffic, but these will also contain the same User-Agent within the packet details, Sqlmap, until we reach packet 105.

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                                                 |
|-----|-----------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------------------------------|
| 84  | 35.206762 | 10.124.211.200 | 10.124.211.96  | HTTP     | 269    | GET /newsdetails.php?id=4616 HTTP/1.1                                                                                |
| 85  | 35.209109 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33036 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134415 TSecr=0 WS=128                              |
| 86  | 35.210593 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33028 [ACK] Seq=1343 Ack=202 Win=15552 Len=0 TSval=230999 TSecr=134406                                          |
| 87  | 35.249943 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33030 [ACK] Seq=1 Ack=204 Win=15552 Len=0 TSval=231009 TSecr=134415                                             |
| 88  | 35.249959 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33036 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=231009 TSecr=134415 WS=4                |
| 89  | 35.249996 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33036 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134426 TSecr=231009                                               |
| 90  | 35.252937 | 10.124.211.96  | 10.124.211.200 | HTTP     | 1248   | HTTP/1.1 200 OK (text/html)                                                                                          |
| 91  | 35.252950 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33030 → 80 [ACK] Seq=204 Ack=1183 Win=32128 Len=0 TSval=134426 TSecr=231009                                          |
| 92  | 35.252977 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33030 [FIN, ACK] Seq=1183 Ack=204 Win=15552 Len=0 TSval=231009 TSecr=134415                                     |
| 93  | 35.253307 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33030 → 80 [FIN, ACK] Seq=204 Ack=1184 Win=32128 Len=0 TSval=134426 TSecr=231009                                     |
| 94  | 35.257209 | 10.124.211.200 | 10.124.211.96  | HTTP     | 269    | GET /newsdetails.php?id=1416 HTTP/1.1                                                                                |
| 95  | 35.260419 | 10.124.211.200 | 10.124.211.96  | TCP      | 74     | 33038 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134428 TSecr=0 WS=128                              |
| 96  | 35.294695 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33030 [ACK] Seq=1184 Ack=205 Win=15552 Len=0 TSval=231009 TSecr=134426                                          |
| 97  | 35.295538 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33032 [ACK] Seq=1 Ack=204 Win=15552 Len=0 TSval=231021 TSecr=134427                                             |
| 98  | 35.295565 | 10.124.211.96  | 10.124.211.200 | TCP      | 74     | 80 → 33038 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=231021 TSecr=134427 WS=4                |
| 99  | 35.295583 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33038 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=134438 TSecr=231021                                               |
| 100 | 35.300920 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | [TCP Previous segment not captured] 80 → 33032 [FIN, ACK] Seq=1183 Ack=204 Win=15552 Len=0 TSval=231022 TSecr=134427 |
| 101 | 35.300926 | 10.124.211.200 | 10.124.211.96  | TCP      | 78     | [TCP Dup ACK 72#1] 33032 → 80 [ACK] Seq=204 Ack=1 Win=32128 Len=0 TSval=134438 TSecr=231021 SLE=1183 SRE=1184        |
| 102 | 35.301386 | 10.124.211.96  | 10.124.211.200 | TCP      | 1248   | [TCP Out-Of-Order] 80 → 33032 [PSH, ACK] Seq=1 Ack=2 Win=15552 Len=1182 TSval=231022 TSecr=134427                    |
| 103 | 35.301315 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33032 → 80 [ACK] Seq=204 Ack=1184 Win=32128 Len=0 TSval=134439 TSecr=231022                                          |
| 104 | 35.301742 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33032 → 80 [FIN, ACK] Seq=204 Ack=1184 Win=32128 Len=0 TSval=134439 TSecr=231022                                     |
| 105 | 35.304677 | 10.124.211.200 | 10.124.211.96  | HTTP     | 290    | GET /newsdetails.php?id=1%27%2C.%2C%2C%2C%22.%2C HTTP/1.1                                                            |
| 106 | 35.310429 | 10.124.211.96  | 10.124.211.96  | TCP      | 74     | 33040 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=134441 TSecr=0 WS=128                              |
| 107 | 35.343916 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33032 [ACK] Seq=1184 Ack=205 Win=15552 Len=0 TSval=231032 TSecr=134439                                          |
| 108 | 35.343937 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33034 [ACK] Seq=1 Ack=225 Win=15552 Len=0 TSval=231032 TSecr=134439                                             |
| 109 | 35.347304 | 10.124.211.96  | 10.124.211.200 | HTTP     | 1357   | HTTP/1.1 200 OK (text/html)                                                                                          |
| 110 | 35.347317 | 10.124.211.200 | 10.124.211.96  | TCP      | 66     | 33034 → 80 [ACK] Seq=225 Ack=1292 Win=32128 Len=0 TSval=134450 TSecr=231033                                          |
| 111 | 35.347328 | 10.124.211.96  | 10.124.211.200 | TCP      | 66     | 80 → 33034 [FIN, ACK] Seq=1292 Ack=225 Win=15552 Len=0 TSval=134450 TSecr=134439                                     |

## 2.7.1.2 Suspicious HTTP

So clearly, we see the packets are getting more interesting due to Sqlmap and its attempts to find and validate SQLi.

```
▶ Frame 105: 290 bytes on wire (2320 bits), 290 bytes captured (2320 bits)
▶ Ethernet II, Src: 1a:3a:46:bf:43:91 (1a:3a:46:bf:43:91), Dst: Vmware_a1:4e:f0 (00:50:56:a1:4e:f0)
▶ Internet Protocol Version 4, Src: 10.124.211.200, Dst: 10.124.211.96
▶ Transmission Control Protocol, Src Port: 33034, Dst Port: 80, Seq: 1, Ack: 1, Len: 224
▼ Hypertext Transfer Protocol
  ▶ GET /newsdetails.php?id=1%27%2C..%2C%2C%2C%22.%2C HTTP/1.1\r\n
    ▶ [Expert Info (Chat/Sequence): GET /newsdetails.php?id=1%27%2C..%2C%2C%2C%22.%2C HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /newsdetails.php?id=1%27%2C..%2C%2C%2C%22.%2C
      Request URI Path: /newsdetails.php
      ▼ Request URI Query: id=1%27%2C..%2C%2C%2C%22.%2C
        Request URI Query Parameter: id=1%27%2C..%2C%2C%2C%22.%2C
      Request Version: HTTP/1.1
      Accept-Encoding: gzip,deflate\r\n
      Host: 10.124.211.96\r\n
      Accept: */*\r\n
      User-Agent: sqlmap/1.1.4#stable (http://sqlmap.org)\r\n
      Connection: close\r\n
      Cache-Control: no-cache\r\n
      \r\n
      [Full request URI: http://10.124.211.96/newsdetails.php?id=1%27%2C..%2C%2C%2C%22.%2C]
      [HTTP request 1/1]
      [Response in frame: 109]
```

## 2.7.1.2 Suspicious HTTP

Here we have the last HTTP Get Request showing a very long query to test for SQLi on the internal web server. Again, Sqlmap seems to be the tool that is being used by the culprit, and from here, we can inspect the web server logs to investigate further.

[illegible]

## 2.7.1.2 Suspicious HTTP

Before inspecting the logs pertaining to the web server of interest, we already have a good idea of the source host from which these queries originated from, 10.124.211.200, which is on the same subnet as the web server. We also know that the individual manually tested for SQLi before running Sqlmap against the web server.

Now the hunt goes to find out if this is a malicious employee or someone has infiltrated the network. That quest is for another time...

## 2.7.2 HTTPS

In the upcoming slides, we'll look at:

- HTTPS traffic
- Some pointers to remember about HTTPS traffic
- Some tips about normal and suspicious HTTPS traffic
- PCAPs to demonstrate each scenario



## 2.7.2 HTTPS

A few things to remember about HTTPS...

HTTPS is the **secure version** of HTTP. The “**S**” refers to Secure Socket Layer/Transport Layer Security (**SSL/TLS**).



## 2.7.2 HTTPS

A few things to remember about HTTPS (CONTINUED)...

HTTPS also establishes a handshake similar to TCP but more complex. Below is a brief summary:

- Both the client and the server need to agree on the protocol version.
- Both the client and the server need to select cryptographic algorithms.
- Optionally, the client and server authenticate to each other.
- Uses public key encryption techniques to establish secure communications.



## 2.7.2 HTTPS

### Some facts to help distinguish normal and suspicious HTTPS traffic.

| Normal HTTPS Traffic                                | Suspicious HTTPS Traffic                                                                                                                       |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Port 443, TCP<br>Port 8443, TCP (used as alternate) | Malicious binaries (backdoors), scripts, web shells, etc. will use this port because typically in all corporate environments the port is open. |
| Encrypted traffic                                   | If the traffic is not encrypted and Secure Sockets Layer packet details are empty within packet details, then that will fall under suspicious. |
| Web server typically in FQDN format.                | Server will point to an IP address instead of FQDN format.                                                                                     |

## 2.7.2 HTTPS

We will not dive too deep into the handshake used by HTTPS traffic but there are certain details within the packets that we should be familiar with. Those details will be pointed out in the upcoming slides. If you want to dive deeper into how HTTPS establishes secure communications, please refer to [RFC 6101](https://tools.ietf.org/html/rfc6101) for SSL 3.0 & [RFC 5246](https://tools.ietf.org/html/rfc5246) for TLS 1.2.

Note: There is a draft for TLS 1.3 [here](#).



## 2.7.2.1 Normal HTTPS

By looking at this screenshot, we are not able to simply glance at the packets to get an idea as to what is going on.

| No. | Time      | Source       | Destination  | Protocol | Length | Info                                                                                                   |
|-----|-----------|--------------|--------------|----------|--------|--------------------------------------------------------------------------------------------------------|
| 3   | 0.049179  | 10.54.15.100 | 10.54.15.15  | TCP      | 74     | 39678 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2640960 TSecr=0 WS=128               |
| 5   | 0.094121  | 10.54.15.15  | 10.54.15.100 | TCP      | 54     | 80 → 39678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0                                                          |
| 9   | 14.560139 | 10.54.15.100 | 10.54.15.15  | TCP      | 74     | 45112 → 443 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2644600 TSecr=0 WS=128              |
| 10  | 14.608349 | 10.54.15.15  | 10.54.15.100 | TCP      | 74     | 443 → 45112 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=2644600 TSecr=0 WS=4     |
| 11  | 14.608374 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45112 → 443 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2644612 TSecr=35434                                |
| 12  | 14.608618 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 233    | Client Hello                                                                                           |
| 13  | 14.654485 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45112 [ACK] Seq=1 Ack=168 Win=15552 Len=0 TSval=35446 TSecr=2644612                              |
| 14  | 14.658826 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 1391   | Server Hello, Certificate                                                                              |
| 15  | 14.658839 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45112 → 443 [ACK] Seq=168 Ack=1326 Win=32128 Len=0 TSval=2644624 TSecr=35447                           |
| 16  | 14.660183 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 73     | Alert (Level: Fatal, Description: Unknown CA)                                                          |
| 17  | 14.660252 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45112 → 443 [FIN, ACK] Seq=175 Ack=1326 Win=32128 Len=0 TSval=2644625 TSecr=35447                      |
| 18  | 14.661516 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 170    | Server Key Exchange Server Hello Done                                                                  |
| 19  | 14.661532 | 10.54.15.100 | 10.54.15.15  | TCP      | 54     | 45112 → 443 [RST] Seq=168 Win=0 Len=0                                                                  |
| 20  | 14.706639 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45112 [FIN, ACK] Seq=1430 Ack=176 Win=15552 Len=0 TSval=35459 TSecr=2644625                      |
| 21  | 14.706656 | 10.54.15.100 | 10.54.15.15  | TCP      | 54     | 45112 → 443 [RST] Seq=176 Win=0 Len=0                                                                  |
| 22  | 22.545448 | 10.54.15.100 | 10.54.15.15  | TCP      | 74     | 45114 → 443 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=2646596 TSecr=0 WS=128              |
| 23  | 22.594263 | 10.54.15.15  | 10.54.15.100 | TCP      | 74     | 443 → 45114 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1337 SACK_PERM=1 TSval=37430 TSecr=2646596 WS=4 |
| 24  | 22.594291 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=2646608 TSecr=37430                                |
| 25  | 22.594487 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 233    | Client Hello                                                                                           |
| 26  | 22.643572 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45114 [ACK] Seq=1 Ack=168 Win=15552 Len=0 TSval=37443 TSecr=2646608                              |
| 27  | 22.646963 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 1391   | Server Hello, Certificate                                                                              |
| 28  | 22.646975 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=168 Ack=1326 Win=32128 Len=0 TSval=2646621 TSecr=37445                           |
| 29  | 22.649560 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 170    | Server Key Exchange Server Hello Done                                                                  |
| 30  | 22.649566 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=168 Ack=1430 Win=32128 Len=0 TSval=2646622 TSecr=37445                           |
| 31  | 22.651191 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 192    | Client Key Exchange, Change Cipher Spec, Hello Request, Hello Request                                  |
| 32  | 22.651236 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 376    | Application Data                                                                                       |
| 33  | 22.700218 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45114 [ACK] Seq=1430 Ack=604 Win=16624 Len=0 TSval=37457 TSecr=2646622                           |
| 34  | 22.700636 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 324    | New Session Ticket, Change Cipher Spec, Encrypted Handshake Message                                    |
| 35  | 22.704803 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 770    | Application Data, Application Data, Application Data, Application Data                                 |
| 36  | 22.704970 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=604 Ack=2392 Win=37504 Len=0 TSval=2646636 TSecr=37457                           |
| 37  | 22.715752 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 376    | Application Data                                                                                       |
| 38  | 22.763897 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 1358   | Application Data, Application Data, Application Data, Application Data                                 |
| 39  | 22.807472 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=913 Ack=3684 Win=40320 Len=0 TSval=2646662 TSecr=37474                           |
| 40  | 27.767987 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 97     | Encrypted Alert                                                                                        |
| 41  | 27.768017 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [FIN, ACK] Seq=944 Ack=3684 Win=40320 Len=0 TSval=2647902 TSecr=37474                      |
| 42  | 27.771149 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 97     | Encrypted Alert                                                                                        |
| 43  | 27.771168 | 10.54.15.100 | 10.54.15.15  | TCP      | 54     | 45114 → 443 [RST] Seq=913 Win=0 Len=0                                                                  |
| 44  | 27.771176 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45114 [ACK] Seq=2715 Ack=913 Win=17608 Len=0 TSval=30725 TSecr=2646662                           |

## 2.7.2.1 Normal HTTPS

**Why?** Because the packets are encrypted, which is what we should be expecting when looking at traffic on port 443.

Let's take a look at packets for HTTPS traffic.



## 2.7.2.1 Normal HTTPS

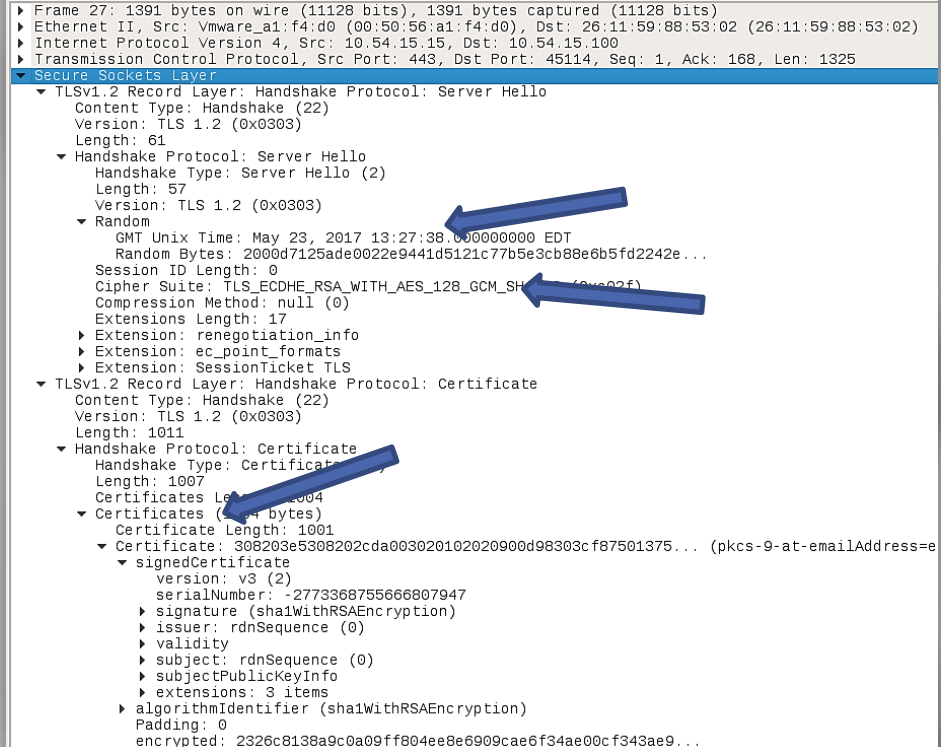
As you may remember, the **Secure Sockets Layer** portion of the packet details should **not** be empty. Here we see pertinent information as to what the client has available in order to attempt to establish secure communications with the server, **Client Hello** packet. We see the following:

- Content Type = Handshake
- Handshake Protocol: Client Hello
- Version: TLS 1.2
- Cipher Suites (11 suites)
- Compression Method (1 method)

```
▶ Frame 25: 233 bytes on wire (1864 bits), 233 bytes captured (1864 bits)
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:61:66 (00:50:56:a1:61:66)
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.15
▶ Transmission Control Protocol, Src Port: 45114, Dst Port: 443, Seq: 1, Ack: 1, Len: 167
▼ Secure Sockets Layer
  ▼ TLSv1.2 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 162
    ▼ Handshake Protocol: Client Hello
      Handshake Type: Client Hello (1)
      Length: 158
      Version: TLS 1.2 (0x0303)
      ▶ Random
      Session ID Length: 0
      Cipher Suites Length: 22
      ▼ Cipher Suites (11 suites)
        Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
        Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
        Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (0xc00a)
        Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (0xc009)
        Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)
        Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)
        Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0x0033)
        Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA (0x0039)
        Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
        Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)
        Cipher Suite: TLS_RSA_WITH_3DES_EDE_CBC_SHA (0x000a)
      Compression Methods Length: 1
      ▼ Compression Methods (1 method)
        Compression Method: null (0)
      Extensions Length: 95
      ▶ Extension: renegotiation_info
      ▶ Extension: elliptic_curves
      ▶ Extension: ec_point_formats
      ▶ Extension: SessionTicket TLS
      ▶ Extension: next_protocol_negotiation
      ▶ Extension: Application Layer Protocol Negotiation
      ▶ Extension: status_request
      ▶ Extension: signature_algorithms
```

## 2.7.2.1 Normal HTTPS

Here is the server's response, **Server Hello** packet. Again, these Hello messages will contain the availability and agreement of encryption algorithms to work with and exchange random values that will be used for generation of key.

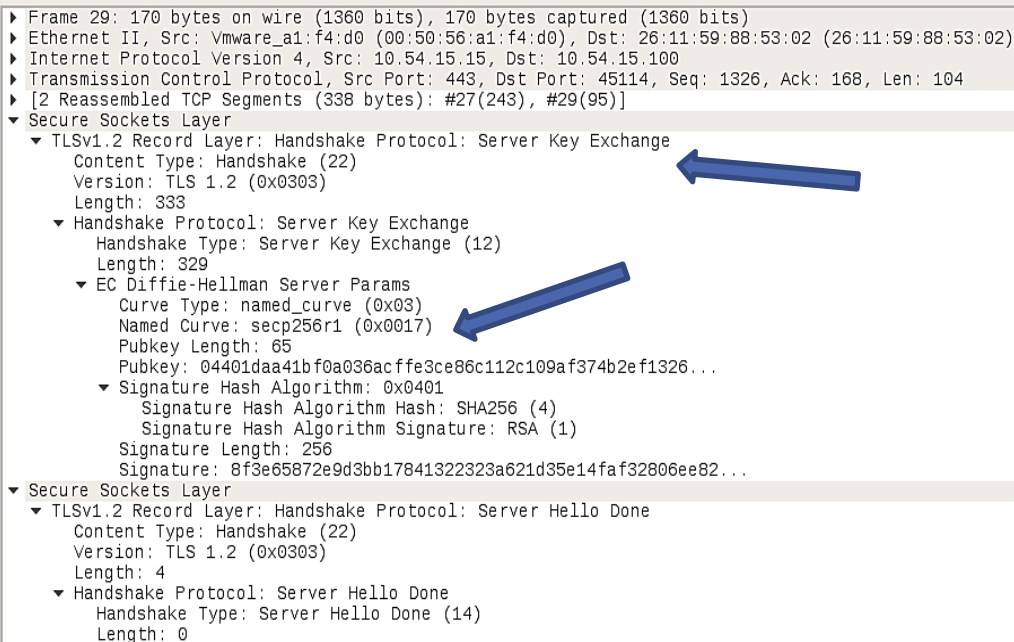


```
▶ Frame 27: 1391 bytes on wire (11128 bits), 1391 bytes captured (11128 bits) on 0
▶ Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
▶ Internet Protocol Version 4, Src: 10.54.15.15, Dst: 10.54.15.100
▶ Transmission Control Protocol, Src Port: 443, Dst Port: 45114, Seq: 1, Ack: 168, Len: 1325
▼ Secure Sockets Layer
  ▼ TLSv1.2 Record Layer: Handshake Protocol: Server Hello
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 61
    ▼ Handshake Protocol: Server Hello
      Handshake Type: Server Hello (2)
      Length: 57
      Version: TLS 1.2 (0x0303)
      Random
      GMT Unix Time: May 23, 2017 13:27:38.000000000 EDT
      Random Bytes: 2000d7125ade0022e9441d5121c77b5e3cb88e6b5fd2242e...
      Session ID Length: 0
      Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
      Compression Method: null (0)
      Extensions Length: 17
      ▶ Extension: renegotiation_info
      ▶ Extension: ec_point_formats
      ▶ Extension: SessionTicket TLS
    ▼ TLSv1.2 Record Layer: Handshake Protocol: Certificate
      Content Type: Handshake (22)
      Version: TLS 1.2 (0x0303)
      Length: 1011
      ▼ Handshake Protocol: Certificate
        Handshake Type: Certificate
        Length: 1007
        Certificates Length: 1004
        ▼ Certificates (1004 bytes)
          Certificate Length: 1001
          ▼ Certificate: 308203e5308202cda003020102020900d98303cf87501375... (pkcs-9-at-emailAddress=...)
            ▼ signedCertificate
              version: v3 (2)
              serialNumber: -2773368755666807947
              ▶ signature (sha1WithRSAEncryption)
              ▶ issuer: rdnSequence (0)
              ▶ validity
              ▶ subject: rdnSequence (0)
              ▶ subjectPublicKeyInfo
              ▶ extensions: 3 items
            ▶ algorithmIdentifier (sha1WithRSAEncryption)
            Padding: 0
            encrypted: 2326c8138a9c0a09ff804ee8e6909cae6f34ae00cf343ae9...
```

## 2.7.2.1 Normal HTTPS

Here we see the **Server Key Exchange**, which will be followed by the **Client Key Exchange** packet.

This will be considered step #3 in the establishment of an SSL/TLS session between a server and a client.

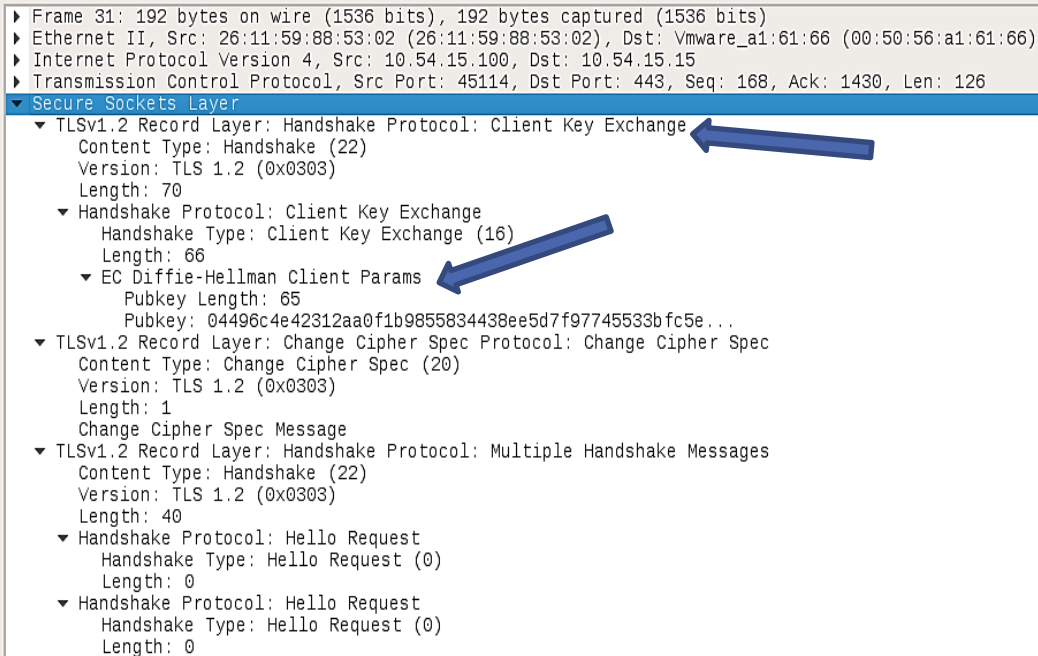


The image shows a Wireshark packet capture of an HTTPS connection. The selected packet is Frame 29, which is a TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange. Two blue arrows point to specific fields: one points to the 'Handshake Type: Server Key Exchange (12)' field, and the other points to the 'EC Diffie-Hellman Server Params' section, specifically the 'Named Curve: secp256r1 (0x0017)' field.

```
▶ Frame 29: 170 bytes on wire (1360 bits), 170 bytes captured (1360 bits) on interface 0
  ▶ Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
  ▶ Internet Protocol Version 4, Src: 10.54.15.15, Dst: 10.54.15.100
  ▶ Transmission Control Protocol, Src Port: 443, Dst Port: 45114, Seq: 1326, Ack: 168, Len: 104
  ▶ [2 Reassembled TCP Segments (338 bytes): #27(243), #29(95)]
  ▼ Secure Sockets Layer
    ▼ TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange
      Content Type: Handshake (22)
      Version: TLS 1.2 (0x0303)
      Length: 333
      ▼ Handshake Protocol: Server Key Exchange
        Handshake Type: Server Key Exchange (12)
        Length: 329
        ▼ EC Diffie-Hellman Server Params
          Curve Type: named_curve (0x03)
          Named Curve: secp256r1 (0x0017)
          Pubkey Length: 65
          Pubkey: 04401daa41bf0a036acffe3ce86c112c109af374b2ef1326...
          ▼ Signature Hash Algorithm: 0x0401
            Signature Hash Algorithm Hash: SHA256 (4)
            Signature Hash Algorithm Signature: RSA (1)
            Signature Length: 256
            Signature: 8f3e65872e9d3bb17841322323a621d35e14faf32806ee82...
        ▼ Secure Sockets Layer
          ▼ TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
            Content Type: Handshake (22)
            Version: TLS 1.2 (0x0303)
            Length: 4
            ▼ Handshake Protocol: Server Hello Done
              Handshake Type: Server Hello Done (14)
              Length: 0
```

## 2.7.2.1 Normal HTTPS

Here we see the  
**Client Key Exchange**  
packet.



```
▶ Frame 31: 192 bytes on wire (1536 bits), 192 bytes captured (1536 bits)
▶ Ethernet II, Src: 26:11:59:88:53:02 (26:11:59:88:53:02), Dst: Vmware_a1:61:66 (00:50:56:a1:61:66)
▶ Internet Protocol Version 4, Src: 10.54.15.100, Dst: 10.54.15.15
▶ Transmission Control Protocol, Src Port: 45114, Dst Port: 443, Seq: 168, Ack: 1430, Len: 126
▼ Secure Sockets Layer
  ▼ TLSv1.2 Record Layer: Handshake Protocol: Client Key Exchange ←
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 70
    ▼ Handshake Protocol: Client Key Exchange
      Handshake Type: Client Key Exchange (16)
      Length: 66
      ▼ EC Diffie-Hellman Client Params ←
        Pubkey Length: 65
        Pubkey: 04496c4e42312aa0f1b9855834438ee5d7f97745533bfc5e...
      ▼ TLSv1.2 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
        Content Type: Change Cipher Spec (20)
        Version: TLS 1.2 (0x0303)
        Length: 1
        Change Cipher Spec Message
      ▼ TLSv1.2 Record Layer: Handshake Protocol: Multiple Handshake Messages
        Content Type: Handshake (22)
        Version: TLS 1.2 (0x0303)
        Length: 40
        ▼ Handshake Protocol: Hello Request
          Handshake Type: Hello Request (0)
          Length: 0
        ▼ Handshake Protocol: Hello Request
          Handshake Type: Hello Request (0)
          Length: 0
```

## 2.7.2.1 Normal HTTPS

This is the last packet and the handshake between the server and client is now complete.

The rest of the packets between these 2 devices will now be encrypted.

```
▶ Frame 34: 324 bytes on wire (2592 bits), 324 bytes captured (2592 bits)
▶ Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
▶ Internet Protocol Version 4, Src: 10.54.15.15, Dst: 10.54.15.100
▶ Transmission Control Protocol, Src Port: 443, Dst Port: 45114, Seq: 1430, Ack: 604, Len: 258
▼ Secure Sockets Layer
  ▼ TLSv1.2 Record Layer: Handshake Protocol: New Session Ticket ←
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 202
    ▼ Handshake Protocol: New Session Ticket
      Handshake Type: New Session Ticket (4)
      Length: 198
      ▼ TLS Session Ticket
        Session Ticket Lifetime Hint: 300
        Session Ticket Length: 192
        Session Ticket: c87ec842e1a7e7c2fd503729435f618d50f9e59487ae8647...
  ▼ TLSv1.2 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
    Content Type: Change Cipher Spec (20)
    Version: TLS 1.2 (0x0303)
    Length: 1
    Change Cipher Spec Message
  ▼ TLSv1.2 Record Layer: Handshake Protocol: Encrypted Handshake Message ←
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 40
    Handshake Protocol: Encrypted Handshake Message
```

## 2.7.2.1 Normal HTTPS

The traffic is unreadable, but if this is internal traffic within our corporate environment, then it is feasible to decrypt this traffic using the private key from the internal server.

```
▶ Frame 35: 770 bytes on wire (6160 bits), 770 bytes captured (6160 bits)
▶ Ethernet II, Src: Vmware_a1:f4:d0 (00:50:56:a1:f4:d0), Dst: 26:11:59:88:53:02 (26:11:59:88:53:02)
▶ Internet Protocol Version 4, Src: 10.54.15.15, Dst: 10.54.15.100
▶ Transmission Control Protocol, Src Port: 443, Dst Port: 45114, Seq: 1688, Ack: 604, Len: 704
▼ Secure Sockets Layer
  ▶ TLSv1.2 Record Layer: Application Data Protocol: http-over-tls
  ▶ TLSv1.2 Record Layer: Application Data Protocol: http-over-tls
  ▶ TLSv1.2 Record Layer: Application Data Protocol: http-over-tls
  ▼ TLSv1.2 Record Layer: Application Data Protocol: http-over-tls
    Content Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 32
    Encrypted Application Data: bb006752c8e53aef6bb13c15ff590c829883685da8b96e44...
```

Most enterprise appliances have this feature built-in, but you will need to perform this task manually if you wish to read the decrypted traffic.

## 2.7.2.1 Normal HTTPS

We'll look at a packet capture file from the Wireshark website and as well as the process to decrypt SSL traffic using the private key.

You can find the packet capture called snakeoil2\_070531.tgz, and private key [here](#).

## 2.7.2.1 Normal HTTPS

**Note:** This packet capture uses SSLv3 and the handshake is different from what we saw in the previous packet capture which used TLSv1.2.

| Time        | Source    | Destination | Protocol | Length | Info                                                                                                          |
|-------------|-----------|-------------|----------|--------|---------------------------------------------------------------------------------------------------------------|
| 1 0.000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 38713 → 443 [SYN] Seq=0 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525562106 TSecr=0 WS=1                    |
| 2 0.000021  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 443 → 38713 [SYN, ACK] Seq=0 Ack=1 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525562115 TSecr=525562106 WS=1 |
| 3 0.000037  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=1 Ack=1 Win=32767 Len=0 TSval=525562115 TSecr=525562115                                 |
| 4 0.000158  | 127.0.0.1 | 127.0.0.1   | SSLv2    | 171    | Client Hello                                                                                                  |
| 5 0.000178  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1 Ack=106 Win=32767 Len=0 TSval=525562115 TSecr=525562115                               |
| 6 0.002160  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 995    | Server Hello, Certificate, Server Hello Done                                                                  |
| 7 0.002609  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=106 Ack=930 Win=32767 Len=0 TSval=525562117 TSecr=525562117                             |
| 8 2.808933  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 278    | Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message                                          |
| 9 2.822770  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 141    | Change Cipher Spec, Encrypted Handshake Message                                                               |
| 10 2.822809 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=318 Ack=1005 Win=32767 Len=0 TSval=525564938 TSecr=525564938                            |
| 11 2.833071 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 503    | Application Data                                                                                              |
| 12 2.873275 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1005 Ack=755 Win=32767 Len=0 TSval=525564989 TSecr=525564948                            |
| 13 2.938485 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 103    | Encrypted Handshake Message                                                                                   |
| 14 2.938750 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 183    | Encrypted Handshake Message                                                                                   |
| 15 2.938761 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1042 Ack=872 Win=32767 Len=0 TSval=525565054 TSecr=525565054                            |
| 16 2.938999 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 1073   | Encrypted Handshake Message, Encrypted Handshake Message, Encrypted Handshake Message                         |
| 17 2.940026 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 337    | Encrypted Handshake Message, Change Cipher Spec, Encrypted Handshake Message                                  |
| 18 2.943406 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 172    | Change Cipher Spec, Encrypted Handshake Message                                                               |
| 19 2.944825 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 5756   | Application Data, Application Data                                                                            |
| 20 2.944864 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=1143 Ack=7845 Win=32767 Len=0 TSval=525565060 TSecr=525565059                           |
| 21 2.964424 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 471    | Application Data                                                                                              |

## 2.7.2.1 Normal HTTPS

As expected, the Secure Sockets Layer field of the packet indeed has information and we see the Encrypted Application Data, that indeed is encrypted.

Now we'll look at the process to manually decrypt this traffic using the server's private key and importing it into Wireshark.

```

▶ Frame 11: 503 bytes on wire (4024 bits), 503 bytes captured (4024 bits)
▶ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▶ Transmission Control Protocol, Src Port: 38713, Dst Port: 443, Seq: 318, Ack: 1005, Len: 437
▲ Secure Sockets Layer
  ▲ SSLv3 Record Layer: Application Data Protocol: http-over-tls
    Content Type: Application Data (23)
    Version: SSL 3.0 (0x0300)
    Length: 432
    Encrypted Application Data: 4ac33e9d7778012cb4bc4c9a84d7b9900c2110f0fa007c16...

0040  7c 0a 17 03 00 01 b0 4a c3 3e 9d 77 78 01 2c b4 |.....J .>.wx..
0050  bc 4c 9a 84 d7 b9 90 0c 21 10 f0 fa 00 7c 16 bb |L.....!...|..
0060  77 fb 72 42 4f ad 50 4a d0 aa 6f aa 44 6c 62 94 |w.rB0.PJ .o.Dlb.
0070  1b c5 fe e9 1c 5e de 85 0b 0e 05 e4 18 6e d2 d3 |.....^.....n.
0080  b5 20 ab 81 fd 18 9a 73 b8 d7 ef c3 dd 74 d7 9c |.....s.....t..
0090  1e 6f 21 6d f8 24 ca 3c 70 78 36 12 7a 8a 9c ac |o.l.m.$.< px6.z..
00a0  4e 1c a8 fb 27 30 ba 9a f4 2f 0a ab 80 6a a1 60 |N... '0.. ./...j.
00b0  74 f0 e3 91 84 e7 90 88 cc f0 95 7b 0a 22 f2 f9 |t.....{..".
00c0  27 e0 dd 38 0c fd e9 03 71 dc 70 a4 6e df e3 72 |'.8.... q.p.n..r
00d0  9e a1 f0 c9 00 d6 03 55 6a 67 5d 9c b8 75 01 b0 |.....U jg]..u..
00e0  01 9f e6 d2 44 18 bc ca 7a 10 39 a6 cf 15 c7 f5 |.....D...z.9....
00f0  35 d4 b3 6d 91 23 84 99 ba b0 7e d0 c9 4c bf 3f |5..m.#...~...L?
0100  33 68 37 b7 7d 44 b0 0b 2c 0f d0 75 a2 6b 5b e1 |3h7.}D...u.k[.
0110  9f d4 69 9a 14 c8 29 b7 d9 10 bb 99 30 9a fb cc |...i...). ....0...
0120  13 1f 76 4e e6 df 14 aa d5 60 bf 91 49 0d 64 42 |..vN.... ^...I.dB
0130  29 a8 64 27 d4 5e 1b 18 03 a8 73 d6 05 6e f7 50 |).d'.^...s..n.P
0140  b0 09 6b 69 7a 12 28 58 ef 5a 86 11 de 71 71 9f |..kiz.(X .Z...qq.
0150  ca bd 79 2a c2 e5 9b 5e 32 e7 cb 97 6e a0 ea a4 |..y*...^ 2....n...
0160  a4 6a 32 f9 37 39 d8 37 6d 63 f3 08 1c dd 06 dd |.j2.79.7 mc....

No.: 11 • Time: 2.833071 • Source: 127.0.0.1 • Destination: 127.0.0.1 • Protocol: SSLv3 • Length: 503 • Info: Application Data
```

### 2.7.2.1 Normal HTTPS

Before doing so, let's take a look at the Export HTTP object list to see if anything is shown that we can export from the packet capture.



## 2.7.2.1 Normal HTTPS

We see from the screenshot below that there is nothing shown that we can export.

Wireshark packet capture showing HTTPS traffic. The packet list on the left shows a series of TCP and SSLv3 packets. The packet details on the right show the SSLv3 handshake process, including the ClientHello and ServerHello messages. The packet bytes pane at the bottom shows the raw data of the selected packet.

4: 511 bytes on wire (4088 bits), 511 bytes captured (4088 bits)

### 2.7.2.1 Normal HTTPS

# Let's decrypt this traffic.

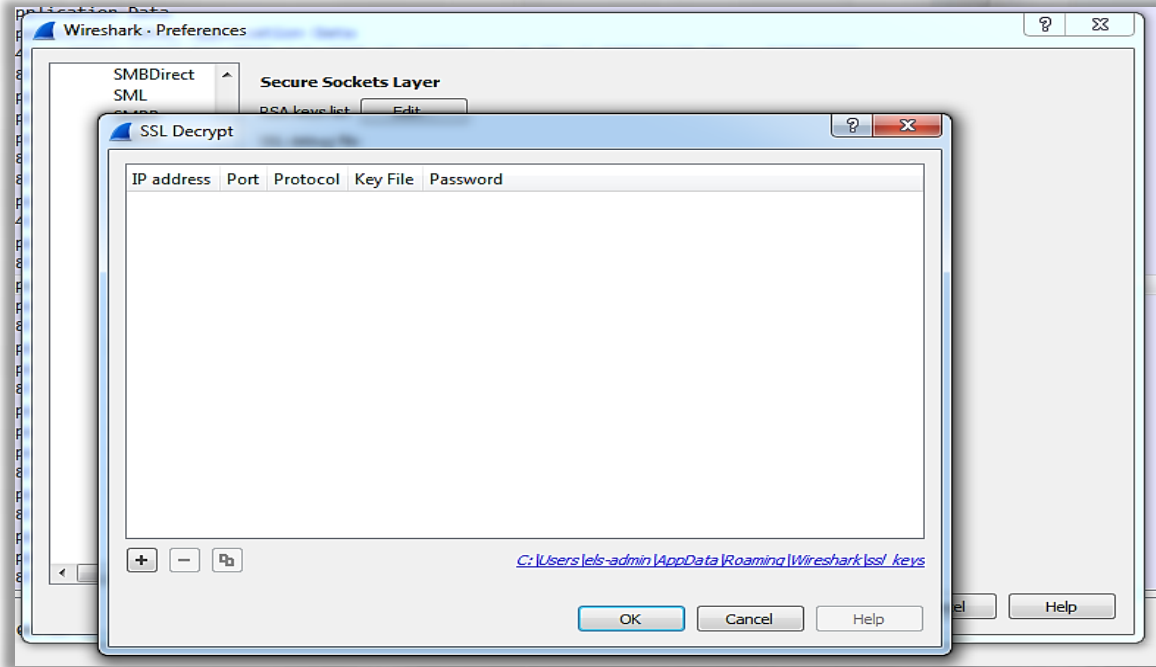
In order to import the private key into Wireshark we'll need to go to **Edit > Preferences > Protocols > SSL**. Now select **Edit** next to **RSA keys list**.

## 2.7.2.1 Normal HTTPS

You should see the following window. Use the + symbol to import the private key.

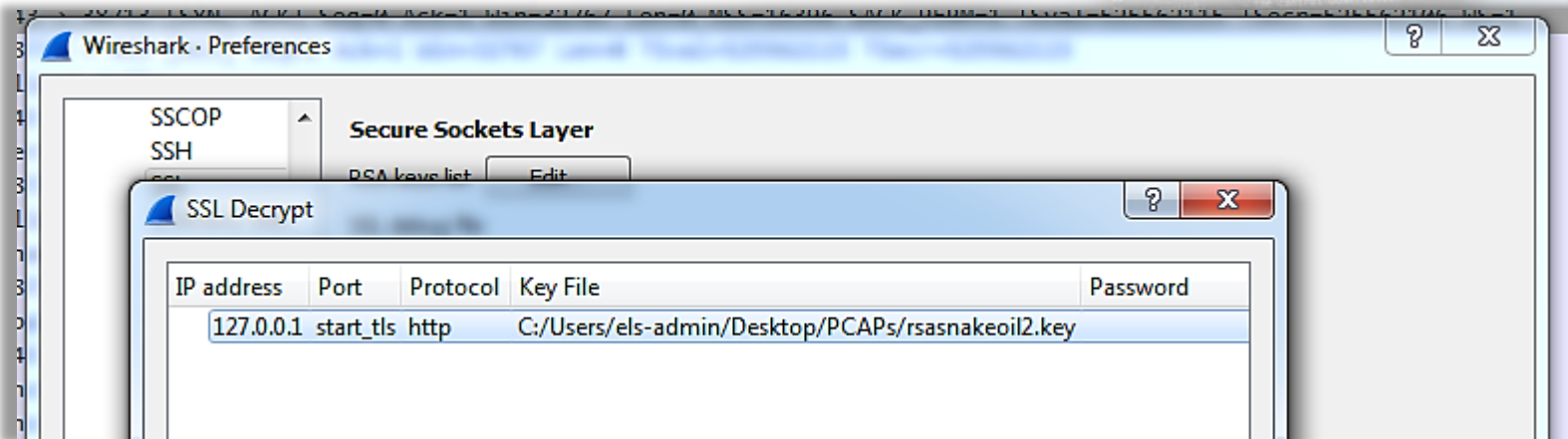
You will need to enter the IP address of the server with the private key, Port, Protocol, and the location of the Key File.

Refer to this [page](https://wiki.wireshark.org/SSL) for more guidance on how to import the key for this specific packet capture.



## 2.7.2.1 Normal HTTPS

Here is the information which was entered. Your configuration should look similar except for the location of the key file of course.



## 2.7.2.1 Normal HTTPS

In this screenshot, we see that the traffic was decrypted. The decrypted traffic is highlighted in green.

| Time        | Source    | Destination | Protocol | Length | Info                                                                                                          |
|-------------|-----------|-------------|----------|--------|---------------------------------------------------------------------------------------------------------------|
| 1 0.000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 38713 → 443 [SYN] Seq=0 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525562106 TSecr=0 WS=1                    |
| 2 0.000021  | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 443 → 38713 [SYN, ACK] Seq=0 Ack=1 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525562115 TSecr=525562106 WS=1 |
| 3 0.000037  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=1 Ack=1 Win=32767 Len=0 TSval=525562115 TSecr=525562115                                 |
| 4 0.000158  | 127.0.0.1 | 127.0.0.1   | SSLv2    | 171    | Client Hello                                                                                                  |
| 5 0.000178  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1 Ack=106 Win=32767 Len=0 TSval=525562115 TSecr=525562115                               |
| 6 0.002160  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 995    | Server Hello, Certificate, Server Hello Done                                                                  |
| 7 0.002609  | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=106 Ack=930 Win=32767 Len=0 TSval=525562117 TSecr=525562117                             |
| 8 2.808933  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 278    | Client Key Exchange, Change Cipher Spec, Finished                                                             |
| 9 2.822770  | 127.0.0.1 | 127.0.0.1   | SSLv3    | 141    | Change Cipher Spec, Finished                                                                                  |
| 10 2.822809 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=318 Ack=1005 Win=32767 Len=0 TSval=525564938 TSecr=525564938                            |
| 11 2.833071 | 127.0.0.1 | 127.0.0.1   | HTTP     | 503    | GET / HTTP/1.1                                                                                                |
| 12 2.873275 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1005 Ack=755 Win=32767 Len=0 TSval=525564989 TSecr=525564948                            |
| 13 2.938485 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 103    | Hello Request                                                                                                 |
| 14 2.938750 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 183    | Client Hello                                                                                                  |
| 15 2.938761 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38713 [ACK] Seq=1042 Ack=872 Win=32767 Len=0 TSval=525565054 TSecr=525565054                            |
| 16 2.938999 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 1073   | Server Hello, Certificate, Server Hello Done                                                                  |
| 17 2.940026 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 337    | Client Key Exchange, Change Cipher Spec, Finished                                                             |
| 18 2.943406 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 172    | Change Cipher Spec, Finished                                                                                  |
| 19 2.944825 | 127.0.0.1 | 127.0.0.1   | HTTP     | 5756   | HTTP/1.1 200 OK (text/html)                                                                                   |
| 20 2.944864 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38713 → 443 [ACK] Seq=1143 Ack=7845 Win=32767 Len=0 TSval=525565060 TSecr=525565059                           |
| 21 2.964424 | 127.0.0.1 | 127.0.0.1   | HTTP     | 471    | GET /icons/jhe061.png HTTP/1.1                                                                                |
| 22 2.964572 | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 38714 → 443 [SYN] Seq=0 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525565080 TSecr=0 WS=1                    |
| 23 2.964588 | 127.0.0.1 | 127.0.0.1   | TCP      | 74     | 443 → 38714 [SYN, ACK] Seq=0 Ack=1 Win=32767 Len=0 MSS=16396 SACK_PERM=1 TSval=525565080 TSecr=525565080 WS=1 |
| 24 2.964598 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38714 → 443 [ACK] Seq=1 Ack=1 Win=32767 Len=0 TSval=525565080 TSecr=525565080                                 |
| 25 2.964810 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 186    | Client Hello                                                                                                  |
| 26 2.964819 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 443 → 38714 [ACK] Seq=1 Ack=121 Win=32767 Len=0 TSval=525565080 TSecr=525565080                               |
| 27 2.992274 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 220    | Server Hello, Change Cipher Spec, Finished                                                                    |
| 28 2.992312 | 127.0.0.1 | 127.0.0.1   | TCP      | 66     | 38714 → 443 [ACK] Seq=121 Ack=155 Win=32767 Len=0 TSval=525565108 TSecr=525565108                             |
| 29 2.992855 | 127.0.0.1 | 127.0.0.1   | HTTP     | 562    | GET /icons/debian/openlogo-25.jpg HTTP/1.1                                                                    |

## 2.7.2.1 Normal HTTPS

If after you enter the information for the key file and the packet capture is not showing decrypted traffic, try closing Wireshark and re-opening it.

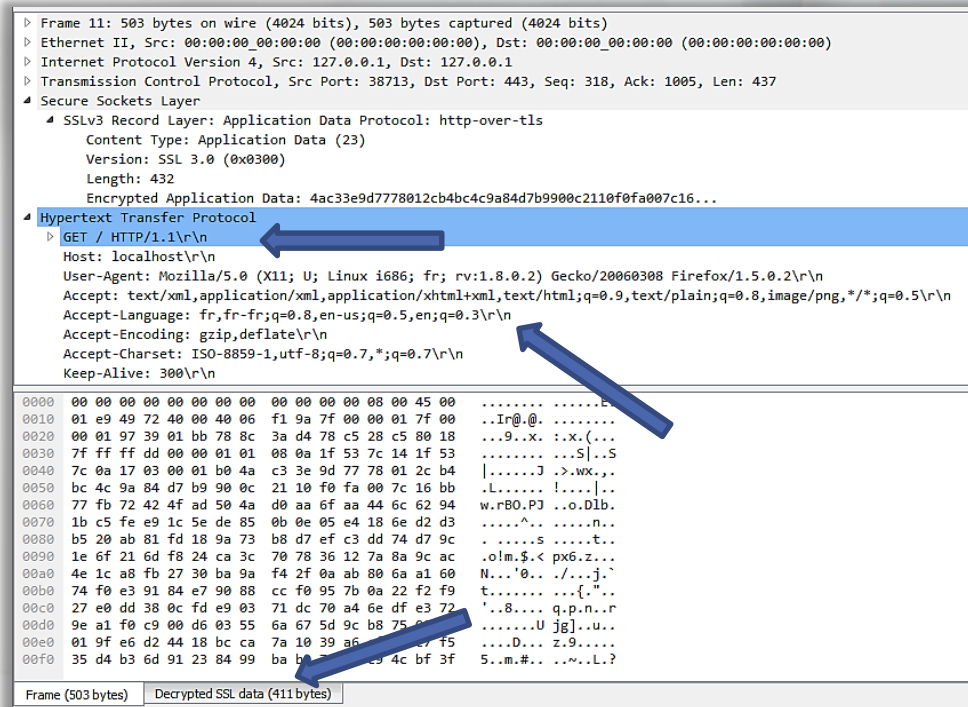
That might resolve your issue if everything was configured correctly.

```
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - @control
33   evaluate_candidates
34
35   freeze
36 end
37
38 # TODO: the experiment's control
39
40 # TODO: the name of the experiment
41 def experiment_name
42   @experiment_name
43 end
44
45 # TODO: was the result a match between all
46 def matched?
47   @matched
48 end
49
50 # TODO: result up to 11
```

## 2.7.2.1 Normal HTTPS

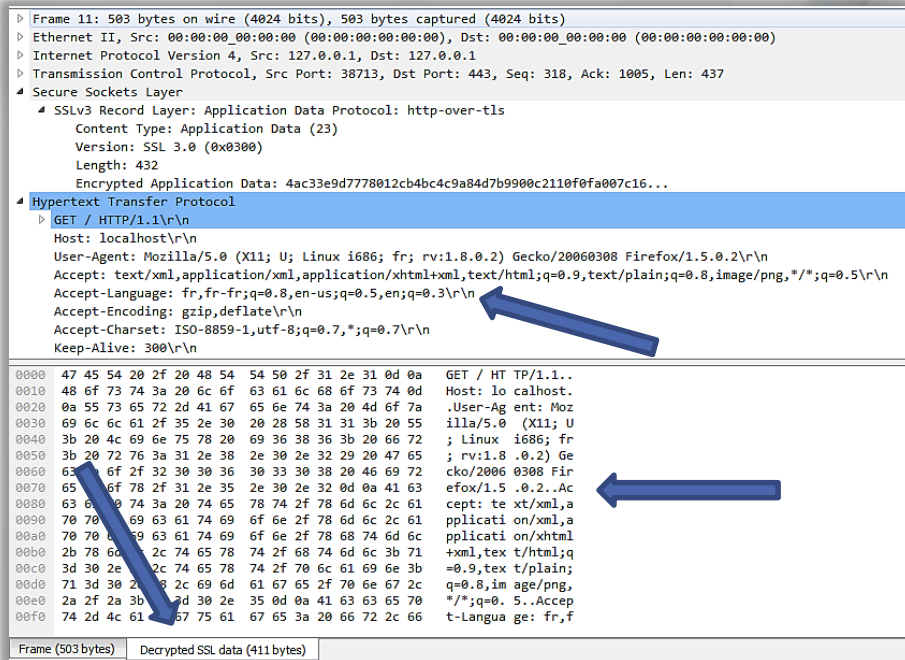
Here is a closer look at a decrypted packet. Under Secure Sockets Layer we now see **Hypertext Transfer Protocol** information. We also see a new tab at the bottom titled **Decrypted SSL Data**.

As you can see, we can still see the decrypted data without going to the tab. The next screen shot will show the details when the tab is selected.



## 2.7.2.1 Normal HTTPS

When the Decrypted SSL data tab is selected, we can see the following details.



```
> Frame 11: 503 bytes on wire (4024 bits), 503 bytes captured (4024 bits) on interface 0
> Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 38713, Dst Port: 443, Seq: 318, Ack: 1005, Len: 437
+ Secure Sockets Layer
  + SSLv3 Record Layer: Application Data Protocol: http-over-tls
    Content Type: Application Data (23)
    Version: SSL 3.0 (0x0300)
    Length: 432
    Encrypted Application Data: 4ac33e9d7778012cb4bc4c9a84d7b9900c2110f0fa007c16...
  + Hypertext Transfer Protocol
    > GET / HTTP/1.1\r\n
      Host: localhost\r\n
      User-Agent: Mozilla/5.0 (X11; U; Linux i686; fr; rv:1.8.0.2) Gecko/20060308 Firefox/1.5.0.2\r\n
      Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5\r\n
      Accept-Language: fr,fr-fr;q=0.8,en-us;q=0.5,en;q=0.3\r\n
      Accept-Encoding: gzip,deflate\r\n
      Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
      Keep-Alive: 300\r\n

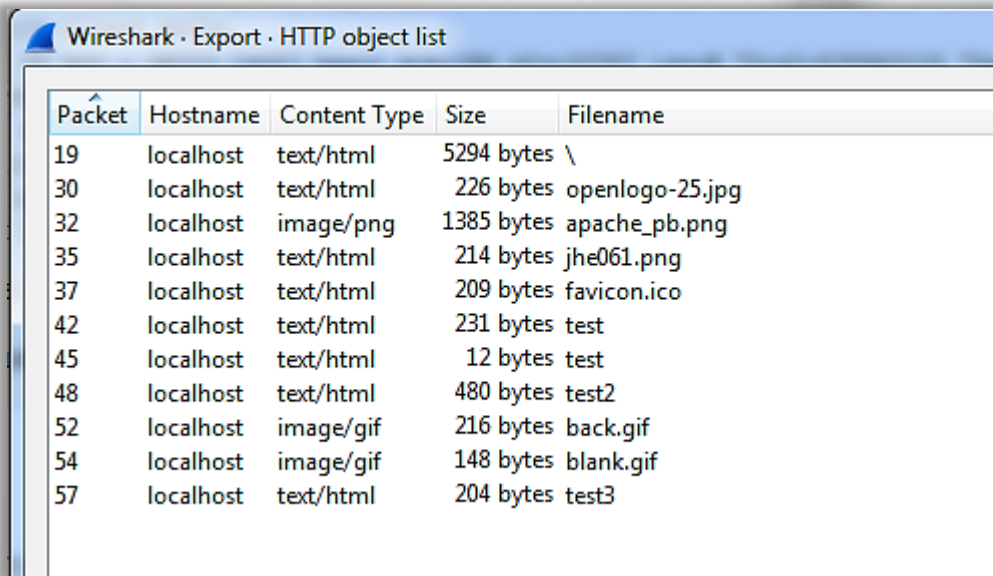
0000  47 45 54 20 2f 20 48 54 54 50 2f 31 2e 31 0d 0a  GET / HT TP/1.1..
0010  48 6f 73 74 3a 20 6c 6f 63 61 6c 68 6f 73 74 0d  Host: lo calhost.
0020  0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f 7a  .User-Ag ent: Moz
0030  69 6c 6c 61 2f 35 2e 30 20 28 58 31 31 3b 20 55  illa/5.0 (X11; U
0040  3b 20 4c 69 6e 75 78 20 69 36 38 36 3b 20 66 72  ; Linux i686; fr
0050  3b 20 72 76 3a 31 2e 38 2e 30 2e 32 29 20 47 65  ; rv:1.8.0.2) Ge
0060  63 6f 2f 32 30 30 36 30 33 30 38 20 46 69 72  cko/2006 0308 Fir
0070  65 6f 78 2f 31 2e 35 2e 30 2e 32 0d 0a 41 63  efox/1.5 .0.2..Ac
0080  63 6f 74 3a 20 74 65 78 74 2f 78 6d 6c 2c 61  cept: te xt/xml,a
0090  70 70 6f 69 63 61 74 69 6f 6e 2f 78 6d 6c 2c 61  pplicati on/xml,a
00a0  70 70 6f 69 63 61 74 69 6f 6e 2f 78 68 74 6d 6c  pplicati on/xhtml
00b0  2b 78 6d 6c 2c 74 65 78 74 2f 68 74 6d 6c 3b 71  +xml,tex t/html;q
00c0  3d 30 2e 2c 74 65 78 74 2f 70 6c 61 69 6e 3b  =0.9,tex t/plain;
00d0  71 3d 30 2e 2c 69 6d 61 67 65 2f 70 6e 67 2c  q=0.8,im age/png,
00e0  2a 2f 2a 3b 2d 30 2e 35 0d 0a 41 63 63 65 70  /*;q=0. 5..Accep
00f0  74 2d 4c 61 67 75 61 67 65 3a 20 66 72 2c 66  t-Langua ge: fr,f
```

Frame (503 bytes)    Decrypted SSL data (411 bytes)

## 2.7.2.1 Normal HTTPS

Now, if we check for any HTTP objects that we can export, we actually see several images and HTML pages from the decrypted traffic.

Again, this is possible due to having access to the private key from the server, which typically in an internal corporate environment you should have access to.

A screenshot of the 'Wireshark · Export · HTTP object list' window. It displays a table of exported HTTP objects from a capture. The table has five columns: Packet, Hostname, Content Type, Size, and Filename. The data rows show various files like HTML pages, images (jpg, png, gif), and icons (ico) all originating from 'localhost'.

| Packet | Hostname  | Content Type | Size       | Filename        |
|--------|-----------|--------------|------------|-----------------|
| 19     | localhost | text/html    | 5294 bytes | \               |
| 30     | localhost | text/html    | 226 bytes  | openlogo-25.jpg |
| 32     | localhost | image/png    | 1385 bytes | apache_pb.png   |
| 35     | localhost | text/html    | 214 bytes  | jhe061.png      |
| 37     | localhost | text/html    | 209 bytes  | favicon.ico     |
| 42     | localhost | text/html    | 231 bytes  | test            |
| 45     | localhost | text/html    | 12 bytes   | test            |
| 48     | localhost | text/html    | 480 bytes  | test2           |
| 52     | localhost | image/gif    | 216 bytes  | back.gif        |
| 54     | localhost | image/gif    | 148 bytes  | blank.gif       |
| 57     | localhost | text/html    | 204 bytes  | test3           |

## 2.7.2.2 Suspicious HTTPS

Now we'll look at some suspicious HTTPS traffic.

In this example we'll look at the [CrypMIC Ransomware](#). Within this malicious traffic, we're only going to focus on how the malware used port 443 to display the ransomware note to the victim.

## 2.7.2.2 Suspicious HTTPS

“CrypMIC and CryptXXX share many similarities; both are spread by the Neutrino Exploit Kit and use the same format for sub-versionID/botID (U[6digits] / UXXXXXX]) and export function name (MS1, MS2).

***Both threats also employed a custom protocol via TCP Port 443 to communicate with their command-and-control (C&C) servers.”*** – Trend Labs

## 2.7.2.2 Suspicious HTTPS

Let's start by taking a look at the different protocols within this packet capture, **Statistics > Protocol Hierarchy**.

| Protocol                        | Percent Packets | Packets | Percent Bytes | Bytes  | Bits/s | End Packets | End Bytes | End Bits/s |
|---------------------------------|-----------------|---------|---------------|--------|--------|-------------|-----------|------------|
| ▼ Frame                         | 100.0           | 878     | 100.0         | 710011 | 23 k   | 0           | 0         | 0          |
| ▼ Ethernet                      | 100.0           | 878     | 1.7           | 12292  | 413    | 0           | 0         | 0          |
| ▼ Internet Protocol Version 4   | 100.0           | 878     | 2.5           | 17560  | 590    | 0           | 0         | 0          |
| ▼ Transmission Control Protocol | 100.0           | 878     | 95.6          | 678479 | 22 k   | 676         | 429856    | 14 k       |
| Secure Sockets Layer            | 4.0             | 35      | 5.0           | 35786  | 1203   | 35          | 35786     | 1203       |
| ▼ Hypertext Transfer Protocol   | 1.3             | 11      | 12.2          | 86590  | 2912   | 6           | 2200      | 73         |
| Media Type                      | 0.1             | 1       | 10.9          | 77493  | 2606   | 1           | 77675     | 2612       |
| Line-based text data            | 0.5             | 4       | 1.5           | 10709  | 360    | 4           | 6562      | 220        |
| Data                            | 17.8            | 156     | 28.8          | 204260 | 6869   | 156         | 204260    | 6869       |

## 2.7.2.2 Suspicious HTTPS

Next, we can look at **Statistics > Endpoints**. We see there is a lot of traffic to several countries of concern, meaning that malware is known to originate from these destinations.

| Ethernet · 2  |       | IPv4 · 5 |  | IPv6  |               | TCP · 15    |               | UDP         |                    |                                            |               |           |           |  |  |  |
|---------------|-------|----------|--|-------|---------------|-------------|---------------|-------------|--------------------|--------------------------------------------|---------------|-----------|-----------|--|--|--|
| Address       | Port  | Packets  |  | Bytes | Packets A → B | Bytes A → B | Packets B → A | Bytes B → A | Country            | AS Number                                  | City          | Latitude  | Longitude |  |  |  |
| 51.254.30.225 | 80    | 532      |  | 445 k | 326           | 431 k       | 206           | 14 k        | France             | AS16276 OVH SAS                            | —             | 48.858200 | 2.338700  |  |  |  |
| 83.217.27.178 | 80    | 24       |  | 2184  | 7             | 839         | 17            | 1345        | Russian Federation | AS200161 DATAPRO Limited Liability Company | —             | 55.738602 | 37.606800 |  |  |  |
| 85.14.243.9   | 443   | 311      |  | 257 k | 200           | 250 k       | 111           | 6792        | Germany            | AS24961 myLoc managed IT AG                | —             | 51.299301 | 9.490900  |  |  |  |
| 89.36.89.80   | 80    | 11       |  | 4558  | 5             | 3861        | 6             | 697         | Romania            | AS43938 SC Gateway Telecom SRL             | Bucharest, 10 | 44.700001 | 26.450001 |  |  |  |
| 192.168.4.196 | 49284 | 11       |  | 4558  | 6             | 697         | 5             | 3861        | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49288 | 6        |  | 366   | 4             | 246         | 2             | 120         | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49289 | 18       |  | 1818  | 13            | 1099        | 5             | 719         | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49293 | 126      |  | 88 k  | 58            | 4721        | 68            | 84 k        | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49294 | 12       |  | 726   | 10            | 606         | 2             | 120         | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49301 | 394      |  | 356 k | 138           | 8799        | 256           | 347 k       | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49310 | 8        |  | 526   | 5             | 352         | 3             | 174         | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49311 | 13       |  | 2441  | 7             | 438         | 6             | 2003        | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49312 | 273      |  | 253 k | 89            | 5358        | 184           | 248 k       | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49313 | 9        |  | 600   | 5             | 320         | 4             | 280         | —                  | —                                          | —             | —         | —         |  |  |  |
| 192.168.4.196 | 49314 | 8        |  | 498   | 5             | 324         | 3             | 174         | —                  | —                                          | —             | —         | —         |  |  |  |

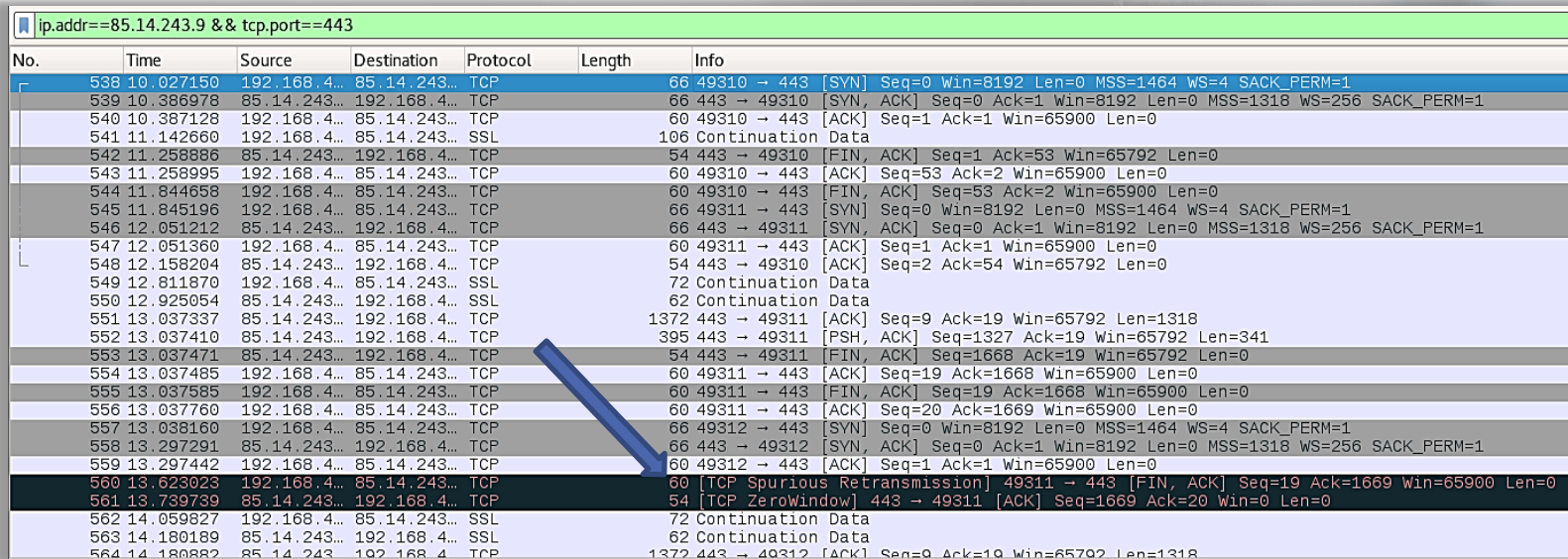
## 2.7.2.2 Suspicious HTTPS

Now we'll look at **Statistics > Conversations**. We are noticing a lot of traffic on port 443 to 85.14.243.9. This matches to what we saw within the **Endpoints** window.

| Ethernet · 1  |        | IPv4 · 4      |  | IPv6   | TCP · 11 |       | UDP           |             |               |             |           |          |              |              |
|---------------|--------|---------------|--|--------|----------|-------|---------------|-------------|---------------|-------------|-----------|----------|--------------|--------------|
| Address A     | Port A | Address B     |  | Port B | Packets  | Bytes | Packets A → B | Bytes A → B | Packets B → A | Bytes B → A | Rel Start | Duration | Bits/s A → B | Bits/s B → A |
| 192.168.4.196 | 49284  | 89.36.89.80   |  | 80     | 11       | 4558  | 6             | 697         | 5             | 3861        | 0.000000  | 1.9715   | 2828         | 15 k         |
| 192.168.4.196 | 49288  | 83.217.27.178 |  | 80     | 6        | 366   | 4             | 246         | 2             | 120         | 1.873187  | 174.1499 | 11           | 5            |
| 192.168.4.196 | 49289  | 83.217.27.178 |  | 80     | 18       | 1818  | 13            | 1099        | 5             | 719         | 1.873282  | 235.9954 | 37           | 24           |
| 192.168.4.196 | 49293  | 51.254.30.225 |  | 80     | 126      | 88 k  | 58            | 4721        | 68            | 84 k        | 3.462278  | 221.1622 | 170          | 3040         |
| 192.168.4.196 | 49294  | 51.254.30.225 |  | 80     | 12       | 726   | 10            | 606         | 2             | 120         | 3.462375  | 232.5032 | 20           | 4            |
| 192.168.4.196 | 49301  | 51.254.30.225 |  | 80     | 394      | 356 k | 138           | 8799        | 256           | 347 k       | 7.199412  | 2.5818   | 27 k         | 1076 k       |
| 192.168.4.196 | 49310  | 85.14.243.9   |  | 443    | 8        | 526   | 5             | 352         | 3             | 174         | 10.027150 | 2.1311   | 1321         | 653          |
| 192.168.4.196 | 49311  | 85.14.243.9   |  | 443    | 13       | 2441  | 7             | 438         | 6             | 2003        | 11.845196 | 1.8945   | 1849         | 8457         |
| 192.168.4.196 | 49312  | 85.14.243.9   |  | 443    | 273      | 253 k | 89            | 5358        | 184           | 248 k       | 13.038160 | 2.5466   | 16 k         | 779 k        |
| 192.168.4.196 | 49313  | 85.14.243.9   |  | 443    | 9        | 600   | 5             | 320         | 4             | 280         | 16.852456 | 1.3813   | 1853         | 1621         |
| 192.168.4.196 | 49314  | 85.14.243.9   |  | 443    | 8        | 498   | 5             | 324         | 3             | 174         | 84.801036 | 1.6513   | 1569         | 842          |

## 2.7.2.2 Suspicious HTTPS

Here we add a filter to focus on the specific IP address and port 443. Now before attempting to view the traffic, which should be encrypted, Wireshark is directing our attention to packets 560 & 561 within this screenshot.



ip.addr==85.14.243.9 && tcp.port==443

| No. | Time      | Source       | Destination  | Protocol | Length | Info                                                                                 |
|-----|-----------|--------------|--------------|----------|--------|--------------------------------------------------------------------------------------|
| 538 | 10.027150 | 192.168.4... | 85.14.243... | TCP      | 66     | 49310 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1                     |
| 539 | 10.386978 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49310 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SACK_PERM=1        |
| 540 | 10.387128 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                                        |
| 541 | 11.142660 | 192.168.4... | 85.14.243... | SSL      | 106    | Continuation Data                                                                    |
| 542 | 11.258886 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49310 [FIN, ACK] Seq=1 Ack=53 Win=65792 Len=0                                  |
| 543 | 11.258995 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [ACK] Seq=53 Ack=2 Win=65900 Len=0                                       |
| 544 | 11.844658 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [FIN, ACK] Seq=53 Ack=2 Win=65900 Len=0                                  |
| 545 | 11.845196 | 192.168.4... | 85.14.243... | TCP      | 66     | 49311 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1                     |
| 546 | 12.051212 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49311 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SACK_PERM=1        |
| 547 | 12.051360 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                                        |
| 548 | 12.158204 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49310 [ACK] Seq=2 Ack=54 Win=65792 Len=0                                       |
| 549 | 12.811870 | 192.168.4... | 85.14.243... | SSL      | 72     | Continuation Data                                                                    |
| 550 | 12.925054 | 85.14.243... | 192.168.4... | SSL      | 62     | Continuation Data                                                                    |
| 551 | 13.037337 | 85.14.243... | 192.168.4... | TCP      | 1372   | 443 → 49311 [ACK] Seq=9 Ack=19 Win=65792 Len=1318                                    |
| 552 | 13.037410 | 85.14.243... | 192.168.4... | TCP      | 395    | 443 → 49311 [PSH, ACK] Seq=1327 Ack=19 Win=65792 Len=341                             |
| 553 | 13.037471 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49311 [FIN, ACK] Seq=1668 Ack=19 Win=65792 Len=0                               |
| 554 | 13.037485 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=19 Ack=1668 Win=65900 Len=0                                    |
| 555 | 13.037585 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [FIN, ACK] Seq=19 Ack=1668 Win=65900 Len=0                               |
| 556 | 13.037760 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=20 Ack=1669 Win=65900 Len=0                                    |
| 557 | 13.038160 | 192.168.4... | 85.14.243... | TCP      | 66     | 49312 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1                     |
| 558 | 13.297291 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49312 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SACK_PERM=1        |
| 559 | 13.297442 | 192.168.4... | 85.14.243... | TCP      | 60     | 49312 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                                        |
| 560 | 13.623023 | 192.168.4... | 85.14.243... | TCP      | 60     | [TCP Spurious Retransmission] 49311 → 443 [FIN, ACK] Seq=19 Ack=1669 Win=65900 Len=0 |
| 561 | 13.739739 | 85.14.243... | 192.168.4... | TCP      | 54     | [TCP ZeroWindow] 443 → 49311 [ACK] Seq=1669 Ack=20 Win=0 Len=0                       |
| 562 | 14.059827 | 192.168.4... | 85.14.243... | SSL      | 72     | Continuation Data                                                                    |
| 563 | 14.180189 | 85.14.243... | 192.168.4... | SSL      | 62     | Continuation Data                                                                    |
| 564 | 14.180882 | 85.14.243... | 192.168.4... | TCP      | 1372   | 443 → 49312 [ACK] Seq=9 Ack=19 Win=65792 Len=1318                                    |

## 2.7.2.2 Suspicious HTTPS

```
560 13.623023 192.168.4... 85.14.243... TCP 60 [TCP Spurious Retransmission] 49311 → 443 [FIN, ACK] Seq=19 Ack=1669 Win=65900 Len=0
```

**TCP Spurious Retransmission** is when the sending host 'thinks' that the receiving host didn't receive the packet and sends it again.

That is what we're seeing in packet 560.



## 2.7.2.2 Suspicious HTTPS

In packet 561, we see **TCP Zero Window**.

According to the Wireshark Wiki, this is something to investigate. You can read more about this [here](https://wiki.wireshark.org/TCPZeroWindow).

## 2.7.2.2 Suspicious HTTPS

If we right-click that packet, packet 561, and follow the **TCP Stream**, we immediately see something of interest.

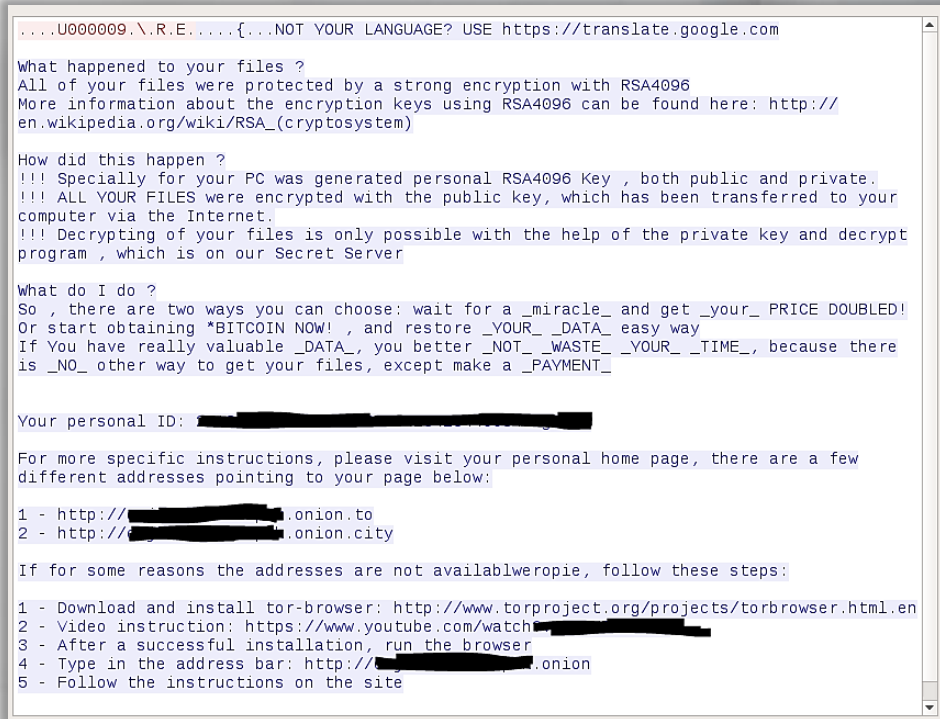
We see cleartext traffic on port 443. More specifically, what we see is what appears to be a ransom note that was displayed to the victim.

## 2.7.2.2 Suspicious HTTPS

Here is a screenshot of the ransom note which was displayed to the victim.

Again, this was sent via port 443, but it's cleartext traffic, not encrypted.

There is also another thing to point out about this traffic that should have been noticeable.

A screenshot of a ransom note displayed in a web browser window. The text is in a monospaced font and contains several sections of information, including instructions on how to decrypt files, a list of payment options, and a list of instructions for downloading and installing Tor Browser. The text is as follows:

```
....U000009.\.R.E....{...NOT YOUR LANGUAGE? USE https://translate.google.com

What happened to your files ?
All of your files were protected by a strong encryption with RSA4096
More information about the encryption keys using RSA4096 can be found here: http://
en.wikipedia.org/wiki/RSA_(cryptosystem)

How did this happen ?
!!! Specially for your PC was generated personal RSA4096 Key , both public and private.
!!! ALL YOUR FILES were encrypted with the public key, which has been transferred to your
computer via the Internet.
!!! Decrypting of your files is only possible with the help of the private key and decrypt
program , which is on our Secret Server

What do I do ?
So , there are two ways you can choose: wait for a _miracle_ and get _your_ PRICE DOUBLED!
Or start obtaining *BITCOIN NOW! , and restore _YOUR_ _DATA_ easy way
If You have really valuable _DATA_, you better _NOT_ _WASTE_ _YOUR_ _TIME_, because there
is _NO_ other way to get your files, except make a _PAYMENT_

Your personal ID: [REDACTED]

For more specific instructions, please visit your personal home page, there are a few
different addresses pointing to your page below:

1 - http://[REDACTED].onion.to
2 - http://[REDACTED].onion.city

If for some reasons the addresses are not available, follow these steps:

1 - Download and install tor-browser: http://www.torproject.org/projects/torbrowser.html.en
2 - Video instruction: https://www.youtube.com/watch/[REDACTED]
3 - After a successful installation, run the browser
4 - Type in the address bar: http://[REDACTED].onion
5 - Follow the instructions on the site
```

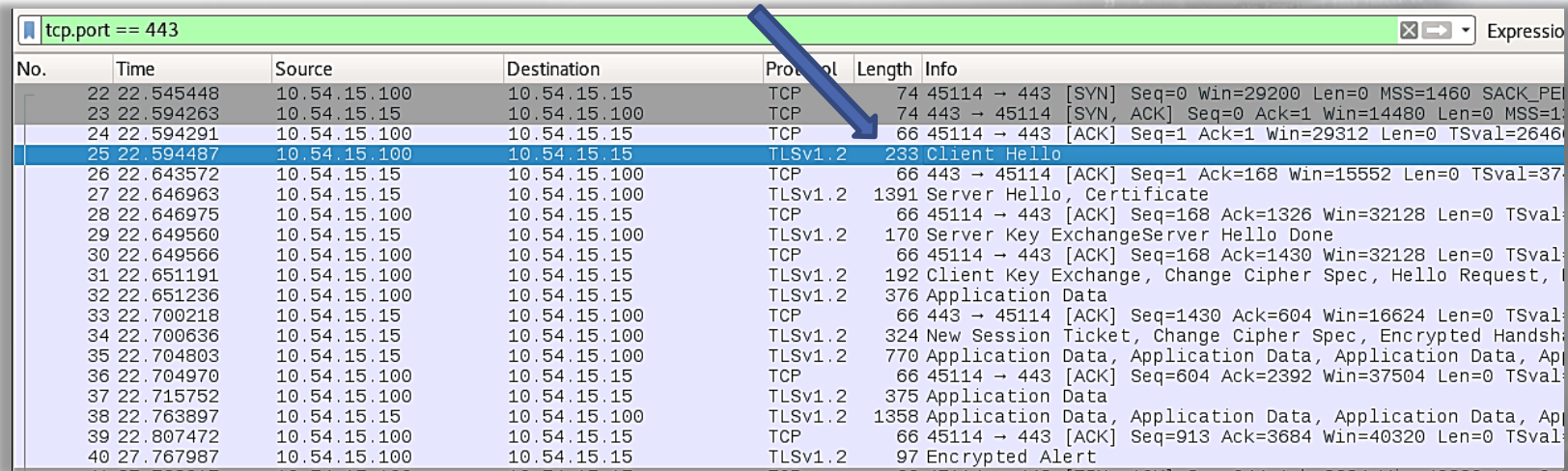
## 2.7.2.2 Suspicious HTTPS

We don't see any packets showing the SSL/TLS Handshake that is required for encrypted communication between both nodes.

| tcp.port == 443 |           |              |              |          |        |                                                                          | Expression... |
|-----------------|-----------|--------------|--------------|----------|--------|--------------------------------------------------------------------------|---------------|
| No.             | Time      | Source       | Destination  | Protocol | Length | Info                                                                     |               |
| 538             | 10.027150 | 192.168.4... | 85.14.243... | TCP      | 66     | 49310 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1         |               |
| 539             | 10.386978 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49310 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SAC... |               |
| 540             | 10.387128 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                            |               |
| 541             | 11.142660 | 192.168.4... | 85.14.243... | SSL      | 106    | Continuation Data                                                        |               |
| 542             | 11.258886 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49310 [FIN, ACK] Seq=1 Ack=53 Win=65792 Len=0                      |               |
| 543             | 11.258995 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [ACK] Seq=53 Ack=2 Win=65900 Len=0                           |               |
| 544             | 11.844658 | 192.168.4... | 85.14.243... | TCP      | 60     | 49310 → 443 [FIN, ACK] Seq=53 Ack=2 Win=65900 Len=0                      |               |
| 545             | 11.845196 | 192.168.4... | 85.14.243... | TCP      | 66     | 49311 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1         |               |
| 546             | 12.051212 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49311 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SAC... |               |
| 547             | 12.051360 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                            |               |
| 548             | 12.158204 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49310 [ACK] Seq=2 Ack=54 Win=65792 Len=0                           |               |
| 549             | 12.811870 | 192.168.4... | 85.14.243... | SSL      | 72     | Continuation Data                                                        |               |
| 550             | 12.925054 | 85.14.243... | 192.168.4... | SSL      | 62     | Continuation Data                                                        |               |
| 551             | 13.037337 | 85.14.243... | 192.168.4... | TCP      | 1372   | 443 → 49311 [ACK] Seq=9 Ack=19 Win=65792 Len=1318                        |               |
| 552             | 13.037410 | 85.14.243... | 192.168.4... | TCP      | 395    | 443 → 49311 [PSH, ACK] Seq=1327 Ack=19 Win=65792 Len=341                 |               |
| 553             | 13.037471 | 85.14.243... | 192.168.4... | TCP      | 54     | 443 → 49311 [FIN, ACK] Seq=1668 Ack=19 Win=65792 Len=0                   |               |
| 554             | 13.037485 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=19 Ack=1668 Win=65900 Len=0                        |               |
| 555             | 13.037585 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [FIN, ACK] Seq=19 Ack=1668 Win=65900 Len=0                   |               |
| 556             | 13.037760 | 192.168.4... | 85.14.243... | TCP      | 60     | 49311 → 443 [ACK] Seq=20 Ack=1669 Win=65900 Len=0                        |               |
| 557             | 13.038160 | 192.168.4... | 85.14.243... | TCP      | 66     | 49312 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1464 WS=4 SACK_PERM=1         |               |
| 558             | 13.297291 | 85.14.243... | 192.168.4... | TCP      | 66     | 443 → 49312 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1318 WS=256 SAC... |               |
| 559             | 13.297442 | 192.168.4... | 85.14.243... | TCP      | 60     | 49312 → 443 [ACK] Seq=1 Ack=1 Win=65900 Len=0                            |               |

## 2.7.2.2 Suspicious HTTPS

This is from a previously shown PCAP and here we're being reminded of how to visually see the SSL/TLS Handshake. The arrow shown below indicates the 1<sup>st</sup> packet in the process.



| No. | Time      | Source       | Destination  | Protocol | Length | Info                                                         |
|-----|-----------|--------------|--------------|----------|--------|--------------------------------------------------------------|
| 22  | 22.545448 | 10.54.15.100 | 10.54.15.15  | TCP      | 74     | 45114 → 443 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 |
| 23  | 22.594263 | 10.54.15.15  | 10.54.15.100 | TCP      | 74     | 443 → 45114 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460  |
| 24  | 22.594291 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=26460    |
| 25  | 22.594487 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 233    | Client Hello                                                 |
| 26  | 22.643572 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45114 [ACK] Seq=1 Ack=168 Win=15552 Len=0 TSval=374    |
| 27  | 22.646963 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 1391   | Server Hello, Certificate                                    |
| 28  | 22.646975 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=168 Ack=1326 Win=32128 Len=0 TSval=    |
| 29  | 22.649560 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 170    | Server Key Exchange, Server Hello Done                       |
| 30  | 22.649566 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=168 Ack=1430 Win=32128 Len=0 TSval=    |
| 31  | 22.651191 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 192    | Client Key Exchange, Change Cipher Spec, Hello Request, I    |
| 32  | 22.651236 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 376    | Application Data                                             |
| 33  | 22.700218 | 10.54.15.15  | 10.54.15.100 | TCP      | 66     | 443 → 45114 [ACK] Seq=1430 Ack=604 Win=16624 Len=0 TSval=    |
| 34  | 22.700636 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 324    | New Session Ticket, Change Cipher Spec, Encrypted Handsh     |
| 35  | 22.704803 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 770    | Application Data, Application Data, Application Data, App    |
| 36  | 22.704970 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=604 Ack=2392 Win=37504 Len=0 TSval=    |
| 37  | 22.715752 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 375    | Application Data                                             |
| 38  | 22.763897 | 10.54.15.15  | 10.54.15.100 | TLSv1.2  | 1358   | Application Data, Application Data, Application Data, App    |
| 39  | 22.807472 | 10.54.15.100 | 10.54.15.15  | TCP      | 66     | 45114 → 443 [ACK] Seq=913 Ack=3684 Win=40320 Len=0 TSval=    |
| 40  | 27.767987 | 10.54.15.100 | 10.54.15.15  | TLSv1.2  | 97     | Encrypted Alert                                              |

## 2.7.2.2 Suspicious HTTPS

One last thing to point out is that the **Secure Sockets Layer** field of the packet shouldn't be empty like how we see it in the packet details below.

- ▶ Frame 541: 106 bytes on wire (848 bits), 106 bytes captured (848 bits)
- ▶ Ethernet II, Src: 50:1f:d0:24:72:5b (50:1f:d0:24:72:5b), Dst: f2:cb:2e:da:80:56 (f2:cb:2e:da:80:56)
- ▶ Internet Protocol Version 4, Src: 192.168.4.196, Dst: 85.14.243.9
- ▶ Transmission Control Protocol, Src Port: 49310, Dst Port: 443, Seq: 1, Ack: 1, Len: 52
- ▶ Secure Sockets Layer

## 2.7.2.2 Suspicious HTTPS

When it comes to HTTP & HTTPS traffic, there are a plethora of attack vectors that we need to look out for because, again, these ports are not monitored, but as time goes on Blue Teamers are beginning to understand the importance of not leaving ports 80, 443, etc. unmonitored.



## 2.7.3 Wireshark Tip

For more information on techniques to filter:

- HTTP traffic: refer to the Wireshark Display Filter Reference for **HTTP** [here](#) and [here](#).
- HTTPS traffic: refer to the Wireshark Display Filter Reference for **HTTPS** [here](#).

<https://www.wireshark.org/docs/dfref/h/http.html>  
<https://www.wireshark.org/docs/dfref/h/http2.html>  
<https://www.wireshark.org/docs/dfref/s/ssl.html>

```
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
1011  
1012  
1013  
1014  
1015  
1016  
1017  
1018  
1019  
1020  
1021  
1022  
1023  
1024  
1025  
1026  
1027  
1028  
1029  
1030  
1031  
1032  
1033  
1034  
1035  
1036  
1037  
1038  
1039  
1040  
1041  
1042  
1043  
1044  
1045  
1046  
1047  
1048  
1049  
1050  
1051  
1052  
1053  
1054  
1055  
1056  
1057  
1058  
1059  
1060  
1061  
1062  
1063  
1064  
1065  
1066  
1067  
1068  
1069  
1070  
1071  
1072  
1073  
1074  
1075  
1076  
1077  
1078  
1079  
1080  
1081  
1082  
1083  
1084  
1085  
1086  
1087  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133  
1134  
1135  
1136  
1137  
1138  
1139  
1140  
1141  
1142  
1143  
1144  
1145  
1146  
1147  
1148  
1149  
1150  
1151  
1152  
1153  
1154  
1155  
1156  
1157  
1158  
1159  
1160  
1161  
1162  
1163  
1164  
1165  
1166  
1167  
1168  
1169  
1170  
1171  
1172  
1173  
1174  
1175  
1176  
1177  
1178  
1179  
1180  
1181  
1182  
1183  
1184  
1185  
1186  
1187  
1188  
1189  
1190  
1191  
1192  
1193  
1194  
1195  
1196  
1197  
1198  
1199  
1200  
1201  
1202  
1203  
1204  
1205  
1206  
1207  
1208  
1209  
1210  
1211  
1212  
1213  
1214  
1215  
1216  
1217  
1218  
1219  
1220  
1221  
1222  
1223  
1224  
1225  
1226  
1227  
1228  
1229  
1230  
1231  
1232  
1233  
1234  
1235  
1236  
1237  
1238  
1239  
1240  
1241  
1242  
1243  
1244  
1245  
1246  
1247  
1248  
1249  
1250  
1251  
1252  
1253  
1254  
1255  
1256  
1257  
1258  
1259  
1260  
1261  
1262  
1263  
1264  
1265  
1266  
1267  
1268  
1269  
1270  
1271  
1272  
1273  
1274  
1275  
1276  
1277  
1278  
1279  
1280  
1281  
1282  
1283  
1284  
1285  
1286  
1287  
1288  
1289  
1290  
1291  
1292  
1293  
1294  
1295  
1296  
1297  
1298  
1299  
1300  
1301  
1302  
1303  
1304  
1305  
1306  
1307  
1308  
1309  
1310  
1311  
1312  
1313  
1314  
1315  
1316  
1317  
1318  
1319  
1320  
1321  
1322  
1323  
1324  
1325  
1326  
1327  
1328  
1329  
1330  
1331  
1332  
1333  
1334  
1335  
1336  
1337  
1338  
1339  
1340  
1341  
1342  
1343  
1344  
1345  
1346  
1347  
1348  
1349  
1350  
1351  
1352  
1353  
1354  
1355  
1356  
1357  
1358  
1359  
1360  
1361  
1362  
1363  
1364  
1365  
1366  
1367  
1368  
1369  
1370  
1371  
1372  
1373  
1374  
1375  
1376  
1377  
1378  
1379  
1380  
1381  
1382  
1383  
1384  
1385  
1386  
1387  
1388  
1389  
1390  
1391  
1392  
1393  
1394  
1395  
1396  
1397  
1398  
1399  
1400  
1401  
1402  
1403  
1404  
1405  
1406  
1407  
1408  
1409  
1410  
1411  
1412  
1413  
1414  
1415  
1416  
1417  
1418  
1419  
1420  
1421  
1422  
1423  
1424  
1425  
1426  
1427  
1428  
1429  
1430  
1431  
1432  
1433  
1434  
1435  
1436  
1437  
1438  
1439  
1440  
1441  
1442  
1443  
1444  
1445  
1446  
1447  
1448  
1449  
1450  
1451  
1452  
1453  
1454  
1455  
1456  
1457  
1458  
1459  
1460  
1461  
1462  
1463  
1464  
1465  
1466  
1467  
1468  
1469  
1470  
1471  
1472  
1473  
1474  
1475  
1476  
1477  
1478  
1479  
1480  
1481  
1482  
1483  
1484  
1485  
1486  
1487  
1488  
1489  
1490  
1491  
1492  
1493  
1494  
1495  
1496  
1497  
1498  
1499  
1500  
1501  
1502  
1503  
1504  
1505  
1506  
1507  
1508  
1509  
1510  
1511  
1512  
1513  
1514  
1515  
1516  
1517  
1518  
1519  
1520  
1521  
1522  
1523  
1524  
1525  
1526  
1527  
1528  
1529  
1530  
1531  
1532  
1533  
1534  
1535  
1536  
1537  
1538  
1539  
1540  
1541  
1542  
1543  
1544  
1545  
1546  
1547  
1548  
1549  
1550  
1551  
1552  
1553  
1554  
1555  
1556  
1557  
1558  
1559  
1560  
1561  
1562  
1563  
1564  
1565  
1566  
1567  
1568  
1569  
1570  
1571  
1572  
1573  
1574  
1575  
1576  
1577  
1578  
1579  
1580  
1581  
1582  
1583  
1584  
1585  
1586  
1587  
1588  
1589  
1590  
1591  
1592  
1593  
1594  
1595  
1596  
1597  
1598  
1599  
1600  
1601  
1602  
1603  
1604  
1605  
1606  
1607  
1608  
1609  
1610  
1611  
1612  
1613  
1614  
1615  
1616  
1617  
1618  
1619  
1620  
1621  
1622  
1623  
1624  
1625  
1626  
1627  
1628  
1629  
1630  
1631  
1632  
1633  
1634  
1635  
1636  
1637  
1638  
1639  
1640  
1641  
1642  
1643  
1644  
1645  
1646  
1647  
1648  
1649  
1650  
1651  
1652  
1653  
1654  
1655  
1656  
1657  
1658  
1659  
1660  
1661  
1662  
1663  
1664  
1665  
1666  
1667  
1668  
1669  
1670  
1671  
1672  
1673  
1674  
1675  
1676  
1677  
1678  
1679  
1680  
1681  
1682  
1683  
1684  
1685  
1686  
1687  
1688  
1689  
1690  
1691  
1692  
1693  
1694  
1695  
1696  
1697  
1698  
1699  
1700  
1701  
1702  
1703  
1704  
1705  
1706  
1707  
1708  
1709  
1710  
1711  
1712  
1713  
1714  
1715  
1716  
1717  
1718  
1719  
1720  
1721  
1722  
1723  
1724  
1725  
1726  
1727  
1728  
1729  
1730  
1731  
1732  
1733  
1734  
1735  
1736  
1737  
1738  
1739  
1740  
1741  
1742  
1743  
1744  
1745  
1746  
1747  
1748  
1749  
1750  
1751  
1752  
1753  
1754  
1755  
1756  
1757  
1758  
1759  
1760  
1761  
1762  
1763  
1764  
1765  
1766  
1767  
1768  
1769  
1770  
1771  
1772  
1773  
1774  
1775  
1776  
1777  
1778  
1779  
1780  
1781  
1782  
1783  
1784  
1785  
1786  
1787  
1788  
1789  
1790  
1791  
1792  
1793  
1794  
1795  
1796  
1797  
1798  
1799  
1800  
1801  
1802  
1803  
1804  
1805  
1806  
1807  
1808  
1809  
1810  
1811  
1812  
1813  
1814  
1815  
1816  
1817  
1818  
1819  
1820  
1821  
1822  
1823  
1824  
1825  
1826  
1827  
1828  
1829  
1830  
1831  
1832  
1833  
1834  
1835  
1836  
1837  
1838  
1839  
1840  
1841  
1842  
1843  
1844  
1845  
1846  
1847  
1848  
1849  
1850  
1851  
1852  
1853  
1854  
1855  
1856  
1857  
1858  
1859  
1860  
1861  
1862  
1863  
1864  
1865  
1866  
1867  
1868  
1869  
1870  
1871  
1872  
1873  
1874  
1875  
1876  
1877  
1878  
1879  
1880  
1881  
1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890  
1891  
1892  
1893  
1894  
1895  
1896  
1897  
1898  
1899  
1900  
1901  
1902  
1903  
1904  
1905  
1906  
1907  
1908  
1909  
1910  
1911  
1912  
1913  
1914  
1915  
1916  
1917  
1918  
1919  
1920  
1921  
1922  
1923  
1924  
1925  
1926  
1927  
1928  
1929  
1930  
1931  
1932  
1933  
1934  
1935  
1936  
1937  
1938  
1939  
1940  
1941  
1942  
1943  
1944  
1945  
1946  
1947  
1948  
1949  
1950  
1951  
1952  
1953  
1954  
1955  
1956  
1957  
1958  
1959  
1960  
1961  
1962  
1963  
1964  
1965  
1966  
1967  
1968  
1969  
1970  
1971  
1972  
1973  
1974  
1975  
1976  
1977  
1978  
1979  
1980  
1981  
1982  
1983  
1984  
1985  
1986  
1987  
1988  
1989  
1990  
1991  
1992  
1993  
1994  
1995  
1996  
1997  
1998  
1999  
2000  
2001  
2002  
2003  
2004  
2005  
2006  
2007  
2008  
2009  
2010  
2011  
2012  
2013  
2014  
2015  
2016  
2017  
2018  
2019  
2020  
2021  
2022  
2023  
2024  
2025  
2026  
2027  
2028  
2029  
2030  
2031  
2032  
2033  
2034  
2035  
2036  
2037  
2038  
2039  
2040  
2041  
2042  
2043  
2044  
2045  
2046  
2047  
2048  
2049  
2050  
2051  
2052  
2053  
2054  
2055  
2056  
2057  
2058  
2059  
2060  
2061  
2062  
2063  
2064  
2065  
2066  
2067  
2068  
2069  
2070  
2071  
2072  
2073  
2074  
2075  
2076  
2077  
2078  
2079  
2080  
2081  
2082  
2083  
2084  
2085  
2086  
2087  
2088  
2089  
2090  
2091  
2092  
2093  
2094  
2095  
2096  
2097  
2098  
2099  
2100  
2101  
2102  
2103  
2104  
2105  
2106  
2107  
2108  
2109  
2110  
2111  
2112  
2113  
2114  
2115  
2116  
2117  
2118  
2119  
2120  
2121  
2122  
2123  
2124  
2125  
2126  
2127  
2128  
2129  
2130  
2131  
2132  
2133  
2134  
2135  
2136  
2137  
2138  
2139  
2140  
2141  
2142  
2143  
2144  
2145  
2146  
2147  
2148  
2149  
2150  
2151  
2152  
2153  
2154  
2155  
2156  
2157  
2158  
2159  
2160  
2161  
2162  
2163  
2164  
2165  
2166  
2167  
2168  
2169  
2170  
2171  
2172  
2173  
2174  
2175  
2176  
2177  
2178  
2179  
2180  
2181  
2182  
2183  
2184  
2185  
2186  
2187  
2188  
2189  
2190  
2191  
2192  
2193  
2194  
2195  
2196  
2197  
2198  
2199  
2200  
2201
```

# Unknown Traffic



## 2.8 Unknown Traffic

As we perform network hunting, we must know that we will not always encounter protocols that we are familiar with. At times you might be faced with a custom or unknown protocol; this is not far fetched.

Referring back to the Trend Labs [article](#) on the CrypMIC ransomware, it was noted that the threat actors used a custom protocol via port 443 to communicate with the C2 servers.

```
24  * @param result the experiment result to return
25  * @param observations an array of Observations, an experiment
26  * @param control the control observation
27
28  def initialize(experiment, observations = [], control = null)
29    @experiment = experiment
30    @observations = observations
31    @control = control
32    @candidates = observations - [control]
33    evaluate_candidates
34
35  freeze
36
37  def context
38    experiment.context
39
40    {
41      experiment_name:
42        experiment.name
43      metadata:
44        metadata
45    }
46  end
47
48  def result
49    result
50  end
51
52  def result
53    result
54  end
55
56  def result
57    result
58  end
59
60  def result
61    result
62  end
63
64  def result
65    result
66  end
67
68  def result
69    result
70  end
71
72  def result
73    result
74  end
75
76  def result
77    result
78  end
79
80  def result
81    result
82  end
83
84  def result
85    result
86  end
87
88  def result
89    result
90  end
91
92  def result
93    result
94  end
95
96  def result
97    result
98  end
99
100 def result
101   result
102 end
```

## 2.8 Unknown Traffic

To give an example of unknown traffic we'll use a PCAP from Forensics Contest called “Anns Bad Aim”, found [here](#), to explain this concept.

We will not go through the entire PCAP or challenge but instead focus on the proprietary protocol, AOL Instant Messenger.

```
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
1011  
1012  
1013  
1014  
1015  
1016  
1017  
1018  
1019  
1020  
1021  
1022  
1023  
1024  
1025  
1026  
1027  
1028  
1029  
1030  
1031  
1032  
1033  
1034  
1035  
1036  
1037  
1038  
1039  
1040  
1041  
1042  
1043  
1044  
1045  
1046  
1047  
1048  
1049  
1050  
1051  
1052  
1053  
1054  
1055  
1056  
1057  
1058  
1059  
1060  
1061  
1062  
1063  
1064  
1065  
1066  
1067  
1068  
1069  
1070  
1071  
1072  
1073  
1074  
1075  
1076  
1077  
1078  
1079  
1080  
1081  
1082  
1083  
1084  
1085  
1086  
1087  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133  
1134  
1135  
1136  
1137  
1138  
1139  
1140  
1141  
1142  
1143  
1144  
1145  
1146  
1147  
1148  
1149  
1150  
1151  
1152  
1153  
1154  
1155  
1156  
1157  
1158  
1159  
1160  
1161  
1162  
1163  
1164  
1165  
1166  
1167  
1168  
1169  
1170  
1171  
1172  
1173  
1174  
1175  
1176  
1177  
1178  
1179  
1180  
1181  
1182  
1183  
1184  
1185  
1186  
1187  
1188  
1189  
1190  
1191  
1192  
1193  
1194  
1195  
1196  
1197  
1198  
1199  
1200  
1201  
1202  
1203  
1204  
1205  
1206  
1207  
1208  
1209  
1210  
1211  
1212  
1213  
1214  
1215  
1216  
1217  
1218  
1219  
1220  
1221  
1222  
1223  
1224  
1225  
1226  
1227  
1228  
1229  
1230  
1231  
1232  
1233  
1234  
1235  
1236  
1237  
1238  
1239  
1240  
1241  
1242  
1243  
1244  
1245  
1246  
1247  
1248  
1249  
1250  
1251  
1252  
1253  
1254  
1255  
1256  
1257  
1258  
1259  
1260  
1261  
1262  
1263  
1264  
1265  
1266  
1267  
1268  
1269  
1270  
1271  
1272  
1273  
1274  
1275  
1276  
1277  
1278  
1279  
1280  
1281  
1282  
1283  
1284  
1285  
1286  
1287  
1288  
1289  
1290  
1291  
1292  
1293  
1294  
1295  
1296  
1297  
1298  
1299  
1300  
1301  
1302  
1303  
1304  
1305  
1306  
1307  
1308  
1309  
1310  
1311  
1312  
1313  
1314  
1315  
1316  
1317  
1318  
1319  
1320  
1321  
1322  
1323  
1324  
1325  
1326  
1327  
1328  
1329  
1330  
1331  
1332  
1333  
1334  
1335  
1336  
1337  
1338  
1339  
1340  
1341  
1342  
1343  
1344  
1345  
1346  
1347  
1348  
1349  
1350  
1351  
1352  
1353  
1354  
1355  
1356  
1357  
1358  
1359  
1360  
1361  
1362  
1363  
1364  
1365  
1366  
1367  
1368  
1369  
1370  
1371  
1372  
1373  
1374  
1375  
1376  
1377  
1378  
1379  
1380  
1381  
1382  
1383  
1384  
1385  
1386  
1387  
1388  
1389  
1390  
1391  
1392  
1393  
1394  
1395  
1396  
1397  
1398  
1399  
1400  
1401  
1402  
1403  
1404  
1405  
1406  
1407  
1408  
1409  
1410  
1411  
1412  
1413  
1414  
1415  
1416  
1417  
1418  
1419  
1420  
1421  
1422  
1423  
1424  
1425  
1426  
1427  
1428  
1429  
1430  
1431  
1432  
1433  
1434  
1435  
1436  
1437  
1438  
1439  
1440  
1441  
1442  
1443  
1444  
1445  
1446  
1447  
1448  
1449  
1450  
1451  
1452  
1453  
1454  
1455  
1456  
1457  
1458  
1459  
1460  
1461  
1462  
1463  
1464  
1465  
1466  
1467  
1468  
1469  
1470  
1471  
1472  
1473  
1474  
1475  
1476  
1477  
1478  
1479  
1480  
1481  
1482  
1483  
1484  
1485  
1486  
1487  
1488  
1489  
1490  
1491  
1492  
1493  
1494  
1495  
1496  
1497  
1498  
1499  
1500  
1501  
1502  
1503  
1504  
1505  
1506  
1507  
1508  
1509  
1510  
1511  
1512  
1513  
1514  
1515  
1516  
1517  
1518  
1519  
1520  
1521  
1522  
1523  
1524  
1525  
1526  
1527  
1528  
1529  
1530  
1531  
1532  
1533  
1534  
1535  
1536  
1537  
1538  
1539  
1540  
1541  
1542  
1543  
1544  
1545  
1546  
1547  
1548  
1549  
1550  
1551  
1552  
1553  
1554  
1555  
1556  
1557  
1558  
1559  
1560  
1561  
1562  
1563  
1564  
1565  
1566  
1567  
1568  
1569  
1570  
1571  
1572  
1573  
1574  
1575  
1576  
1577  
1578  
1579  
1580  
1581  
1582  
1583  
1584  
1585  
1586  
1587  
1588  
1589  
1590  
1591  
1592  
1593  
1594  
1595  
1596  
1597  
1598  
1599  
1600  
1601  
1602  
1603  
1604  
1605  
1606  
1607  
1608  
1609  
1610  
1611  
1612  
1613  
1614  
1615  
1616  
1617  
1618  
1619  
1620  
1621  
1622  
1623  
1624  
1625  
1626  
1627  
1628  
1629  
1630  
1631  
1632  
1633  
1634  
1635  
1636  
1637  
1638  
1639  
1640  
1641  
1642  
1643  
1644  
1645  
1646  
1647  
1648  
1649  
1650  
1651  
1652  
1653  
1654  
1655  
1656  
1657  
1658  
1659  
1660  
1661  
1662  
1663  
1664  
1665  
1666  
1667  
1668  
1669  
1670  
1671  
1672  
1673  
1674  
1675  
1676  
1677  
1678  
1679  
1680  
1681  
1682  
1683  
1684  
1685  
1686  
1687  
1688  
1689  
1690  
1691  
1692  
1693  
1694  
1695  
1696  
1697  
1698  
1699  
1700  
1701  
1702  
1703  
1704  
1705  
1706  
1707  
1708  
1709  
1710  
1711  
1712  
1713  
1714  
1715  
1716  
1717  
1718  
1719  
1720  
1721  
1722  
1723  
1724  
1725  
1726  
1727  
1728  
1729  
1730  
1731  
1732  
1733  
1734  
1735  
1736  
1737  
1738  
1739  
1740  
1741  
1742  
1743  
1744  
1745  
1746  
1747  
1748  
1749  
1750  
1751  
1752  
1753  
1754  
1755  
1756  
1757  
1758  
1759  
1760  
1761  
1762  
1763  
1764  
1765  
1766  
1767  
1768  
1769  
1770  
1771  
1772  
1773  
1774  
1775  
1776  
1777  
1778  
1779  
1780  
1781  
1782  
1783  
1784  
1785  
1786  
1787  
1788  
1789  
1790  
1791  
1792  
1793  
1794  
1795  
1796  
1797  
1798  
1799  
1800  
1801  
1802  
1803  
1804  
1805  
1806  
1807  
1808  
1809  
1810  
1811  
1812  
1813  
1814  
1815  
1816  
1817  
1818  
1819  
1820  
1821  
1822  
1823  
1824  
1825  
1826  
1827  
1828  
1829  
1830  
1831  
1832  
1833  
1834  
1835  
1836  
1837  
1838  
1839  
1840  
1841  
1842  
1843  
1844  
1845  
1846  
1847  
1848  
1849  
1850  
1851  
1852  
1853  
1854  
1855  
1856  
1857  
1858  
1859  
1860  
1861  
1862  
1863  
1864  
1865  
1866  
1867  
1868  
1869  
1870  
1871  
1872  
1873  
1874  
1875  
1876  
1877  
1878  
1879  
1880  
1881  
1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890  
1891  
1892  
1893  
1894  
1895  
1896  
1897  
1898  
1899  
1900  
1901  
1902  
1903  
1904  
1905  
1906  
1907  
1908  
1909  
1910  
1911  
1912  
1913  
1914  
1915  
1916  
1917  
1918  
1919  
1920  
1921  
1922  
1923  
1924  
1925  
1926  
1927  
1928  
1929  
1930  
1931  
1932  
1933  
1934  
1935  
1936  
1937  
1938  
1939  
1940  
1941  
1942  
1943  
1944  
1945  
1946  
1947  
1948  
1949  
1950  
1951  
1952  
1953  
1954  
1955  
1956  
1957  
1958  
1959  
1960  
1961  
1962  
1963  
1964  
1965  
1966  
1967  
1968  
1969  
1970  
1971  
1972  
1973  
1974  
1975  
1976  
1977  
1978  
1979  
1980  
1981  
1982  
1983  
1984  
1985  
1986  
1987  
1988  
1989  
1990  
1991  
1992  
1993  
1994  
1995  
1996  
1997  
1998  
1999  
2000  
2001  
2002  
2003  
2004  
2005  
2006  
2007  
2008  
2009  
2010  
2011  
2012  
2013  
2014  
2015  
2016  
2017  
2018  
2019  
2020  
2021  
2022  
2023  
2024  
2025  
2026  
2027  
2028  
2029  
2030  
2031  
2032  
2033  
2034  
2035  
2036  
2037  
2038  
2039  
2040  
2041  
2042  
2043  
2044  
2045  
2046  
2047  
2048  
2049  
2050  
2051  
2052  
2053  
2054  
2055  
2056  
2057  
2058  
2059  
2060  
2061  
2062  
2063  
2064  
2065  
2066  
2067  
2068  
2069  
2070  
2071  
2072  
2073  
2074  
2075  
2076  
2077  
2078  
2079  
2080  
2081  
2082  
2083  
2084  
2085  
2086  
2087  
2088  
2089  
2090  
2091  
2092  
2093  
2094  
2095  
2096  
2097  
2098  
2099  
2100  
2101  
2102  
2103  
2104  
2105  
2106  
2107  
2108  
2109  
2110  
2111  
2112  
2113  
2114  
2115  
2116  
2117  
2118  
2119  
2120  
2121  
2122  
2123  
2124  
2125  
2126  
2127  
2128  
2129  
2130  
2131  
2132  
2133  
2134  
2135  
2136  
2137  
2138  
2139  
2140  
2141  
2142  
2143  
2144  
2145  
2146  
2147  
2148  
2149  
2150  
2151  
2152  
2153  
2154  
2155  
2156  
2157  
2158  
2159  
2160  
2161  
2162  
2163  
2164  
2165  
2166  
2167  
2168  
2169  
2170  
2171  
2172  
2173  
2174  
2175  
2176  
2177  
2178  
2179  
2180  
2181  
2182  
2183  
2184  
2185  
2186  
2187  
2188  
2189  
2190  
2191  
2192  
2193  
2194  
2195  
2196  
2197  
2198  
2199  
2200  
2201  
2202  
2203  
2204  
2205  
2206  
2207  
2208  
2209  
2210  
2211  
2212  
2213  
2214  
2215  
2216  
2217  
2218  
2219  
2220  
2221  
2222  
2223  
2224  
2225  
2226  
2227  
2228  
2229  
2230  
2231
```

## 2.8 Unknown Traffic

A filter was used to focus simply on the traffic on port 443. Remember that traffic on this port should be encrypted.

In the following screenshot, we'll see that the traffic is not encrypted and the Secure Sockets Layer field in the packet is empty, there are no details to show to us because is not really SSL traffic based on the protocol.



## 2.8 Unknown Traffic

| No. | Time      | Source        | Destination   | Protocol | Length | Info                                              |
|-----|-----------|---------------|---------------|----------|--------|---------------------------------------------------|
| 1   | 0.000000  | 192.168.1.158 | 64.12.24.50   | SSL      | 60     | Continuation Data                                 |
| 2   | 0.000579  | 64.12.24.50   | 192.168.1.158 | TCP      | 60     | 443 → 51128 [ACK] Seq=1 Ack=7 Win=64240 Len=0     |
| 3   | 15.044068 | 192.168.1.158 | 64.12.24.50   | SSL      | 243    | Continuation Data                                 |
| 4   | 15.044588 | 64.12.24.50   | 192.168.1.158 | TCP      | 60     | 443 → 51128 [ACK] Seq=1 Ack=196 Win=64240 Len=0   |
| 5   | 15.135701 | 192.168.1.158 | 64.12.24.50   | SSL      | 94     | Continuation Data                                 |
| 6   | 15.135706 | 64.12.24.50   | 192.168.1.158 | TCP      | 60     | 443 → 51128 [ACK] Seq=1 Ack=236 Win=64240 Len=0   |
| 7   | 15.152349 | 64.12.24.50   | 192.168.1.158 | SSL      | 263    | Continuation Data                                 |
| 8   | 15.154639 | 64.12.24.50   | 192.168.1.158 | SSL      | 92     | Continuation Data                                 |
| 9   | 15.155906 | 192.168.1.158 | 64.12.24.50   | TCP      | 60     | 51128 → 443 [ACK] Seq=236 Ack=210 Win=62780 Len=0 |
| 10  | 15.155911 | 192.168.1.158 | 64.12.24.50   | TCP      | 60     | 51128 → 443 [ACK] Seq=236 Ack=248 Win=62742 Len=0 |
| 11  | 39.587870 | 192.168.1.158 | 64.12.24.50   | SSL      | 182    | Continuation Data                                 |
| 12  | 39.590958 | 64.12.24.50   | 192.168.1.158 | TCP      | 60     | 443 → 51128 [ACK] Seq=248 Ack=364 Win=64240 Len=0 |
| 13  | 39.697807 | 64.12.24.50   | 192.168.1.158 | SSL      | 263    | Continuation Data                                 |
| 14  | 39.698818 | 192.168.1.158 | 64.12.24.50   | TCP      | 60     | 51128 → 443 [ACK] Seq=364 Ack=457 Win=62742 Len=0 |
| 15  | 39.700370 | 64.12.24.50   | 192.168.1.158 | SSL      | 92     | Continuation Data                                 |
| 16  | 39.702540 | 192.168.1.158 | 64.12.24.50   | TCP      | 60     | 51128 → 443 [ACK] Seq=364 Ack=405 Win=62742 Len=0 |

▶ Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)  
▶ Ethernet II, Src: HewlettP\_45:a4:bb (00:12:79:45:a4:bb), Dst: Vmware\_b0:8d:62 (00:0c:29:b0:8d:62)  
▶ Internet Protocol Version 4, Src: 192.168.1.158, Dst: 64.12.24.50  
▶ Transmission Control Protocol, Src Port: 51128, Dst Port: 443, Seq: 1, Ack: 1, Len: 6  
Secure Sockets Layer

## 2.8 Unknown Traffic

This PCAP contains evidence of a file transfer.

At the beginning of the TCP Stream, you will see **OFT2**.  
OFT2 is AOL instant messaging protocol, OSCAR (**O**pen  
**S**ystem for **C**ommunic**A**tion in **R**ealtime).



## 2.8 Unknown Traffic

Within TCP Stream 5 snippet below, we can see **OFT2**.



```
OFT2.....d.....Cool
FileXfer.....recipe.docx.....
.....OFT2....7174647.....d.....Cool FileXfer.....
.....recipe.docx.....
```

## 2.8 Unknown Traffic

You can read about the details of OSCAR within a Google Summer of Code project titled [On Sending Files via OSCAR](http://www.cs.cmu.edu/~jhclark/aim/On%20Sending%20Files%20via%20OSCAR.pdf).

## 2.8 Unknown Traffic

Knowing what specific protocol is associated with this traffic, we can decode the traffic using Wireshark's **Decode As** feature.

According to the Wireshark [documentation](https://www.wireshark.org/docs/wsug_html_chunked/ChCustProtocolDissectionSection.html), the “Decode As” functionality lets you temporarily divert specific protocol dissections. Each protocol has its own dissector, so dissecting a complete packet will typically involve several dissectors.

## 2.8 Unknown Traffic

Within **Analyze > Enable Protocols**, you should see several listed related to AIM.

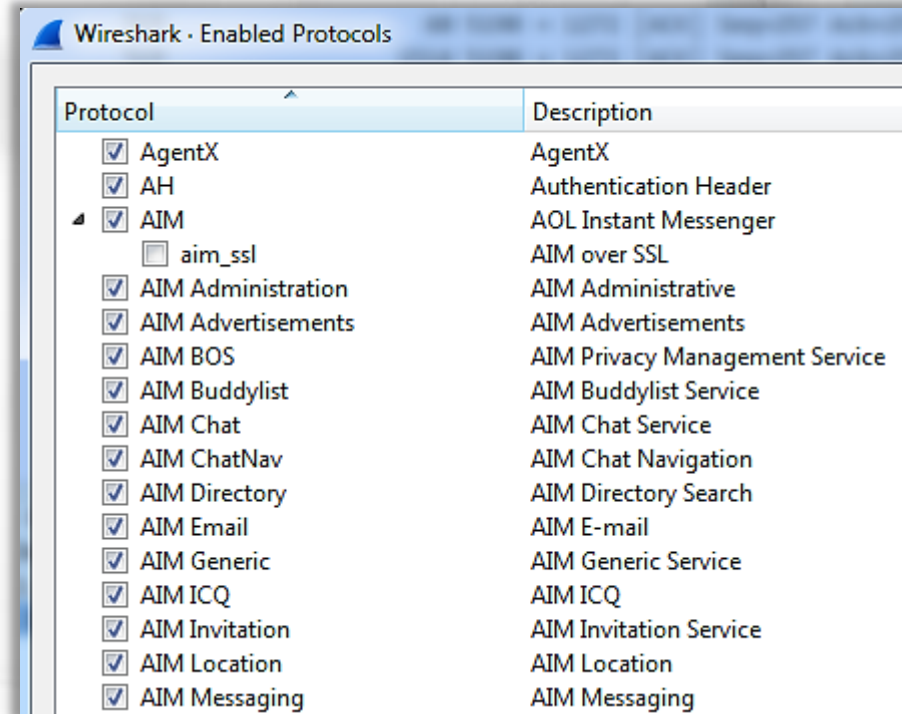
You can read more about the AOL Instant Messenger (AIM) dissector [here](https://wiki.wireshark.org/AIM).

```
24 * @param result - the Experiment data result to return
25 * @param observations - an array of Observations, or nil
26 * @param control - the control Observation
27
28 def initialize(experiment, observations = [], control = nil)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   freeze
36
37   @context =
38     experiment.context
39   end
40
41   # TODO: the name of the experiment
42   def experiment_name
43     experiment.name
44   end
45
46   # TODO: the result a match between all
47   def matched?
48     ...
49   end
50
51   @experiment/result.rb 1.1
```

## 2.8 Unknown Traffic

This is a screenshot to show the enabled protocols within our installation of Wireshark.

Your settings might be different from ours.



## 2.8 Unknown Traffic

Before showing an example of using the Decode As feature within Wireshark, let's take a look at the packets before decoding them with the AIM dissector.



## 2.8 Unknown Traffic

Before

| ssl |           |               |               |          |        |                   |
|-----|-----------|---------------|---------------|----------|--------|-------------------|
| No. | Time      | Source        | Destination   | Protocol | Length | Info              |
| 135 | 61.286572 | 64.12.25.91   | 192.168.1.159 | TLSv1    | 284    | Application Data  |
| 136 | 61.287474 | 64.12.24.50   | 192.168.1.158 | SSL      | 183    | Continuation Data |
| 159 | 67.283887 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 115    | Application Data  |
| 161 | 67.395187 | 64.12.24.50   | 192.168.1.158 | SSL      | 94     | Continuation Data |
| 164 | 69.467940 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 236    | Application Data  |
| 166 | 69.578661 | 64.12.25.91   | 192.168.1.159 | TLSv1    | 284    | Application Data  |
| 167 | 69.578667 | 64.12.24.50   | 192.168.1.158 | SSL      | 280    | Continuation Data |
| 177 | 77.713532 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 115    | Application Data  |
| 179 | 77.834511 | 64.12.24.50   | 192.168.1.158 | SSL      | 94     | Continuation Data |
| 181 | 83.374949 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 254    | Application Data  |
| 183 | 83.487629 | 64.12.25.91   | 192.168.1.159 | TLSv1    | 284    | Application Data  |
| 184 | 83.489593 | 64.12.24.50   | 192.168.1.158 | SSL      | 298    | Continuation Data |
| 188 | 83.793749 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 115    | Application Data  |
| 190 | 83.906524 | 64.12.24.50   | 192.168.1.158 | SSL      | 94     | Continuation Data |
| 194 | 84.883656 | 192.168.1.159 | 64.12.25.91   | TLSv1    | 115    | Application Data  |
| 196 | 84.995484 | 64.12.24.50   | 192.168.1.158 | SSL      | 94     | Continuation Data |
| 200 | 87.726166 | 192.168.1.158 | 64.12.24.50   | SSL      | 94     | Continuation Data |
| 202 | 87.832875 | 64.12.25.91   | 192.168.1.159 | TLSv1    | 115    | Application Data  |
| 210 | 90.788876 | 192.168.1.158 | 64.12.24.50   | SSL      | 64     | Continuation Data |

## 2.8 Unknown Traffic

On one of the packets, we need to right-click and select Decode As.

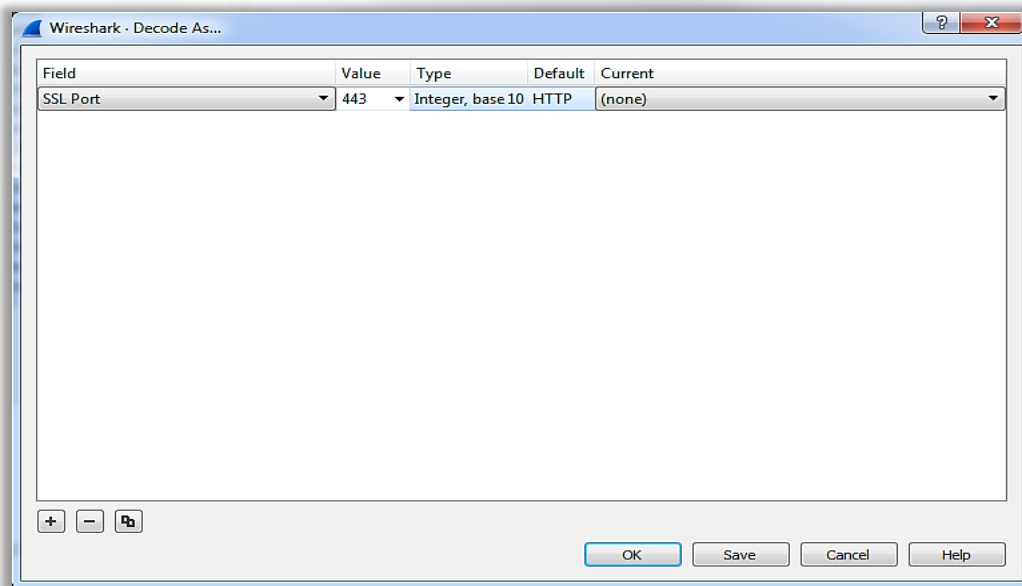
A new window will appear titled “**Wireshark Decode As...**”.

The first line will be prepopulated with information it acquired from the packet that was right-clicked on.



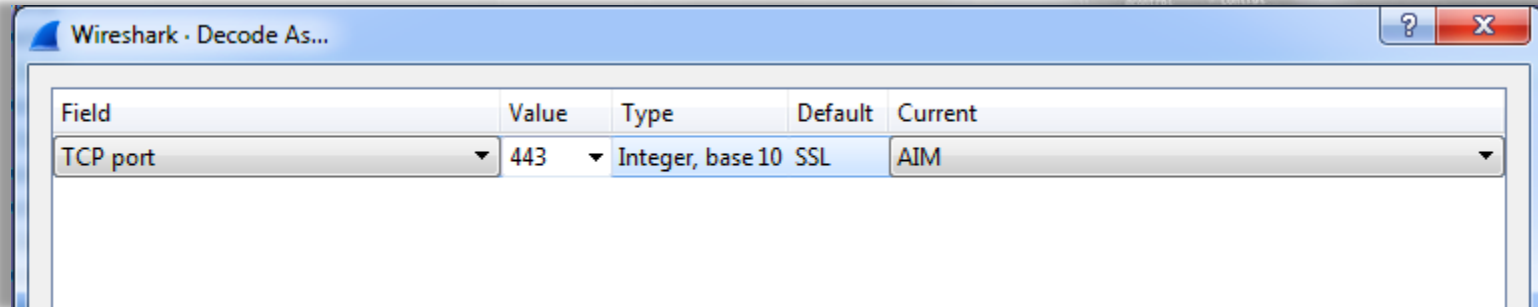
## 2.8 Unknown Traffic

Before



## 2.8 Unknown Traffic

To properly use the AIM dissector with this PCAP, we need to set the fields as such.



## 2.8 Unknown Traffic

After



| No. | Time      | Source        | Destination   | Protocol      | Length | Info                                                                                  |
|-----|-----------|---------------|---------------|---------------|--------|---------------------------------------------------------------------------------------|
| 51  | 34.761557 | 205.188.13.12 | 192.168.1.159 | AIM Generic   | 831    | AIM Generic, Rate Info                                                                |
| 52  | 34.763039 | 192.168.1.159 | 205.188.13.12 | AIM Generic   | 80     | AIM Generic, Rate Info Ack                                                            |
| 54  | 34.768974 | 192.168.1.159 | 205.188.13.12 | AIM Generic   | 86     | AIM Generic, Client Ready                                                             |
| 56  | 34.770551 | 192.168.1.159 | 205.188.13.12 | AIM SST       | 81     | SNAC data, AIM SST, Subtype: 0x0006                                                   |
| 83  | 35.313502 | 205.188.13.12 | 192.168.1.159 | AIM SST       | 1395   | SNAC data, AIM SST, Subtype: 0x0007                                                   |
| 92  | 58.458768 | 192.168.1.158 | 64.12.24.50   | AIM Messaging | 182    | AIM Messaging, Outgoing to: Sec558user1                                               |
| 94  | 58.568705 | 64.12.24.50   | 192.168.1.158 | AIM Generic   | 263    | AIM Generic, Rate Change                                                              |
| 97  | 58.571268 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 92     | AIM Messaging, Acknowledge                                                            |
| 136 | 61.287474 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 183    | AIM Messaging, Incoming                                                               |
| 161 | 67.395187 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 94     | AIM Messaging, Mini Typing Notifications (MTN)                                        |
| 167 | 69.578667 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 280    | AIM Messaging, Incoming                                                               |
| 179 | 77.834511 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 94     | AIM Messaging, Mini Typing Notifications (MTN)                                        |
| 184 | 83.489593 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 298    | AIM Messaging, Incoming                                                               |
| 190 | 83.906524 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 94     | AIM Messaging, Mini Typing Notifications (MTN)                                        |
| 196 | 84.995484 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 94     | AIM Messaging, Mini Typing Notifications (MTN)                                        |
| 200 | 87.726166 | 192.168.1.158 | 64.12.24.50   | AIM Messaging | 94     | AIM Messaging, Mini Typing Notifications (MTN)                                        |
| 212 | 90.816866 | 192.168.1.158 | 64.12.24.50   | AIM Messaging | 164    | AIM Messaging, Outgoing to: Sec558user1AIM Messaging, Mini Typing Notifications (MTN) |
| 214 | 91.003314 | 64.12.24.50   | 192.168.1.158 | AIM Generic   | 263    | AIM Generic, Rate Change                                                              |
| 216 | 91.004650 | 64.12.24.50   | 192.168.1.158 | AIM Messaging | 92     | AIM Messaging, Acknowledge                                                            |

## 2.8 Unknown Traffic

Here we can see packet 167 after using the AIM dissector.

We now can see specific information regarding the protocol that was used on port 443, which was OSCAR (OFT2).

```
▶ Frame 167: 280 bytes on wire (2240 bits), 280 bytes captured (2240 bits)
▶ Ethernet II, Src: Vmware_b0:8d:62 (00:0c:29:b0:8d:62), Dst: HewlettP_45:a4:bb (00:12:79:45:a4:bb)
▶ Internet Protocol Version 4, Src: 64.12.24.50, Dst: 192.168.1.158
▶ Transmission Control Protocol, Src Port: 443, Dst Port: 51128, Seq: 664, Ack: 364, Len: 226
  ▲ AOL Instant Messenger
    Command Start: 0x2a
    Channel ID: SNAC Data (0x02)
    Sequence Number: 22234
    Data Field Length: 220
    ▶ FNAC: Family: AIM Messaging (0x0004), Subtype: Incoming (0x0007)
  ▲ AIM Messaging, Incoming
    ICBM Cookie: d41b4ece87df1f77
    Message Channel ID: 0x0001
    ▶ Buddy: Sec558user1
    Warning Level: 0
    TLV Count: 5
    ▶ TLV: User class
    ▶ TLV: Member since
    ▶ TLV: BART Info
    ▶ TLV: Session Length (sec)
    ▶ TLV: Online since
    ▶ TLV: Message Block
    ▶ TLV: Unknown
    ▶ TLV: Non-direct connect typing notification
    ▶ TLV: Unknown
```

## 2.8 Unknown Traffic

This is packet 167 prior to decoding the traffic with the AIM dissector.

```
▶ Frame 167: 280 bytes on wire (2240 bits), 280 bytes captured (2240 bits)
▶ Ethernet II, Src: Vmware_b0:8d:62 (00:0c:29:b0:8d:62), Dst: HewlettP_45:a4:bb (00:12:79:45:a4:bb)
▶ Internet Protocol Version 4, Src: 64.12.24.50, Dst: 192.168.1.158
▶ Transmission Control Protocol, Src Port: 443, Dst Port: 51128, Seq: 664, Ack: 364, Len: 226
  Secure Sockets Layer
```

## 2.8 Unknown Traffic

Wireshark has many dissectors which we can use. We can also add dissectors to Wireshark.

You can read more about that [here](https://www.wireshark.org/docs/wsdg_html_chunked/ChDissectAdd.html). You can also find dissectors on the web.

```
23 def initialize(experiment, observations = [], control = null)
24   @experiment = experiment
25   @observations = observations
26   @control = control
27   @candidates = observations - @control
28   evaluate_candidates
29
30   freeze
31
32   @context =
33     experiment.context
34   end
35
36   # Fetch the name of the experiment
37   def experiment_name
38     experiment.name
39   end
40
41   # Fetch the result a match between all
42   def matched?
43     @candidates == @control
44   end
45
46   @candidates/result.rb 1.1
```

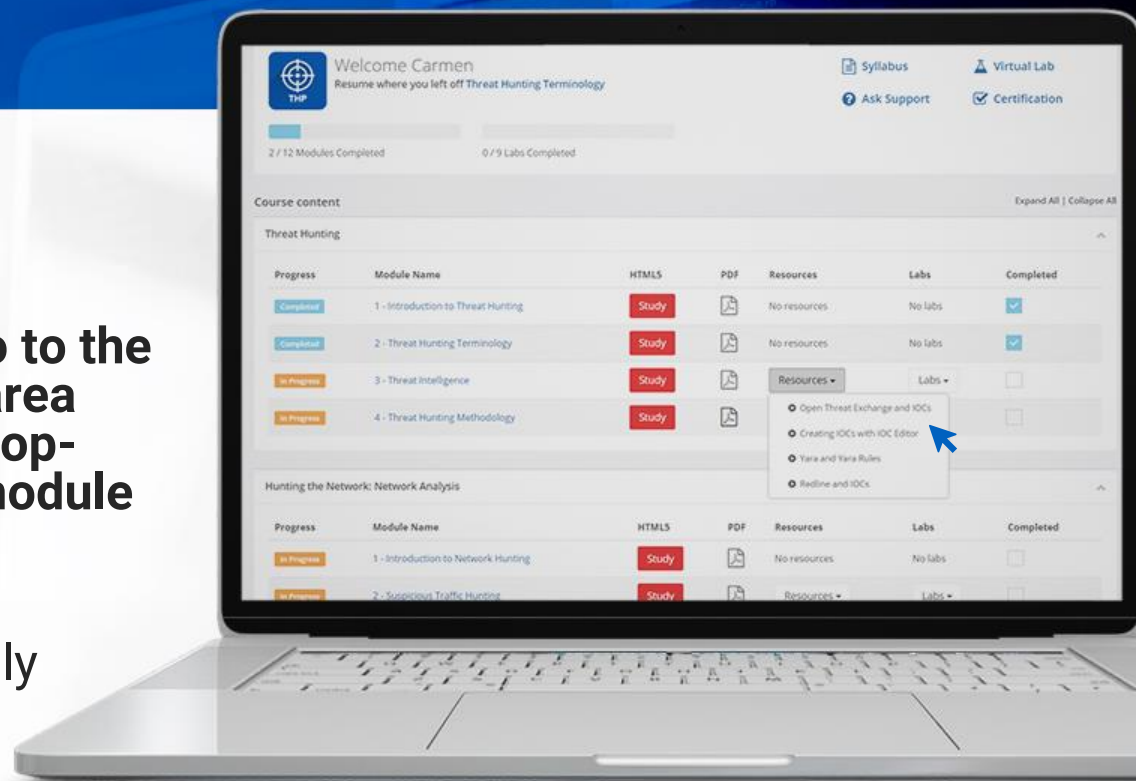


## 2.8.1 Video #1

Check out the video on **Wireshark!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course.  
To upgrade, click **[LINK](#)**.



# More Hunting Tools



## 2.9 More Hunting Tools

We'll look at a two more tools that will assist us in our packet hunting engagements. The first tool will be NetworkMiner.

The following description is from the [Netresec](http://www.netresec.com/?page=NetworkMiner) website.

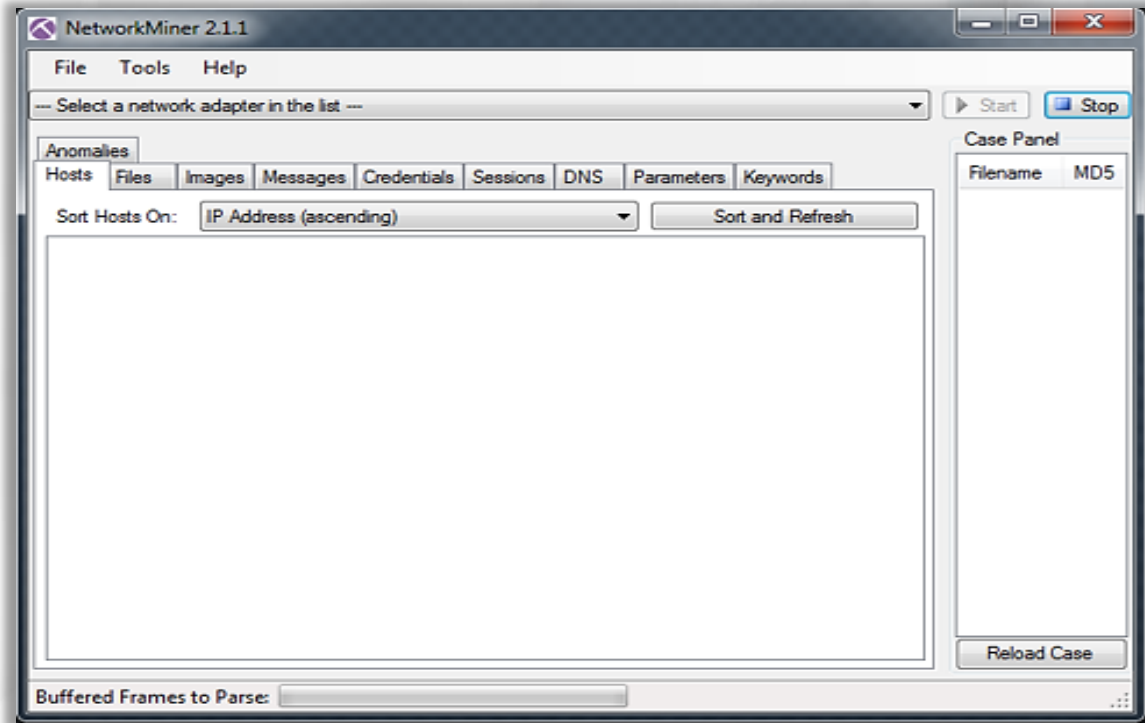
## 2.9.1 NetworkMiner

“NetworkMiner is a Network Forensic Analysis Tool for Windows (but also works in Linux / Mac OS X / FreeBSD). NetworkMiner can be used as a passive network sniffer/packet capturing tool in order to detect operating systems, sessions, hostnames, open ports etc. without putting any traffic on the network. NetworkMiner can also parse PCAP files for off-line analysis and to regenerate/reassemble transmitted files and certificates from PCAP files.”

## 2.9.1 NetworkMiner

This is a screenshot of the NetworkMiner interface.

We will look at NetworkMiner closer within the Videos section and you'll be using it within the Labs section.

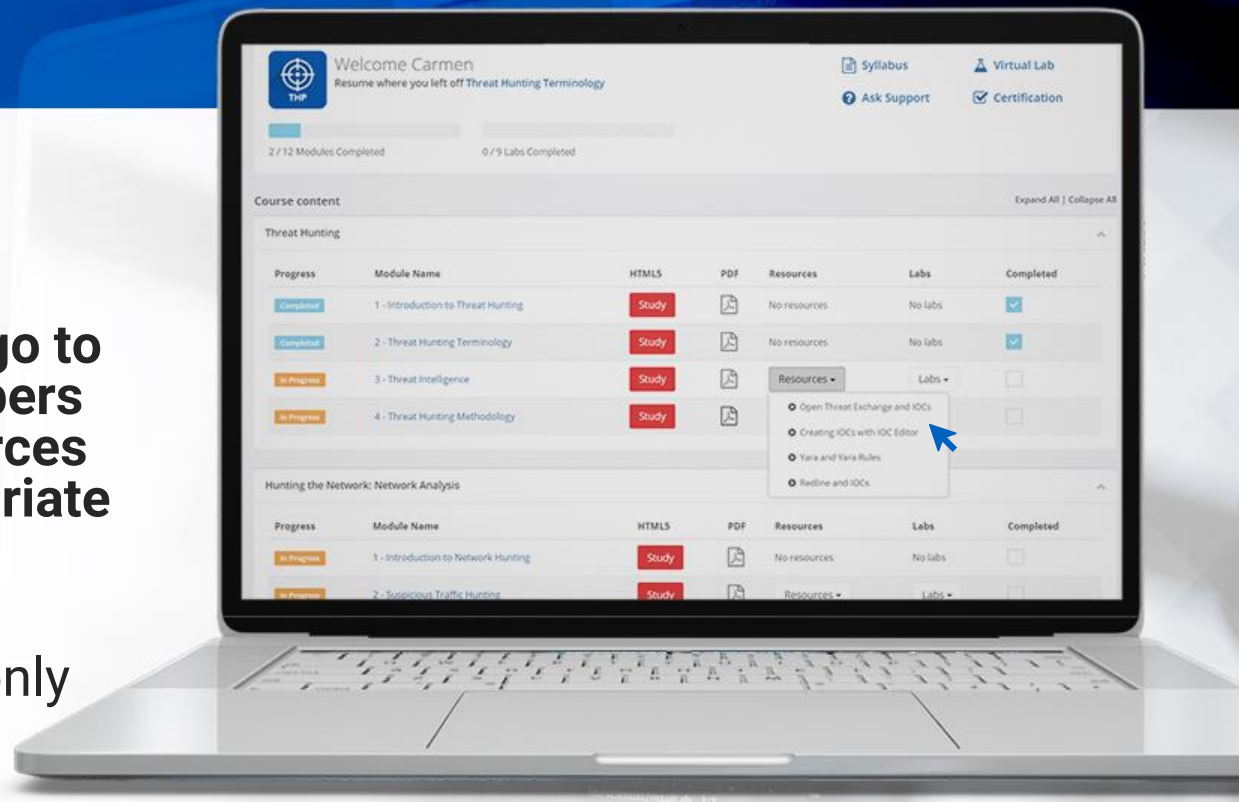


## 2.9.1.1 Video #2

Check out the video on **Network Miner!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).



## 2.9.2 RSA NetWitness Investigator

The last tool we'll look at is [RSA NetWitness Investigator](#).

The freeware version of the tool can be downloaded [here](#).  
The following is an official description of the tool.

## 2.9.2 RSA NetWitness Investigator

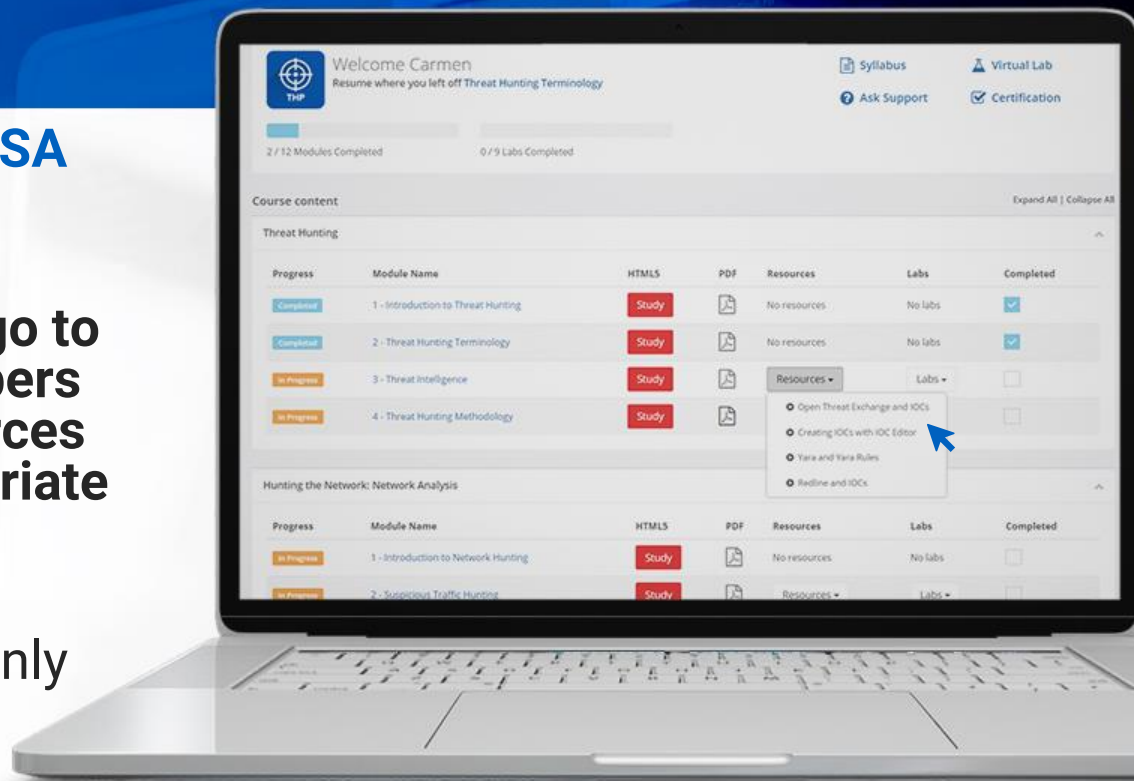
“Investigator is the award-winning interactive threat analysis application of the RSA NetWitness product suite. Investigator provides security operations staff, auditors, and fraud and forensics investigators the power to perform free-form contextual analysis of raw network data. Investigator collects network traffic live or accepts imported .pcap files providing unprecedented detail, interactive navigation, and dynamic analysis across all network layers.”

## 2.9.2.1 Video #3

Check out the video on **RSA Witness Investigator!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course.  
To upgrade, click **[LINK](#)**.

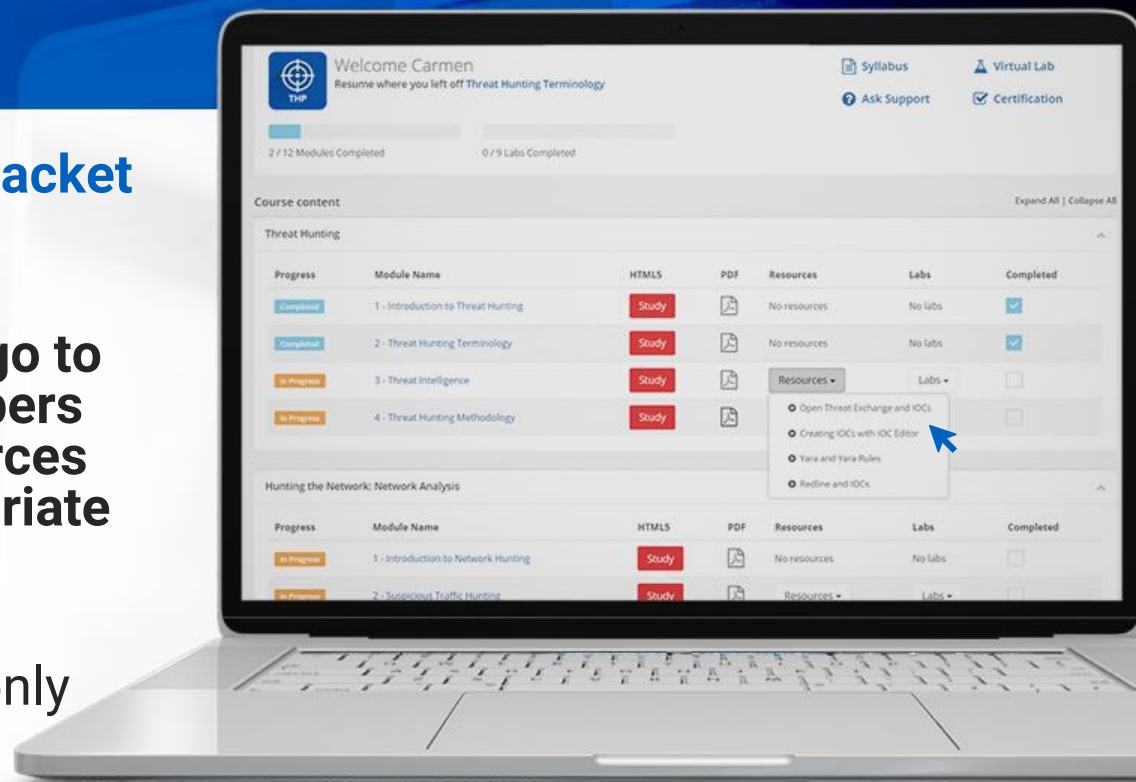


## 2.9.3 Video #4

Check out the video on **Packet Hunting!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click **[LINK](#)**.

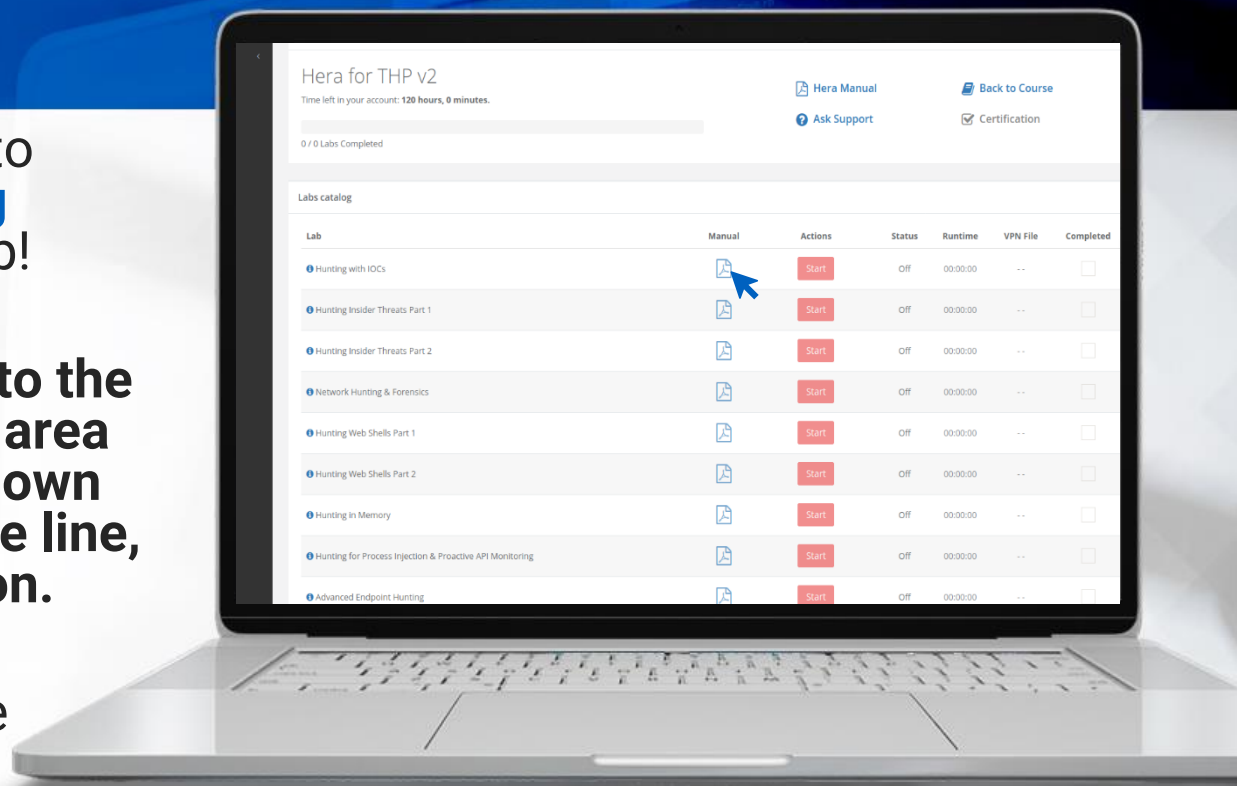


## 2.9.4 Lab #1

Put what you've learned to practice with the **Hunting Insider Threats Part 1** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).



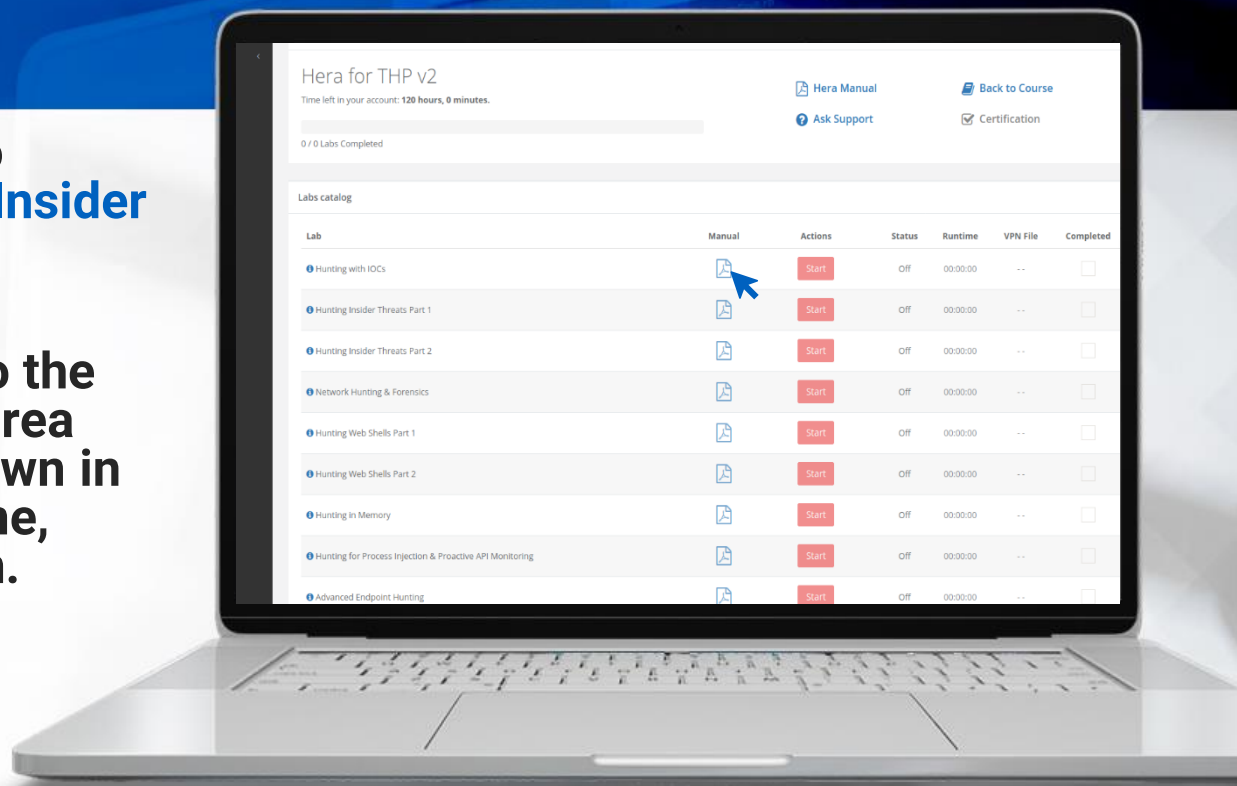
**\*NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

## 2.9.5 Lab #2

Put what you've learned to practice with the **Hunting Insider Threats Part 2** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).



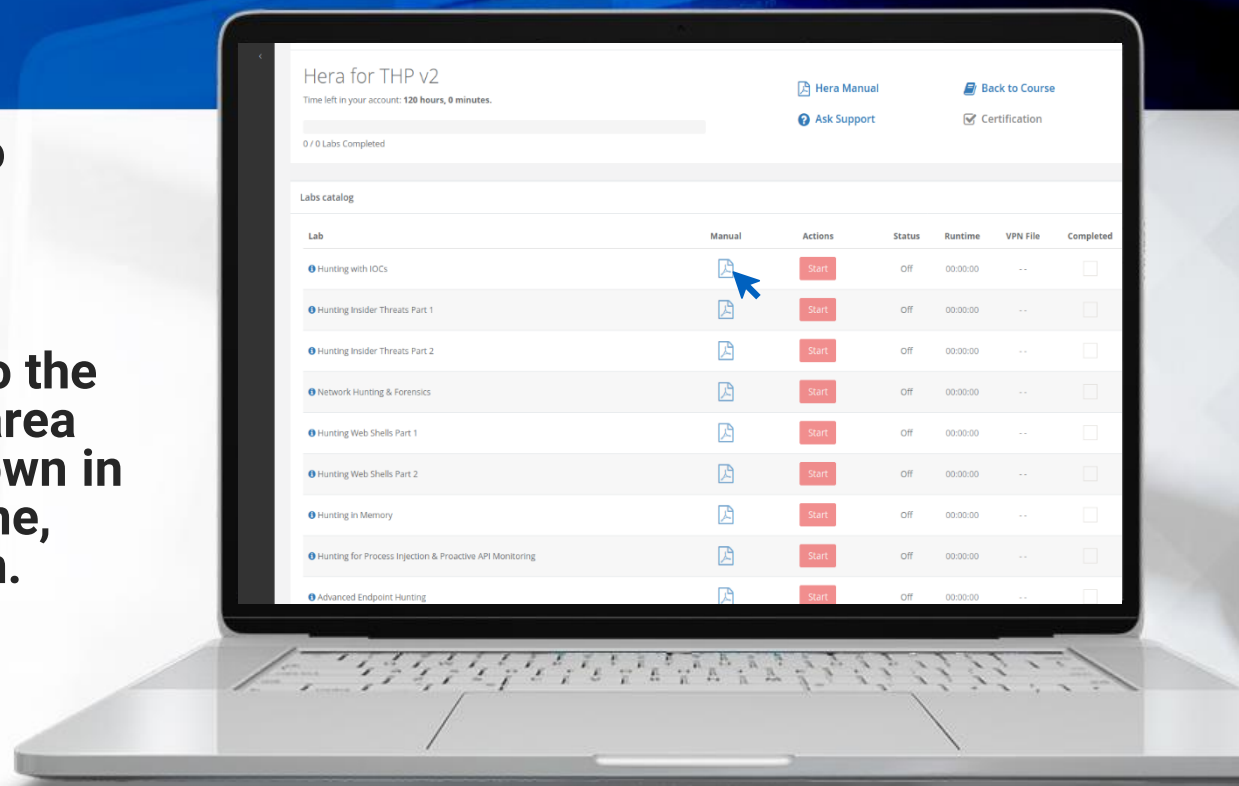
**\*NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

## 2.9.6 Lab #3

Put what you've learned to practice with the **Network Hunting & Forensics** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).



**\*NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

# Module Conclusion

This concludes this module on packet hunting. We have covered:

- Some of the core protocols and how to recognize what is normal and malicious network traffic.
- Unknown protocols, such as OSCAR.
- Extra tools to help us in our hunts.

# References



# References

## [ARP \(RFC 826\)](https://www.ietf.org/rfc/rfc826.txt)

<https://www.ietf.org/rfc/rfc826.txt>

## [Sample PCAPs](https://wiki.wireshark.org/SampleCaptures)

<https://wiki.wireshark.org/SampleCaptures>

## [ARP Display Filters](https://www.wireshark.org/docs/dfref/a/arp.html)

<https://www.wireshark.org/docs/dfref/a/arp.html>

## [ICMP \(RFC 792\)](https://tools.ietf.org/html/rfc792)

<https://tools.ietf.org/html/rfc792>



# References

## [ICMP Display Filters](https://www.wireshark.org/docs/dfref/i/icmp.html)

<https://www.wireshark.org/docs/dfref/i/icmp.html>

## [TCP \(RFC 793\)](https://tools.ietf.org/html/rfc793)

<https://tools.ietf.org/html/rfc793>

## [TCP Relative Sequence Numbers](https://wiki.wireshark.org/TCP_Relative_Sequence_Numbers)

[https://wiki.wireshark.org/TCP\\_Relative\\_Sequence\\_Numbers](https://wiki.wireshark.org/TCP_Relative_Sequence_Numbers)

## [NMAP Port Scanning Techniques](https://nmap.org/book/man-port-scanning-techniques.html)

<https://nmap.org/book/man-port-scanning-techniques.html>



# References

## [TCP Display Filters](https://www.wireshark.org/docs/dfref/t/tcp.html)

<https://www.wireshark.org/docs/dfref/t/tcp.html>

## [DHCP \(RFC 2131\)](https://www.ietf.org/rfc/rfc2131.txt)

<https://www.ietf.org/rfc/rfc2131.txt>

## [DHCP Display Filters](https://wiki.wireshark.org/DHCP)

<https://wiki.wireshark.org/DHCP>

## [DNS \(RFC 1034\)](https://www.ietf.org/rfc/rfc1034.txt)

<https://www.ietf.org/rfc/rfc1034.txt>



# References

## [DNS \(RFC 1035\)](https://www.ietf.org/rfc/rfc1035.txt)

<https://www.ietf.org/rfc/rfc1035.txt>

## [DNS Display Filters](https://www.wireshark.org/docs/dfref/d/dns.html)

<https://www.wireshark.org/docs/dfref/d/dns.html>

## [HTTP Methods \(RFC 2616\)](https://www.ietf.org/rfc/rfc2616.txt)

<https://www.ietf.org/rfc/rfc2616.txt>

## [HTTPS/SSL 3.0 \(RFC 6101\)](https://tools.ietf.org/html/rfc6101)

<https://tools.ietf.org/html/rfc6101>



# References

## [HTTPS/TLS 1.2 \(RFC 5246\)](https://tools.ietf.org/html/rfc5246)

<https://tools.ietf.org/html/rfc5246>

## [HTTPS/TLS 1.3](https://tools.ietf.org/html/draft-ietf-tls-tls13-18)

<https://tools.ietf.org/html/draft-ietf-tls-tls13-18>

## [Wireshark Wiki \(SSL\)](https://wiki.wireshark.org/SSL)

<https://wiki.wireshark.org/SSL>

## [Trend Labs \(CrypMIC Ransomware\)](http://blog.trendmicro.com/trendlabs-security-intelligence/crypmic-ransomware-wants-to-follow-cryptxxx/)

<http://blog.trendmicro.com/trendlabs-security-intelligence/crypmic-ransomware-wants-to-follow-cryptxxx/>



# References

## [Wireshark Wiki \(TCP Zero Window\)](https://wiki.wireshark.org/TCP%20ZeroWindow)

<https://wiki.wireshark.org/TCP%20ZeroWindow>

## [HTTP Display Filters 1](https://www.wireshark.org/docs/dfref/h/http.html)

<https://www.wireshark.org/docs/dfref/h/http.html>

## [HTTP Display Filters 2](https://www.wireshark.org/docs/dfref/h/http2.html)

<https://www.wireshark.org/docs/dfref/h/http2.html>

## [HTTPS Display Filters](https://www.wireshark.org/docs/dfref/s/ssl.html)

<https://www.wireshark.org/docs/dfref/s/ssl.html>



# References

## [Ann's Bad Aim PCAP](#)

<http://forensicscontest.com/2009/09/25/puzzle-1-anns-bad-aim>

## [On Sending Files via OSCAR \(PDF\)](#)

[http://www.cs.cmu.edu/~jhclark/aim/On Sending Files via OSCAR.pdf](http://www.cs.cmu.edu/~jhclark/aim/On%20Sending%20Files%20via%20OSCAR.pdf)

## [Wireshark "Decode As"](#)

[https://www.wireshark.org/docs/wsug\\_html\\_chunked/ChCustProtocolDissectionSection.html](https://www.wireshark.org/docs/wsug_html_chunked/ChCustProtocolDissectionSection.html)

## [Wireshark AIM Dissector](#)

[https://www.wireshark.org/docs/wsug\\_html\\_chunked/ChCustProtocolDissectionSection.html](https://www.wireshark.org/docs/wsug_html_chunked/ChCustProtocolDissectionSection.html)



# References

## [Wireshark Dissector \(Add\)](https://www.wireshark.org/docs/wsdg_html_chunked/ChDissectAdd.html)

[https://www.wireshark.org/docs/wsdg\\_html\\_chunked/ChDissectAdd.html](https://www.wireshark.org/docs/wsdg_html_chunked/ChDissectAdd.html)

## [NetworkMiner](http://www.netresec.com/?page=NetworkMiner)

<http://www.netresec.com/?page=NetworkMiner>

## [RSA NetWitness Investigator](https://community.rsa.com/community/products/netwitness/investigator)

<https://community.rsa.com/community/products/netwitness/investigator>

## [RSA NetWitness Investigator \(download\)](https://community.rsa.com/thread/188114)

<https://community.rsa.com/thread/188114>



# References

## [Wireshark Color codes](https://www.wireshark.org/docs/wsug_html_chunked/ChCustColorizationSection.html)

[https://www.wireshark.org/docs/wsug\\_html\\_chunked/ChCustColorizationSection.html](https://www.wireshark.org/docs/wsug_html_chunked/ChCustColorizationSection.html)

## [DNS Zone Transfer](https://www.acunetix.com/blog/articles/dns-zone-transfers-axfr/)

<https://www.acunetix.com/blog/articles/dns-zone-transfers-axfr/>

## [DNS Protocol](https://technet.microsoft.com/en-us/library/dd197470(v=ws.10).aspx)

[https://technet.microsoft.com/en-us/library/dd197470\(v=ws.10\).aspx](https://technet.microsoft.com/en-us/library/dd197470(v=ws.10).aspx)

## [AIM - AOL Instant Messenger](https://wiki.wireshark.org/AIM)

<https://wiki.wireshark.org/AIM>



# Videos

Here's a list of all videos in this module. To [ACCESS](#) your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).

**Wireshark**

**Network Miner**

**RSA Witness Investigator**

**Packet Hunting**





## Hunting Insider Threats Part 1

You are placed on a weekly hunting schedule. Using hypothesis-based hunting you want to hunt for insider threat activity. You decide to capture network traffic from various subnets. You are tasked to review some of the daily PCAP files.

## Hunting Insider Threats Part 2

You and Charles are aware that 2 rogue machines were on the network a few days ago. After speaking with management, you and Charles convince them to allow you to monitor the threats instead of totally eradicating them next time you spot them on the network because you need to learn more about these threat actors, who seem to be malicious insiders. You have several IOCs created and you are running full packet captures in order to see if there is any evidence of these maliciousness on the network. Now you're hunting based on intelligence but don't forget that an adversary can change their methods. Keep that in mind as you hunt.

***\*Labs are only available in Full or Elite Editions of the course. To [ACCESS](#) your labs, go to the course in your members area and click the labs drop-down in the appropriate module line. To UPGRADE to gain access, click [LINK](#).***



# Labs



## Network Hunting & Forensics

In this lab you will practice hunting for suspicious network connections and communications. Zeek, RITA and Wireshark will be used to analyze multiple malicious traffic samples.



*\*Labs are only available in Full or Elite Editions of the course. To [ACCESS](#) your labs, go to the course in your members area and click the labs drop-down in the appropriate module line. To UPGRADE to gain access, click [LINK](#).*